

PENERAPAN LIGHT GRADIENT BOOSTING MACHINE UNTUK PERAMALAN HARGA KAMAR BERBASIS DERET WAKTU

(Studi Kasus: PT Genius Utama Indonesia)

LAPORAN KERJA PRAKTIK



RAMA PANJI NARARENDRA CAHAYA

5231811033

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS & TEKNOLOGI
UNIVERSITAS TEKNOLOGI YOGYAKARTA
2026**

KATA PENGANTAR

Puji syukur dipanjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat-Nya, sehingga penyusunan proposal kerja praktik berjudul “Penerapan Light Gradient Boosting Machine untuk Peramalan Harga Kamar Berbasis Deret Waktu (Studi Kasus: PT Genius Utama Indonesia)” dapat diselesaikan dengan baik. Dokumen ini disusun untuk memenuhi sebagian persyaratan akademik guna memperoleh gelar Sarjana Sains Data pada Fakultas Sains dan Teknologi, Universitas Teknologi Yogyakarta.

Penyusunan naskah ini dapat terlaksana berkat dukungan, arahan, serta kemudahan dari berbagai pihak. Oleh karena itu, apresiasi dan ucapan terima kasih disampaikan kepada:

1. Dr. Bambang Moertono Setiawan, M.M., Akt., C.A., selaku Rektor Universitas Teknologi Yogyakarta atas kebijakan dan dukungan institusional yang diberikan.
2. Dr. Endy Marlina, S.T., M.T., selaku Dekan Fakultas Sains dan Teknologi, Universitas Teknologi Yogyakarta.
3. Sri Wulandari, S.Kom., M.Cs., selaku Ketua Program Studi Sains Data, Universitas Teknologi Yogyakarta.
4. Edi Rahmadi, selaku *Digital & Revenue* di PT Genius Utama Indonesia sekaligus pembimbing lapangan, atas penyediaan akses data serta bimbingan teknis selama proses perancangan sistem.
5. M. Kadri, S.T., dan Andrian Yosaredo, mewakili jajaran manajemen PT Genius Utama Indonesia, atas izin pelaksanaan kerja praktik.
6. Jajaran dosen dan tenaga kependidikan Program Studi Sains Data yang telah memberikan bekal keilmuan dan layanan administrasi selama masa perkuliahan.
7. Pihak keluarga yang senantiasa memberikan dukungan moril maupun materil selama masa studi hingga penyusunan naskah ini.
8. Rekan sejawat mahasiswa Program Studi Sains Data Universitas Teknologi

Yogyakarta atas diskusi teknis dan kerja sama yang membangun.

Disadari bahwa proposal ini masih memiliki keterbatasan, baik dari segi penyajian materi maupun kedalaman analisis. Oleh sebab itu, kritik dan saran yang konstruktif sangat diharapkan guna tahap penyempurnaan lebih lanjut. Harapannya, rancangan penelitian ini dapat memberikan kontribusi praktis bagi instansi terkait dan memperluas literatur terapan di bidang sains data.

Yogyakarta, 30 Januari 2026,

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	iii
DAFTAR TABEL	v
DAFTAR GAMBAR	vi
DAFTAR SINGKATAN	vii
DAFTAR ISTILAH	viii
DAFTAR PERSAMAAN	ix
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan Kerja Praktik	3
1.5 Manfaat Kerja Praktik	3
1.6 Sistemika Penulisan	4
BAB II KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI	6
2.1 Kajian Hasil Penelitian	6
2.2 Landasan Teori	9
2.2.1 Manajemen Pendapatan Hotel	10
2.2.2 <i>Time Series Forecasting</i>	12
2.2.3 Decision Tree	18
2.2.4 <i>Gradient Boosting</i>	32
2.2.5 <i>Light Gradient Boosting Machine (LightGBM)</i>	43
2.2.6 Evaluasi Kinerja Model	68
2.2.7 <i>Flowchart</i>	72
2.2.8 Optuna	75
BAB III TINJAUAN UMUM INSTANSI	80
3.1 Gambaran Umum	80
3.2 Struktur Organisasi	81
3.3 Jasa	83
3.4 Proses Bisnis	84
3.5 Deskripsi Data	85
BAB IV METODE KERJA PRAKTIK	89
4.1 Analisis Penelitian	89
4.2 Analisis Kebutuhan	91
4.2.1 Kebutuhan Fungsional	91
4.2.2 Kebutuhan Nonfungsional	92
4.2.3 Bahan Penelitian	92
4.3 Tahapan Penelitian	93
4.4 Perancangan Sistem	96

4.4.1 Alur Kerja Sistem.....	96
4.4.2 Perancangan Antarmuka.....	98
4.5 Perancangan Model.....	99
4.5.1 Pemilihan Model	99
4.5.2 Skenario Pembagian Data	100
4.5.3 Konfigurasi Parameter Pelatihan.....	100
4.5.4 Evaluasi Model.....	100
BAB V IMPLEMENTASI DAN PEMBAHASAN	102
5.1 Implementasi.....	102
5.1.1 Penyiapan Data dan Agregasi Bulanan	102
5.1.2 Pembentukan <i>Grid</i> Bulanan dan Pembersihan Data	103
5.1.3 Pembentukan Fitur	105
5.1.4 Pelatihan Model LightGBM.....	107
5.1.5 Implementasi <i>Retaining</i> Produksi dan Evaluasi.....	115
5.2 Pembahasan	116
5.2.1 Pengumpulan dan <i>Pre-processing</i> Data	116
5.2.2 Pembahasan Model	118
DAFTAR PUSTAKA	121
LAMPIRAN.....	123

DAFTAR TABEL

Tabel 2.1 Kajian Pustaka.....	8
Tabel 2.2 Data ARR.....	13
Tabel 2.3 Perhitungan Komponen OLS	15
Tabel 2.4 Hasil Perhitungan Tren dan Deviasi	16
Tabel 2.5 Dataset Peramalan.....	22
Tabel 2.6 Dataset <i>Node Kiri Level 1</i>	26
Tabel 2.7 Dataset <i>Node Kanan Level 1</i>	28
Tabel 2.8 Dataset <i>Node Kiri Level 3</i>	29
Tabel 2.9 Dataset <i>Node Kanan Level 2</i>	29
Tabel 2.10 Data <i>Node Kiri dan Kanan Sebelum Pruning</i>	30
Tabel 2.11 Dataset Untuk Inferensi	31
Tabel 2.12 Dataset Untuk <i>Gradient Boosting</i>	37
Tabel 2.13 Dataset Latih <i>Weak Learner</i>	38
Tabel 2.14 Dataset <i>Weak Learner Level 1 Node Kiri</i>	40
Tabel 2.15 Dataset <i>Weak Learner Level 1 Node Kanan</i>	40
Tabel 2.16 Dataset Untuk Prediksi <i>Weak Learner</i>	41
Tabel 2.17 Dataset Untuk Testing.....	42
Tabel 2.18 Dataset Untuk LightGBM.....	52
Tabel 2.19 <i>Dataset Sparse</i>	53
Tabel 2.20 Dataset Bundle 1	56
Tabel 2.21 Dataset Baru Untuk Data 1	56
Tabel 2.22 Dataset Baru Untuk Data 2	57
Tabel 2.23 Dataset Gabungan	57
Tabel 2.24 Data <i>Testing Numerik</i>	67
Tabel 2.25 Data <i>Testing Sparse</i>	67
Tabel 2.26 Data <i>Testing LightGBM</i>	67
Tabel 2.27 Dataset RMSE.....	70
Tabel 2.28 Dataset MAPE.....	71
Tabel 2.29 Simbol <i>Flowchart</i> dan Fungsinya.....	75
Tabel 2. 30 Riwayat Pendistribusian Uji Coba Pencarian <i>Hyperparameter</i>	77
Tabel 3.2 Sampel Dataset Tingkat Inflasi.....	87
Tabel 4.1 Perangkat Keras (Hardware) yang Digunakan.....	92
Tabel 4.2 Perangkat Lunak yang Digunakan	92
Tabel 5.2 Hasil Evaluasi	116
Tabel 5.3 Hasil Evaluasi	119

DAFTAR GAMBAR

Gambar 2.2 Sampel Data Transaksi.....	11
Gambar 2.3 <i>Decision Tree Level 0</i>	26
Gambar 2.4 <i>Decision Tree Node Kiri Level 1</i>	28
Gambar 2.5 <i>Decision Tree Node Kanan Level 1</i>	28
Gambar 2.6 <i>Decision Tree Node Kiri Level 2</i>	29
Gambar 2.7 <i>Decision Tree Node Kanan Level 2</i>	29
Gambar 2.8 <i>Decision Tree Akhir</i>	31
Gambar 2.9 <i>Arsitektur Gradient Boosting (Deng dkk., 2021)</i>	33
Gambar 2.10 <i>Weak Learner Root</i>	39
Gambar 2.11 <i>Weak Learner Level 1 Node Kiri</i>	40
Gambar 2.12 <i>Weak Learner Level 1 Node Kanan</i>	41
Gambar 2.13 <i>Weak Learner Akhir</i>	43
Gambar 2.14 <i>Arsitektur LightGBM (Bhupatiraju, 2025)</i>	47
Gambar 2.15 <i>Algoritma Greedy Bundling</i>	51
Gambar 2.16 <i>Algoritma Merge Exclusive Features</i>	52
Gambar 2.17 <i>Weak Learner Root</i>	64
Gambar 2.18 <i>Hasil Akhir Weak Learner</i>	66
Gambar 2.19 <i>Weak Learner Decision Tree</i>	68
Gambar 3.2 Kaluku Gili Resort – PT Genius Utama Indonesia.....	81
Gambar 3.3 Struktur Organisasi PT Genius Utama Indonesia.....	82
Gambar 3.4 Proses Bisnis Penetapan Harga Kamar.....	85
Gambar 3.5 <i>Sample Dataset</i>	87
Gambar 4.1 Alur Kerangka Penelitian.....	89

DAFTAR SINGKATAN

Singkatan	Kepanjangan
ADR	<i>Average Daily Rate</i>
API	<i>Application Programming Interface</i>
ARR	<i>Average Room Rate</i>
ARIMA	<i>AutoRegressive Integrated Moving</i>
EFB	<i>Exclusive Feature Bundling</i>
GHM	<i>Genius Hospitality Management</i>
GOSS	<i>Gradient-based One-Side Sampling</i>
KDD	<i>Knowledge Discovery in Database</i>
LightGBM	<i>Light Gradient Boosting Machine</i>
MAE	<i>Mean Absolute Error</i>
MAPE	<i>Mean Absolute Percentage Error</i>
MSE	<i>Mean Squared Error</i>
OTA	<i>Online Travel Agent</i>
PT	Perseroan Terbatas
RMSE	<i>Root Mean Squared Error</i>
XGBoost	<i>eXtreme Gradient Boosting</i>

DAFTAR ISTILAH

Istilah	Keterangan / Definisi
<i>Booking Channel</i>	Saluran atau platform pemesanan kamar hotel secara daring
<i>Data Cleaning</i>	Proses mendeteksi dan memperbaiki (atau menghapus) data yang rusak atau tidak akurat dari dataset.
<i>Dynamic Pricing</i>	Strategi penetapan harga fleksibel di mana harga disesuaikan secara <i>real-time</i> berdasarkan permintaan pasar.
<i>Feature Engineering</i>	Proses menggunakan pengetahuan domain untuk mengekstrak fitur baru dari data mentah guna meningkatkan kinerja algoritma machine learning.
<i>Forecasting</i>	Proses memproyeksikan nilai masa depan berdasarkan analisis data historis dan tren saat ini.
<i>High Season</i>	Periode waktu tertentu dalam satu tahun di mana permintaan terhadap akomodasi wisata meningkat signifikan.
<i>Lag</i>	Fitur dalam analisis deret waktu yang merepresentasikan nilai variabel pada periode waktu sebelumnya.
<i>Leaf-wise</i>	Strategi pertumbuhan pohon keputusan pada LightGBM yang memprioritaskan pembelahan pada daun dengan <i>loss</i> terbesar.
<i>Occupancy</i>	Tingkat hunian kamar hotel pada periode tertentu.
<i>Outlier</i>	Data pencilan atau nilai yang menyimpang jauh dari pola umum data lainnya.
<i>Overfitting</i>	Kondisi di mana model <i>machine learning</i> terlalu menyesuaikan diri dengan data latih sehingga performanya buruk pada data baru.
<i>Property ID</i>	Identitas unik berbentuk numerik yang mewakili satu unit properti atau hotel dalam sistem basis data.
<i>Revenue Management</i>	Penerapan analitik untuk memprediksi perilaku konsumen dan mengoptimalkan ketersediaan serta harga produk untuk memaksimalkan pendapatan.
<i>Rolling Window</i>	Teknik statistik yang menghitung metrik (seperti rata-rata) pada sekumpulan data dalam jendela waktu yang bergerak.
<i>Room ID</i>	Identitas unik berbentuk numerik yang mewakili tipe kamar spesifik dalam sebuah properti.
<i>Seasonality</i>	Pola berulang pada data deret waktu yang muncul secara periodik dalam interval waktu tertentu, seperti mingguan, bulanan, atau tahunan.
<i>Time Series</i>	Rangkaian data yang dikumpulkan, dicatat, atau diamati pada interval waktu yang berurutan.
<i>Trend</i>	Arah pergerakan jangka panjang pada data deret waktu yang menunjukkan kecenderungan meningkat, menurun, atau stabil.

DAFTAR PERSAMAAN

(2.1)	10
(2.2)	12
(2.3)	19
(2.4)	20
(2.5)	20
(2.6)	20
(2.7)	21
(2.8)	34
(2.9)	34
(2.10)	34
(2.11)	35
(2.12)	35
(2.13)	35
(2.14)	35
(2.15)	35
(2.16)	36
(2.17)	36
(2.18)	36
(2.19)	36
(2.20)	36
(2.21)	36
(2.22)	36
(2.23)	36
(2.24)	36
(2.25)	36
(2.26)	36
(2.27)	37
(2.28)	37
(2.29)	37
(2.30)	37
(2.31)	37
(2.32)	44
(2.33)	46
(2.34)	47
(2.35)	47
(2.36)	47
(2.37)	48
(2.38)	48
(2.39)	48
(2.40)	48
(2.41)	48
(2.42)	48
(2.43)	48

(2.44)	48
(2.45)	49
(2.46)	49
(2.47)	49
(2.48)	49
(2.49)	49
(2.50)	49
(2.51)	49
(2.52)	50
(2.53)	50
(2.54)	50
(2.55)	69
(2.56)	71
(2.57)	76
(2.58)	76

BAB I

PENDAHULUAN

1.1 Latar Belakang

Ketepatan manajemen dalam menyesuaikan harga sewa kamar secara dinamis itu merupakan hal yang diperlukan industri perhotelan modern (Talón-Ballesteros dkk., 2022). Untuk memaksimalkan pendapat dan menjaga daya saing pasar kemampuan meramalkan rata-rata harga kamar atau *Average Room Rate* (ARR) secara akurat bisa menjadi salah satu faktor penentu bagi kelangsungan bisnis di industri perhotelan modern. Praktik pengelolaan pendapatan hotel mensyaratkan analisis data historis yang sangat komprehensif. Oleh karena itu, pemanfaatan teknik *machine learning* dapat digunakan untuk membaca pola permintaan dan merumuskan strategi harga yang jauh lebih presisi dibandingkan metode konvensional.

Tantangan penetapan rata-rata harga kamar juga dialami oleh PT Genius Utama Indonesia (Genius Hospitality Management), sebagai perusahaan konsultan manajemen yang menaungi berbagai properti di wilayah kepulauan Nusa Tenggara Barat, salah satunya Kaluku Gili Resort. Dalam kegiatan operasionalnya, perusahaan mengelola inventaris kamar yang dipasarkan melalui berbagai saluran distribusi, mulai dari pemesanan langsung (*direct booking*) melalui telepon hingga penggunaan *Online Travel Agent* (OTA) seperti Agoda dan Booking.com. Kompleksitas mulai muncul ketika manajemen harus memproyeksikan harga optimal untuk setiap tipe kamar (*room_id*) di berbagai properti (*property_id*) yang memiliki karakteristik berbeda-beda. Proses analisis manual terhadap arus data transaksi dari beragam saluran dirasa tidak efisien, mengingat tingginya volume data serta fluktuasi pola musiman pariwisata yang berubah dengan cepat.

Kendala teknis lain yang memengaruhi operasional perusahaan adalah inkonsistensi kualitas data dari berbagai saluran pemesanan. Pengamatan awal pada data internal menunjukkan banyaknya rekam data harga sewa kamar yang kosong pada sebagian saluran. Untuk menyiasati hal tersebut, penggunaan data difokuskan secara khusus pada transaksi dari saluran Booking.com yang dinilai memiliki

kelengkapan historis paling baik. Meskipun demikian, data transaksi internal saja dirasa belum cukup untuk memproyeksikan harga secara komprehensif. Daya beli pasar dan fluktuasi ekonomi kemungkinan besar memiliki dampak signifikan terhadap rata-rata harga kamar. Oleh karena itu, data makroekonomi berupa tingkat inflasi bulanan dari Bank Indonesia perlu diintegrasikan. Selain itu, atribut fisik berupa kuantitas kamar (*roomqty*) juga ditambahkan untuk melengkapi pemahaman algoritma terhadap kapasitas masing-masing properti.

Berdasarkan serangkaian pertimbangan tersebut, penelitian kerja praktik ini disusun untuk merancang sistem peramalan rata-rata harga kamar bulanan. Sistem ini mengimplementasikan kerangka kerja 12 model LightGBM terpisah yang dirancang menggunakan pendekatan *direct multi-step forecasting* untuk memprediksi cakrawala waktu hingga 12 bulan ke depan. Proyeksi tren harga ini dibuat spesifik untuk masing-masing tipe kamar, sehingga manajemen dapat memanfaatkannya sebagai landasan taktis dalam mengeksekusi strategi harga dinamis.

1.2 Rumusan Masalah

Berdasarkan uraian pada latar belakang, persoalan utama yang akan diselesaikan melalui kerja praktik adalah bagaimana algoritma *Light Gradient Boosting Machine* (LightGBM) dapat diimplementasikan untuk meramalkan rata-rata harga kamar bulanan secara spesifik berdasarkan tipe kamar (*room_id*) dan lokasi properti (*property_id*) di PT Genius Utama Indonesia.

1.3 Batasan Masalah

Berikut merupakan batasan masalah dari penelitian kerja praktik agar pembahasan dalam kerja praktik tidak terlalu melebar dan tetap berpegang pada tujuan awal, diantaranya:

1. Data yang diproses merupakan gabungan dari rekaman transaksi historis pemesanan kamar milik PT Genius Utama Indonesia (periode kedatangan 11 Juni 2022 hingga 27 Desember 2026) serta data makroekonomi publik berupa tingkat inflasi bulanan dari Bank Indonesia.
2. Sumber pengumpulan data dibatasi secara eksklusif pada satu saluran

pemasaran daring saja, yaitu Booking.com. Keputusan pembatasan sumber pengumpulan data diambil untuk menjaga kelengkapan dan konsistensi kualitas data yang kerap menjadi kendala pada saluran lain.

3. Fokus prediksi dilakukan pada level properti dengan mempertimbangkan identitas properti (*property_id*) dan tipe kamar (*room_id*).
4. Variabel target yang menjadi acuan keluaran peramalan adalah rata-rata harga kamar bulanan (*average room rate*).
5. Sistem peramalan memproyeksikan harga selama 12 bulan ke depan menggunakan pendekatan 12 model prediktif yang berjalan secara independen untuk masing-masing cakrawala waktu.

1.4 Tujuan Kerja Praktik

Berdasarkan dengan rumusan masalah yang telah diidentifikasi pelaksanaan kerja praktik disusun untuk mencapai beberapa target berikut:

1. Membangun 12 model peramalan secara independen menggunakan algoritma LightGBM untuk memproyeksikan harga kamar bulanan. Model ini dirancang agar mampu memproses integrasi fitur kompleks, seperti identitas properti, tipe kamar, kuantitas kamar, serta tingkat inflasi.
2. Menghasilkan estimasi harga kamar yang mendekati akurat untuk periode waktu ke depan pada berbagai tipe kamar di bawah pengelolaan PT Genius Utama Indonesia.
3. Mengukur tingkat keandalan dan metrik kesalahan model LightGBM saat berhadapan dengan karakteristik data deret waktu dari riwayat pemesanan di Booking.com.

1.5 Manfaat Kerja Praktik

Penelitian yang dilakukan diharapkan memberikan kontribusi nyata kepada instansi tempat studi kasus dilakukan melalui hasil kerja praktik. Rincian manfaat yang kemungkinan besar bisa diperoleh oleh PT Genius Utama Indonesia meliputi:

1. Menyediakan landasan analitis bagi manajemen PT Genius Utama Indonesia untuk mengambil keputusan penentuan harga dinamis kamar hotel.

2. Membantu tim bagian *Revenue* di perusahaan agar lebih mudah memetakan proyeksi harga rata-rata pada setiap tipe kamar di masa mendatang, yang dapat digunakan untuk menyusun langkah penjualan yang lebih tajam dan akurat untuk memaksimalkan profitabilitas.
3. Mengurangi beban analisis tren harga secara manual yang memakan waktu, mengingat sistem yang dibangun diharapkan mampu memproses data historis yang bervariasi secara lebih terotomatisasi.

1.6 Sistemika Penulisan

Guna memudahkan pembaca dalam menelusuri alur pemikiran dan gambaran utuh dari laporan kerja praktik, materi pembahasan disusun berdasarkan sistematika berikut:

BAB I PENDAHULUAN

Bab I menguraikan dasar pemikiran mengapa kerja praktik dilaksanakan. Uraian dalam Bab I mencakup latar belakang permasalahan, perumusan masalah, batasan penelitian agar penelitian terarah, hingga tujuan dan manfaat yang bisa didapatkan oleh instansi, lalu ditutup dengan sistematika penyusunan laporan.

BAB II KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI

Bab II berisi ulasan ringkas mengenai beberapa penelitian terdahulu yang relevan sebagai acuan. Selain itu, bab II juga membedah teori-teori dasar yang menopang penyelesaian masalah, mulai dari konsep peramalan data deret waktu hingga prinsip kerja dari algoritma LightGBM yang digunakan dalam pembangunan model.

BAB III TINJAUAN UMUM INSTANSI

Bab III membahas mengenai instansi tempat pelaksanaan kerja praktik, yaitu PT Genius Utama Indonesia. Pembahasan pada bab III meliputi gambaran umum perusahaan, struktur organisasi, serta produk dan jasa yang dihasilkan oleh perusahaan. Penjelasan tersebut bertujuan untuk memberikan pemahaman mengenai profil perusahaan serta kegiatan operasional yang berkaitan dengan topik kerja praktik.

BAB IV ANALISIS DAN PERANCANGAN

Bab IV Analisis dan Perancangan menguraikan kerangka metodologis serta pendekatan teknis yang diadopsi untuk menangani data reservasi dari saluran Booking.com. Bab IV merinci tahapan eksplorasi data, mulai dari penanganan ketidakkonsistenan *dataset* hingga strategi pra-pemrosesan yang diperlukan untuk membentuk variabel prediktor yang valid. Selain itu, arsitektur model *Light Gradient Boosting Machine* (LightGBM) yang dirancang untuk memprediksi rata-rata harga kamar berdasarkan *property_id* dan *room_id* dipaparkan secara struktural, termasuk skema pembagian data latih dan uji yang digunakan.

BAB V IMPLEMENTASI DAN PEMBAHASAN

Bab V Implementasi dan Pembahasan mendemonstrasikan realisasi dari rancangan model *forecasting* yang telah disusun. Fokus utama bab V terletak pada proses pelatihan model terhadap data historis serta evaluasi kinerjanya menggunakan metrik pengukuran yang relevan untuk mengukur tingkat akurasi prediksi. Interpretasi terhadap hasil prediksi disajikan secara mendalam untuk melihat kemampuan model dalam menangkap pola fluktuasi harga di berbagai tipe kamar, disertai diskusi mengenai faktor-faktor yang kemungkinan memengaruhi deviasi antara nilai prediksi dan kondisi aktual di lapangan.

BAB VI PENUTUP

Bab VI Penutup merangkum intisari temuan yang diperoleh selama pelaksanaan kerja praktik, sekaligus menjawab tujuan penelitian yang telah ditetapkan di awal. Bagian akhir, bab VI turut menyumbangkan sejumlah saran konstruktif, baik untuk pengembangan teknis model prediksi di masa mendatang maupun rekomendasi operasional bagi Genius Hospitality Management dalam memanfaatkan wawasan data yang dihasilkan.

BAB II

KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI

2.1 Kajian Hasil Penelitian

Penerapan algoritma *machine learning* untuk memprediksi harga dan meramalkan deret waktu telah dieksplorasi pada berbagai sektor industri. Tinjauan terhadap beberapa studi sebelumnya diperlukan untuk memetakan posisi kerja praktik, sekaligus melihat potensi metode *Light Gradient Boosting Machine* (LightGBM) saat dihadapkan pada karakteristik data pemesanan kamar hotel.

Pada penelitian Astarani dkk. (2025), dilakukan uji komparasi antara algoritma XGBoost dan LightGBM untuk prediksi penjualan ritel menggunakan dataset Walmart. Hasil pengujian menunjukkan bahwa LightGBM mencatat tingkat kesalahan yang lebih rendah, dengan *Mean Squared Error* (MSE) sebesar 0,0341 dan *Root Mean Squared Error* (RMSE) sebesar 0,1847. Kapasitas LightGBM dalam memproses data berskala besar mengindikasikan kecocokannya untuk menangani data transaksi log perhotelan.

Penelitian oleh Rahman dkk. (2025) menggunakan algoritma XGBoost untuk mengestimasi harga kos di Samarinda berdasarkan fitur fasilitas dan lokasi. Meskipun model tersebut menghasilkan *Mean Absolute Percentage Error* (MAPE) sekitar 18%, studi tersebut menunjukkan bahwa pemilihan atribut yang relevan sangat menentukan akurasi regresi. Temuan tersebut mendukung rencana penggunaan atribut pembeda spesifik seperti identitas properti (*property_id*) dan identitas kamar (*room_id*) pada pemodelan yang akan dibangun.

Penelitian Pramita dkk. (2026) telah berhasil memprediksi permintaan hotel menggunakan metode *Random Forest Regression*. Model yang dikembangkan menunjukkan performa yang sangat baik, dengan nilai R-squared mencapai 99,83% pada data uji, 99,95% pada data latih, serta 88,24% pada prediksi tiga bulan ke depan, disertai nilai MAE, MSE, dan RMSE yang relatif rendah. Hasil tersebut menunjukkan bahwa *Random Forest* mampu menangkap variasi permintaan hotel secara efektif, terutama pada tipe kamar dengan pola permintaan yang stabil. Berdasarkan analisis studi, variabel internal hotel seperti tingkat hunian, jumlah

reservasi, dan pembatalan memiliki kontribusi signifikan terhadap akurasi peramalan. Pendekatan berbasis data historis internal sejalan dengan arah kerja praktik yang menitikberatkan pada riwayat pemesanan aktual sebagai dasar dalam penentuan harga.

Dari aspek optimasi komputasi, studi Aulady dkk. (2024) mengimplementasikan LightGBM dengan optimasi *Hyper-Parameter Tuning* melalui *GridSearchCV* pada kasus regresi harga mobil bekas. Pendekatan tersebut terbukti efisien dalam mempercepat waktu pelatihan sekaligus menjaga performa, dibuktikan dengan nilai *R-Square* 0,89 dan MSE 0,034. Karakteristik algoritma yang efisien tersebut sangat relevan untuk diterapkan saat memproses ragam tipe kamar hotel secara simultan.

Pada sisi pemodelan statistik konvensional, Pujiningrum dan Sulistijanti (2024) meramalkan tingkat hunian hotel bintang lima di Bali menggunakan metode ARIMA dan *Fuzzy Time Series*. Walaupun model ARIMA (1,1,1) mampu mencatat kesalahan di bawah 10% untuk data yang pola musimannya stabil, pendekatan statistik biasa kerap menemui hambatan pada data yang fluktuasinya tidak teratur. Senada dengan hal tersebut, studi Herlambang dan Azies (2026) yang membandingkan ARIMA dan Prophet menyimpulkan bahwa metode tradisional umumnya hanya optimal untuk jangka pendek. Pada deret waktu yang lebih dinamis dan kompleks, algoritma non-linier modern lebih diperlukan untuk menangkap variabilitas perubahan tren.

Berdasarkan tinjauan literatur tersebut, pemodelan berbasis *gradient boosting* seperti LightGBM terbukti tangguh untuk kasus regresi pada set data berskala besar. Namun demikian, terlihat adanya celah penelitian (*research gap*) pada implementasinya di industri perhotelan. Mayoritas studi baru sebatas memprediksi tingkat permintaan umum atau estimasi harga properti secara global. Belum banyak model peramalan yang diarahkan untuk memprediksi *Average Room Rate* (ARR) secara terperinci hingga ke level identitas tipe kamar (*room_id*) per lokasi hotel (*property_id*). Oleh karena itu, kerja praktik difokuskan untuk mengisi batasan tersebut. Dengan memanfaatkan data historis tunggal dari saluran Booking.com demi menjaga konsistensi fitur, luaran dari model peramalan

ditujukan murni sebagai solusi taktis bagi PT Genius Utama Indonesia. Hasil proyeksi harga model ini akan langsung digunakan oleh manajemen instansi sebagai dasar validasi untuk menetapkan strategi harga dinamis (*dynamic pricing*) yang lebih akurat per tipe kamarnya.

Tabel 2.1 Kajian Pustaka

No	Judul Penelitian	Peneliti	Metode	Hasil
1	Analisis Perbandingan XGBoost dan LightGBM dalam Prediksi Penjualan Ritel Walmart Store Sales	I Gusti Ayu Riyani Astarani dan I Gede Surya Rahayuda (2025)	XGBoost, LightGBM	LightGBM lebih unggul dari XGBoost pada data besar (MSE 0,0341; RMSE 0,1847).
2	Penerapan Algoritma XGBoost Dalam Prediksi Harga Sewa Kos Di Kota Samarinda	Amalia Rahman, Amelia Yusnita, dan Hanifah Ekawati (2025)	XGBoost	MAPE ~18%; atribut relevan meningkatkan akurasi regresi.
3	<i>Forecasting Hotel Demand with Time Series Prediction Model Using Random Forest Regression</i>	Dewa Ayu Kadek Pramita, Ni Wayan Sumartini Saraswati, I Putu Dedy Sandana, Dewa Ayu Putu Rasmika Dewi, dan Ni Kadek Bumi Krismentari (2026)	<i>Random Forest Regression</i>	Model menunjukkan akurasi tinggi (R^2 99,83% pada data uji; 88,24% untuk prediksi tiga bulan), dengan variabel internal berkontribusi signifikan terhadap hasil peramalan.
4	<i>Improve Metode LightGBM untuk Prediksi Harga Mobil Bekas Menggunakan Hyper-Parameter Tuning</i>	Moch. Aqil Aulady, Ahmad Hudawi AS., dan Zainal Arifin (2024)	LightGBM, <i>Hyper-Parameter Tuning</i> (GridSearchCV)	<i>Hyper-parameter tuning</i> efisien tanpa menurunkan kinerja (R^2 0,89; MSE 0,034).

Tabel 2.1 Kajian Pustaka (lanjutan)

No	Judul Penelitian	Peneliti	Metode	Hasil
5	Peramalan Tingkat Penghunian Kamar Hotel Bintang Lima Provinsi Bali Menggunakan Metode ARIMA dan <i>Fuzzy Time Series</i> Lee	Rahayu Dwiastuti Pujiningrum dan Wellie Sulistijanti (2024)	ARIMA, <i>Fuzzy Time Series</i> Lee	ARIMA (1,1,1) <10% <i>error</i> pada musiman stabil; lemah pada fluktuasi tak teratur.
6	<i>Time Series Forecasting</i> Saham PT Astra Menggunakan Algoritma <i>Autoregressive Integrated Moving Average</i> dan <i>Prophet</i>	Naufal Malik Herlambang dan Harun Al Azies (2026)	ARIMA, <i>Prophet</i>	Metode tradisional optimal jangka pendek; non-linier untuk data kompleks.

Berdasarkan rangkuman kajian pustaka pada Tabel 2.1, mayoritas penerapan algoritma pembelajaran mesin maupun metode statistik konvensional masih difokuskan pada prediksi tingkat permintaan umum, estimasi tarif properti secara global, serta peramalan penjualan ritel. Celah penelitian terlihat dari belum banyaknya pemodelan yang meramalkan tarif kamar rata-rata secara terperinci hingga mencapai level identitas tipe kamar pada masing-masing identitas properti perhotelan. Pelaksanaan kerja praktik dirancang guna mengisi batasan kajian melalui pengembangan model Light Gradient Boosting Machine untuk memproyeksikan harga per tipe kamar pada sejumlah properti hotel kelolaan Genius Hospitality Management. Guna menjaga tingkat konsistensi dan kelengkapan atribut, ekstraksi data historis dibatasi secara eksklusif pada rekam jejak transaksi dari saluran Booking.com. Hasil proyeksi pemodelan ditujukan secara langsung sebagai solusi taktis bagi PT Genius Utama Indonesia, sehingga pihak manajemen instansi memperoleh landasan validasi objektif dalam menyusun strategi harga dinamis secara lebih akurat pada masing-masing tipe kamar.

2.2 Landasan Teori

Berikut akan dijelaskan mengenai teori-teori yang menjadi landasan dalam penelitian ini, diawali dengan konsep manajemen pendapatan hotel.

2.2.1 Manajemen Pendapatan Hotel

Praktik manajemen pendapatan di sektor perhotelan biasanya berpusat pada melaraskan antara ketersediaan aset dengan perubahan permintaan konsumen. Pendekatan ini tidak sekadar tentang mengatur tarif dasar penginapan, melainkan lebih jauh melibatkan strategi penetapan harga dinamis (*dynamic pricing*) yang dapat merespons berbagai faktor.

Beberapa faktor yang biasanya memengaruhi penyesuaian harga ini meliputi tingkat hunian saat ini (*occupancy*), tren pemesanan dari saluran daring, hingga siklus musiman pariwisata seperti periode sepi maupun puncak liburan (Santos dkk., 2024). Karakteristik kamar hotel sebagai produk jasa yang tidak dapat disimpan atau dijual kembali pada hari berikutnya menyebabkan kegagalan dalam menyewakan kamar pada suatu malam berpotensi memicu hilangnya pemasukan secara permanen bagi manajemen. Oleh karena itu, pengelola properti bisa mendapatkan manfaat dari analisis riwayat data transaksi untuk memproyeksikan harga yang paling ideal. Pihak manajemen kemudian dapat menyasar segmen tamu yang tepat pada waktu yang paling menguntungkan proyeksi harga tersebut.

Dalam industri perhotelan bisa menggunakan indikator rata-rata harga kamar atau juga dikenal *Average Room Rate* (ARR) untuk mengevaluasi seberapa efektif strategi penetapan harga dinamis. Sebagian literatur perhotelan juga sering menyebut metrik yang sama dengan istilah *Average Daily Rate* (ADR). Indikator ini merepresentasikan rata-rata uang yang didapat dari setiap unit kamar yang berhasil disewakan dalam kurun waktu tertentu. Dalam kajian prediksi berbasis deret waktu, nilai ARR kerap kali diposisikan sebagai variabel target peramalan utama karena kemampuannya memotret realitas harga transaksi di lapangan. Secara matematis, besaran nilai ARR diperoleh melalui pembagian antara total pendapatan dari penjualan kamar dengan jumlah unit yang laku terjual (Darma dan Wirawati, 2019).

Bentuk perhitungan tersebut dijabarkan lebih lanjut pada Persamaan 2.1.

$$ARR = \frac{\text{Total Pendapatan Kamar}}{\text{Total Kamar Terjual}} \quad (2.1)$$

Pemahaman mengenai fluktuasi ARR ini nantinya akan menjadi landasan penting dalam mengoptimalkan algoritma peramalan untuk memproyeksikan harga sewa yang paling kompetitif pada masa mendatang.

Contoh 2.1

	A	B	C	D
1	propertyId	roomId	arrival	price
2	201589	431071	2023-12-31	450000
3	158990	462634	2023-12-31	872370
4	163277	359142	2023-12-31	800000
5	158990	350464	2023-12-31	2634120
6	158990	462632	2023-12-31	1858788
7	158986	350455	2023-12-31	375000
8	169082	370150	2023-12-30	174545
9	169082	370150	2023-12-30	174545
10	194842	419126	2023-12-30	4374000
11	193394	416398	2023-12-31	1458000
12	194837	419105	2023-12-30	4851000
13	201589	431071	2023-12-31	900000
14	179195	388812	2023-12-31	5187000
15	179195	388810	2023-12-31	4719000
16	193394	416398	2023-12-31	1000000

Gambar 2.1 Sampel Data Transaksi

Gambar di atas menampilkan sebagian kecil sampel dari kumpulan data transaksi. Melalui perhitungan agregasi terhadap keseluruhan dataset transaksi kamar pada bulan Juni 2022, diperoleh hasil sebagai berikut:

1. Total Pendapatan Kamar = 750.776.200.000 Rupiah
2. Total Kamar Terjual = 793 kamar

Average Room Rate (ARR) dihitung dengan membagi total pendapatan kamar dengan jumlah kamar yang terjual, sesuai dengan Persamaan 2.1 diperoleh: Masukkan nilai ke dalam Persamaan 2.1:

$$ARR = \frac{750.776.200.000}{793}$$

$$ARR \approx \text{Rp } 946.754 \text{ per kamar per malam}$$

Nilai ARR sebesar Rp 946.754 menunjukkan bahwa rata-rata pendapatan yang diperoleh dari setiap kamar yang terjual pada bulan Juni 2022 adalah sekitar Rp 946.754 per malam.

Perhitungan ini menunjukkan bagaimana ARR dapat digunakan sebagai indikator performa pendapatan kamar setiap bulan. Metode ini juga dapat diterapkan pada bulan-bulan berikutnya dengan cara yang sama, hanya mengganti nilai total pendapatan dan kamar terjual sesuai data masing-masing bulan.

2.2.2 Time Series Forecasting

Deret waktu (*time series*) merupakan sekumpulan hasil pengamatan terhadap suatu variabel yang dicatat secara berurutan berdasarkan waktu (Montgomery dan Jennings, 2024). Proses peramalan pada data semacam ini pada intinya berupaya mengekstrak pola dari riwayat masa lalu untuk memproyeksikan angka di masa depan (Truong dan Novák, 2023). Khususnya pada peramalan kuantitatif jangka pendek hingga menengah, keberhasilannya sangat bergantung pada seberapa baik pola historis tersebut diidentifikasi dan diekstrapolasi. Bagi manajemen perhotelan, informasi proyeksi ini sering kali menjadi acuan krusial dalam menyusun perencanaan pendapatan operasional.

Secara umum, struktur data deret waktu diasumsikan terbentuk dari empat komponen utama. Keempat komponen tersebut meliputi tren yang memperlihatkan arah pergerakan jangka panjang, siklus, pola musiman yang merepresentasikan naik-turunnya data pada rentang waktu berulang, serta gangguan acak (*noise*) yang sifatnya tidak terstruktur (Truong dan Novák, 2023). Sektor perhotelan menunjukkan pola musiman dan tren yang jelas, di mana permintaan serta harga kamar mengalami fluktuasi yang cenderung mengikuti pola tertentu seperti libur panjang atau akhir pekan. Dekomposisi deret waktu secara matematis dapat dirumuskan pada Persamaan 2.2 (Truong dan Novák, 2023).

$$X(t) = Tr(t) + C(t) + S(t) + R(t) \quad (2.2)$$

Keterangan:

$X(t)$ = Nilai deret waktu pada saat t

$Tr(t)$ = Komponen tren (*trend*)

$C(t)$ = Komponen siklus (*cycle*)

$S(t)$ = Komponen musiman (*seasonality*)

$R(t)$ = Komponen acak (*random noise*)

Penerapan algoritma *machine learning* pada deret waktu tidak bisa dilakukan begitu saja. Data mentah perlu melalui tahap transformasi agar model dapat mengenali sifat autokorelasi, yakni kecenderungan di mana suatu nilai pada waktu tertentu dipengaruhi oleh rentetan nilai sebelumnya (Montgomery dan Jennings, 2024). Agar data ini dapat diproses dalam kerangka *supervised learning*, penerapan rekayasa fitur (*feature engineering*) mutlak diperlukan. Beberapa teknik yang lazim dipakai adalah pembuatan variabel *lag*, yang menggeser observasi masa lalu menjadi fitur masukan baru, serta rata-rata bergerak (*rolling window*) guna meratakan fluktuasi data jangka pendek.

Melalui pendekatan rekayasa fitur tersebut, algoritma prediktif dapat mempelajari riwayat pergerakan harga kamar secara lebih spesifik. Model pada akhirnya mampu menangkap pola historis unik dari tiap identitas properti (*property_id*) maupun tipe kamar (*room_id*), sehingga proyeksi rata-rata harga bulanan yang dihasilkan bisa lebih presisi dan dapat diandalkan untuk strategi penetapan harga dinamis.

Contoh 2.2

Sebagai ilustrasi perhitungan dekomposisi deret waktu, digunakan data *Average Room Rate* (ARR) di sejumlah properti kelolaan PT Genius Utama Indonesia pada periode bulanan dari Juni 2022 hingga Mei 2023 sebagaimana disajikan pada Tabel 2.2 sebagai berikut:

Tabel 2.2 Data ARR

Tahun	Bulan	Total Pendapatan Kamar (Rp)	Total Kamar Terjual (kamar)	ARR (Rp)
2022	Juni	750.776.200.000	793	946.754

Tabel 2.2 Data ARR (lanjutan)

Tahun	Bulan	Total Pendapatan Kamar (Rp)	Total Kamar Terjual (kamar)	ARR (Rp)
2022	Juli	2.193.926.000.000	1.716	1.278.512
2022	Agustus	8.304.128.000.000	6.032	1.376.679
2022	September	7.042.278.000.000	4.875	1.444.570
2022	Oktober	6.234.973.000.000	4.563	1.366.420
2022	November	4.787.362.000.000	3.601	1.329.453
2022	Desember	7.011.261.000.000	3.952	1.774.105
2023	Januari	6.333.431.000.000	3.627	1.746.190
2023	Februari	5.528.212.000.000	3.055	1.809.562
2023	Maret	7.399.731.000.000	4.485	1.649.884
2023	April	9.900.916.000.000	5.694	1.738.833
2023	Mei	15.789.140.000.000	10.218	1.545.228

Setiap nilai ARR pada bulan ke- t dinotasikan sebagai $X(t)$, sehingga rentang observasi yang tersedia adalah $t = 1, 2, \dots, 12$, berturut-turut mewakili periode Juni 2022 hingga Mei 2023. Berdasarkan Persamaan 2.2, nilai deret waktu pada setiap bulan dapat diurai menjadi:

$$X(t) = Tr(t) + C(t) + S(t) + R(t)$$

Dalam konteks data ini, komponen siklus $C(t)$ diasumsikan bersifat konstan dalam jangka pendek sehingga tidak dapat diidentifikasi secara terpisah dari komponen musiman pada cakupan data 12 bulan. Oleh karena itu, pada ilustrasi ini $C(t)$ dimasukkan ke dalam residu gabungan bersama $S(t)$ dan $R(t)$. Fokus perhitungan diarahkan pada estimasi komponen tren $Tr(t)$ menggunakan metode regresi linear kuadrat terkecil *Ordinary Least Squares* (OLS), kemudian menghitung deviasi tiap observasi terhadap garis tren tersebut sebagai estimasi gabungan $S(t) + C(t) + R(t)$.

Pada tahap awal lakukan penomoran periode dan statistik dasar. Setiap bulan observasi diberi indeks t secara berurutan mulai dari $t = 1$ (Juni 2022) hingga $t = 12$ (Mei 2023). Berdasarkan data pada Tabel 2.2, diperoleh statistik ringkas sebagai berikut:

Tabel 2.3 Perhitungan Komponen OLS

t	x	t^2	tx
1	946.754	1	946.754
2	1.278.512	4	2.557.024
3	1.376.679	9	4.130.037
4	1.444.570	16	5.778.280
5	1.366.420	25	6.832.100
6	1.329.453	36	7.976.718
7	1.774.105	49	12.418.735
8	1.746.190	64	13.969.520
9	1.809.562	81	16.286.058
10	1.649.884	100	16.498.840
11	1.738.833	121	19.127.163
12	1.545.228	144	18.542.736
$\Sigma t = 78$	$\Sigma x = 18.006.190$	$\Sigma t^2 = 650$	$\Sigma tx = 125.063.965$

Berdasarkan hasil perhitungan tersebut, diperoleh nilai keluaran sebagai berikut:

$$\begin{aligned}
 \bar{t} &= \frac{78}{12} \\
 &= 6,5 \\
 \bar{x} &= \frac{18.006.190}{12} \\
 &= 1.500.515,83
 \end{aligned}$$

Selanjutnya, estimasi komponen tren dilakukan dengan regresi linier. Komponen tren $Tr(t)$ diestimasi menggunakan persamaan garis lurus $Tr(t) = \beta_0 + \beta_1 t$. Koefisien kemiringan β_1 dan intersep β_0 diperoleh melalui rumus OLS berikut:

$$\begin{aligned}
 \beta_1 &= \frac{\Sigma tx - n\bar{t} \cdot \bar{x}}{\Sigma t^2 - n\bar{t}^2} \\
 \beta_1 &= \frac{125.063.965 - 12 \times 6,5 \times 1.500.515,83}{650 - 12 \times 6,5^2} \\
 \beta_1 &= \frac{125.063.965 - 117.040.235}{650 - 507}
 \end{aligned}$$

$$\beta_1 = \frac{8.023.730}{143} \approx 56.110$$

$$\beta_0 = \bar{x} - \beta_1 \bar{t} = 1.500.515,83 - 56.110 \times 6,5 = 1.135.800,83$$

Dengan demikian, persamaan tren yang diperoleh adalah:

$$Tr(t) = 1.135.800,83 + 56.110 \times t$$

Nilai $\beta_1 = 56.110$ mengindikasikan bahwa secara rata-rata, ARR mengalami kenaikan sebesar Rp 56.110 setiap bulannya sepanjang periode observasi. Hal ini mencerminkan adanya tren jangka panjang yang cenderung meningkat pada tarif kamar.

Selanjutnya, dihitung nilai tren dan deviasi untuk seluruh periode. Tabel berikut menyajikan nilai tren $Tr(t)$ yang diperoleh dari persamaan regresi, serta deviasi $X(t) - Tr(t)$ yang merepresentasikan gabungan komponen musiman, siklus, dan acak untuk setiap periode:

Tabel 2.4 Hasil Perhitungan Tren dan Deviasi

t	Periode	$X(t) - ARR(Rp)$	$Tr(t)$	$X(t) - Tr(t)$	Keterangan
1	2022-06	946.754	1.191.911	-245.157	Di bawah tren (<i>low season awal</i>)
2	2022-07	1.278.512	1.248.021	+30.491	Sedikit di atas tren
3	2022-08	1.376.679	1.304.131	+72.548	Di atas tren (mulai <i>high season</i>)
4	2022-09	1.444.570	1.360.241	+84.329	Di atas tren
5	2022-10	1.366.420	1.416.351	-49.931	Sedikit di bawah tren
6	2022-11	1.329.453	1.472.461	-143.008	Di bawah tren (<i>low season</i>)
7	2022-12	1.774.105	1.528.571	+245.534	Jauh di atas tren (<i>peak season</i>)
8	2023-01	1.746.190	1.584.681	+161.509	Di atas tren
9	2023-02	1.809.562	1.640.791	+168.771	Di atas tren
10	2023-03	1.649.884	1.696.901	-47.017	Mendekati tren
11	2023-04	1.738.833	1.753.011	-14.178	Mendekati tren
12	2023-05	1.545.228	1.809.121	-263.893	Di bawah tren

Langkah 4:

Langkah berikutnya, dilakukan substitusi nilai ke persamaan dekomposisi. Sebagai ilustrasi konkret, berikut disajikan substitusi nilai aktual ke dalam Persamaan 2.2 untuk dua periode representatif, yakni Desember 2022 ($t = 7$) yang mencerminkan puncak musim ramai (*peak season*), dan Mei 2023 ($t = 12$) yang berada di bawah garis tren.

a) Periode Desember 2022 ($t = 7$):

Nilai ARR aktual pada bulan Desember 2022 adalah $X(7) = 1.774.105$. Nilai tren pada periode tersebut diperoleh dengan mensubstitusi $t = 7$ ke dalam persamaan regresi:

$$\begin{aligned} Tr(7) &= 1.135.800,83 + 56.110 \times 7 = 1.135.800,83 + 392.770 \\ &= 1.528.570,83 \approx 1.528.571 \end{aligned}$$

Dengan demikian, deviasi gabungan komponen musiman, siklus, dan acak adalah:

$$S(7) + C(7) + R(7) = X(7) - Tr(7) = 1.774.105 - 1.528.571 = +245.534$$

Nilai deviasi positif sebesar Rp 245.534 mengindikasikan bahwa ARR pada Desember 2022 berada jauh di atas garis tren jangka panjang. Kondisi ini kemungkinan besar dipengaruhi oleh lonjakan permintaan pada periode liburan akhir tahun (*peak season*), yang tercermin sebagai komponen musiman positif pada periode tersebut.

b) Periode Mei 2023 ($t = 12$):

Nilai ARR aktual pada bulan Mei 2023 adalah $X(12) = 1.545.228$. Nilai tren diperoleh melalui substitusi $t = 12$:

$$\begin{aligned} Tr(12) &= 1.135.800,83 + 56.110 \times 12 = 1.135.800,83 + 673.320 \\ &= 1.809.120,83 \approx 1.809.121 \end{aligned}$$

Deviasi gabungan komponen non-tren:

$$\begin{aligned} S(12) + C(12) + R(12) &= X(12) - Tr(12) = 1.545.228 - 1.809.121 \\ &= -263.893 \end{aligned}$$

Deviasi negatif sebesar -Rp 263.893 menunjukkan bahwa meskipun garis tren linear masih memprediksi kenaikan harga pada $t = 12$, nilai ARR aktual justru

tercatat lebih rendah dari proyeksi tren. Hal ini dapat mengindikasikan pengaruh musiman negatif atau komponen acak yang menekan harga pada bulan tersebut.

Interpretasi Hasil Dekomposisi

Berdasarkan hasil perhitungan pada seluruh dua belas periode observasi, tren jangka panjang ARR menunjukkan peningkatan sebesar Rp 56.110 per bulan. Pola ini konsisten dengan pertumbuhan permintaan akomodasi di wilayah Gili Islands selama periode pengamatan. Adapun deviasi positif yang cukup besar terlihat pada bulan-bulan akhir tahun 2022 (November–Februari), yang bersesuaian dengan periode libur panjang dan *high season* pariwisata. Sebaliknya, deviasi negatif dominan muncul pada periode peralihan musim seperti November dan Mei. Pola berulang ini mengindikasikan keberadaan komponen musiman $S(t)$ yang signifikan dalam data ARR, meskipun estimasi numerik kuantitatif komponen tersebut secara terpisah memerlukan cakupan data multitahunan yang lebih panjang.

Kerangka dekomposisi ini menjadi landasan penting dalam perancangan fitur-fitur masukan pada model LightGBM. Pola tren yang teridentifikasi melalui analisis ini akan direpresentasikan ke dalam fitur lag dan *rolling window* yang memungkinkan algoritma mempelajari autokorelasi temporal dalam data ARR secara lebih sistematis pada tahap rekayasa fitur (*feature engineering*).

2.2.3 Decision Tree

Pohon keputusan (*decision tree*) merupakan salah satu model *supervised machine learning* yang dapat difungsikan untuk memecahkan persoalan regresi maupun klasifikasi. Karena target prediksi dalam penentuan tarif kamar berupa nilai kontinu, pembahasan difokuskan pada pendekatan pohon regresi (*regression trees*). Model regresi *decision tree* bekerja secara spesifik dengan mempartisi ruang prediktor atau sekumpulan fitur *input* ke dalam beberapa wilayah yang terpisah dan tidak saling tumpang tindih.

Secara konseptual, pembentukan partisi wilayah $R_1, R_2, \dots, R_J$ bertujuan untuk menemukan batas yang meminimalkan *Residual Sum of Squares* (RSS).

Kriteria kesalahan RSS kemungkinan besar dievaluasi pada setiap daun (*leaf*) di struktur pohon melalui Persamaan (2.3):

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 \quad (2.3)$$

Keterangan:

RSS : *Residual Sum of Squares*, yaitu total jumlah kuadrat galat atau tingkat kesalahan dari model secara keseluruhan.

J : Total jumlah wilayah partisi atau daun (*leaf nodes*) yang terbentuk pada akhir percabangan pohon keputusan.

j : Indeks yang merepresentasikan wilayah partisi tertentu, dihitung dari wilayah ke-1 hingga wilayah ke- J .

R_j : Wilayah partisi ke- j . Area ini menampung sekumpulan observasi data yang dikelompokkan berdasarkan aturan pemisahan tertentu.

i : Indeks observasi atau baris data tunggal.

$i \in R_j$: Menunjukkan bahwa data observasi ke- i merupakan anggota atau jatuh ke dalam wilayah partisi R_j .

y_i : Nilai target aktual dari observasi ke- i (dalam konteks ini, harga kamar yang sebenarnya).

$\hat{y}_{R_j}$: Nilai prediksi untuk wilayah R_j . Pada pohon regresi, angka ini diperoleh dari rata-rata seluruh nilai target aktual yang berada di dalam partisi R_j .

Dengan $\hat{y}_{R_j}$ adalah rata-rata nilai observasi data latih yang jatuh ke dalam wilayah ke- j (James dkk., 2023). Komputasi komprehensif dilakukan untuk mencari bentuk wilayah paling optimal secara menyeluruh diduga sangat mustahil dilakukan karena keterbatasan sumber daya. Sebagai jalan tengah, algoritma pohon keputusan lazimnya menerapkan mekanisme *greedy* dari atas ke bawah yang dikenal sebagai *recursive binary splitting*.

Mekanisme *recursive binary splitting* menyeleksi sebuah variabel prediktor X_j beserta titik potong (*cutpoint*) s sedemikian rupa, sehingga ruang observasi yang terbelah mampu menyumbangkan penurunan RSS paling maksimal. Dua area bagian tersebut dapat diekspresikan secara matematis pada Persamaan (2.5):

$$R_1(j, s) = \{X \mid X_j < s\} \quad (2.4)$$

$$R_2(j, s) = \{X \mid X_j \geq s\} \quad (2.5)$$

Pada setiap langkah percabangan, variabel j dan batas s terus dicari untuk menekan nilai kerugian gabungan. Formulasi optimasi pada pemisahan simpul (*node*) ditunjukkan oleh Persamaan (2.6) berikut:

$$RSS(j, s) = \sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2 \quad (2.6)$$

Keterangan:

$RSS(j, s)$: Residual Sum of Squares, yaitu total galat kuadrat yang dihasilkan jika ruang data dipisah berdasarkan variabel prediktor ke- j pada titik potong s .

j : Indeks variabel prediktor (fitur) yang sedang dievaluasi untuk membelah data (misalnya fitur month atau *lag_1*).

s : Titik potong (*cutpoint* atau *threshold*) pada variabel ke- j yang digunakan sebagai batas pemisah percabangan.

$R_1(j, s)$: Wilayah partisi (cabang) pertama. Wilayah ini berisi himpunan observasi di mana nilai variabel ke- j biasanya lebih kecil dari titik potong s .

$R_2(j, s)$: Wilayah partisi (cabang) kedua. Wilayah ini berisi himpunan observasi di mana nilai variabel ke- j lebih besar atau sama dengan titik potong s .

i : Indeks observasi atau baris data tunggal.

$x_i \in R_1(j, s)$: Menunjukkan bahwa observasi ke- i jatuh ke dalam wilayah partisi pertama. Begitu pula untuk R_2 .

y_i : Nilai target aktual dari observasi ke- i .

$\hat{y}_{R_1}$: Nilai prediksi untuk wilayah R_1 , yang diperoleh dari nilai rata-rata seluruh target aktual y_i di dalam cabang pertama tersebut.

$\hat{y}_{R_2}$: Nilai prediksi untuk wilayah R_2 , yang diperoleh dari nilai rata-rata seluruh target aktual y_i di dalam cabang kedua tersebut.

Berdasarkan Persamaan (2.6), $\hat{y}_{R_1}$ dan $\hat{y}_{R_2}$ berturut-turut merupakan nilai rata-rata variabel target pada masing-masing kelompok $R_1(j, s)$ dan $R_2(j, s)$. Langkah pemisahan biner ini akan terus berulang hingga menemui kriteria penghentian tertentu, misalnya batas minimum jumlah data dalam sebuah daun.

Meskipun proses pelatihannya relatif mudah dipahami, struktur pohon tunggal yang dibiarkan tumbuh terlalu dalam dapat memicu terjadinya *overfitting*. Untuk meniasati risiko tersebut, strategi *cost complexity pruning* kerap kali disertakan untuk mengidentifikasi bentuk sub-pohon yang memiliki performa paling seimbang (James dkk., 2023). Fungsi kerugian yang dikenakan penalti parameter kompleksitas (α) diuraikan pada Persamaan (2.7):

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (2.7)$$

Keterangan:

y_i : Nilai aktual pada *leaf* ke- m

$\hat{y}_{R_m}$: Rata-rata nilai y pada *leaf* ke- m

α : Parameter penalti kompleksitas (*ccp_alpha*)

$|T|$: Jumlah *leaf* pada pohon

Penambahan variabel $|T|$ yang merepresentasikan total daun bertujuan memberikan batasan penalti, sehingga peningkatan nilai α akan memaksa model memangkas cabang yang kurang esensial. Di sisi lain, *decision tree* tunggal mungkin masih menyimpan kelemahan dari aspek varians prediksi. Oleh karena alasan inilah, fondasi pemodelan pohon tersebut sering dimanfaatkan sebagai pembelajar dasar (*base learner*) yang dikombinasikan secara bertahap dalam algoritma *gradient boosting*, dengan tujuan akhir merangkai sebuah fungsi prediktor yang jauh lebih presisi dan stabil (Gonçalves dan Carvalho, 2025).

Contoh 2.3

Diketahui bahwa dataset yang digunakan untuk proses peramalan (*forecasting*) adalah sebagaimana telah disajikan sebelumnya.

Tabel 2.5 Dataset Peramalan

bulan	<i>lag_1</i>	harga
5	406.650	833.627,5
6	833.627,5	800.829,7
7	800.829,7	1.048.560

Rumus yang digunakan untuk menghitung nilai *split* pada *decision tree* adalah sebagai berikut:

$$RSS(j, s) = \sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

Nilai RSS tersebut dihitung untuk setiap fitur (kolom) dan untuk setiap kandidat titik pemisah s , yang diperoleh dari nilai tengah (*midpoint*) antara dua observasi berurutan pada kolom yang sama setelah diurutkan.

Berikut merupakan tahapan perhitungan untuk menentukan *split* optimal berdasarkan nilai *Residual Sum of Squares* (RSS).

1. Node pada level 0:

Untuk $j=1$:

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 1$ (kolom bulan) yang memuat observasi bernilai 5, 6, dan 7, perhitungan titik tengah antara nilai 5 dan 6 dijabarkan sebagai berikut:

$$s = \frac{5 + 6}{2} = 5,5$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel month ($j = 1$) dengan titik potong $s = 5,5$, wilayah R_1 berisi seluruh observasi dengan nilai month lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai *month* lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

$$R_1(1; 5,5) = \{x_1\}$$

$$R_2(1; 5,5) = \{x_2, x_3\}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom *month* sebagai berikut:

Pertama, hitung kedua bagian yang dipakai pada RSS:

$$\begin{aligned} \sum_{i: x_i \in R_1(1,5,5)} (y_i - \hat{y}_{R_1})^2 &= (833.627,46 - 833.627,4648)^2 \\ &= 0,00002304 \\ \sum_{i: x_i \in R_2(1,5,5)} (y_i - \hat{y}_{R_2})^2 &= (800.829,74 - 924.694,6689)^2 + (1.048.559,60 - 924.694,6689)^2 \\ &= 30.684.013.203,067 \end{aligned}$$

Nilai RSS kemudian dikalkulasikan menggunakan kedua komponen yang telah diperoleh sebelumnya:

$$\begin{aligned} RSS(1; 5,5) &= \sum_{i: x_i \in R_1(1,5,5)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(1,5,5)} (y_i - \hat{y}_{R_2})^2 \\ &= 30.684.013.203,07 \end{aligned}$$

Langkah selanjutnya adalah menentukan titik potong s di antara dua observasi yang berdekatan pada baris data yang telah diurutkan, dengan s bertindak sebagai titik tengah (*midpoint*). Mengingat variabel yang dievaluasi adalah $j = 1$ (kolom bulan) yang memuat nilai 5, 6, dan 7, perhitungan titik tengah pada tahap ini difokuskan pada angka 6 dan 7, yaitu:

$$s = \frac{6 + 7}{2} = 6,5$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel bulan ($j = 1$) dengan titik potong $s = 6,5$, wilayah R_1 berisi seluruh observasi dengan nilai bulan lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai bulan lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

$$R_1(1; 6,5) = \{x_1, x_2\}$$

$$R_2(1; 6,5) = \{x_3\}$$

Selanjutnya hitung nilai RSS untuk mendapatkan nilai *split* pada kolom bulan sebagai berikut:

Pertama, hitung kedua bagian yang dipakai pada RSS:

$$\begin{aligned}
\sum_{i:x_i \in R_1(1,6.5)} (y_i - \hat{y}_{R_1})^2 &= (833.627,46 - 817.228,6023)^2 + (800.829,74 - 817.228,6023)^2 \\
&= 537.845.218,5992097 \\
\sum_{i:x_i \in R_2(1,6.5)} (y_i - \hat{y}_{R_2})^2 &= (1.048.559,60 - 1.048.559,5981)^2 \\
&= 0,000004
\end{aligned}$$

Nilai RSS kemudian dikalkulasikan menggunakan kedua komponen yang telah diperoleh sebelumnya:

$$\begin{aligned}
RSS(1; 6,5) &= \sum_{i:x_i \in R_1(1,6.5)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(1,6.5)} (y_i - \hat{y}_{R_2})^2 \\
&= 537.845.218,5992132425
\end{aligned}$$

Untuk j=2:

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 2$ (kolom *lag_1*) yang memuat observasi bernilai 406.650, 800.830, dan 833.627,5, perhitungan titik tengah antara nilai 406.650 dan 800.830 dijabarkan sebagai berikut:

$$s = \frac{406.650 + 800.830}{2} = 603.740$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel *lag_1* ($j = 2$) dengan titik potong $s = 603.740$, wilayah R_1 berisi seluruh observasi dengan nilai *lag_1* lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai *lag_1* lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

$$\begin{aligned}
R_1(2; 603.740) &= \{x_1\} \\
R_2(2; 603.740) &= \{x_2, x_3\}
\end{aligned}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom *lag_1* sebagai berikut:

Pertama, hitung kedua term yang dipakai pada RSS:

$$\begin{aligned}
\sum_{i:x_i \in R_1(2,603.740)} (y_i - \hat{y}_{R_1})^2 &= (833.627,46 - 833.627,4648)^2 \\
&= 0,00002304
\end{aligned}$$

$$\begin{aligned}\sum_{i:x_i \in R_2(2,603.740)} (y_i - \hat{y}_{R_2})^2 &= (800.829,74 - 924.694,66)^2 + (1.048.559,6 - 924.694,66)^2 \\ &= 30.685.041.767,8097991943\end{aligned}$$

Nilai RSS kemudian dikalkulasikan menggunakan kedua komponen yang telah diperoleh sebelumnya:

$$\begin{aligned}RSS(2; 603.740) &= \sum_{i:x_i \in R_1(2,603.740)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(2,603.740)} (y_i - \hat{y}_{R_2})^2 \\ &= 30.685.041.767,8098220825\end{aligned}$$

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 2$ (kolom *lag_1*) yang memuat observasi bernilai 406.650, 800.830, dan 833.627,5, perhitungan titik tengah antara nilai 800.830 dan 833.627 dijabarkan sebagai berikut:

$$s = \frac{800.830 + 833.627}{2} = 817.229$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel *lag_1* ($j = 2$) dengan titik potong $s = 817.229$, wilayah R_1 berisi seluruh observasi dengan nilai *lag_1* lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai *lag_1* lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

$$R_1(2; 817.229) = \{x_1, x_3\}$$

$$R_2(2; 817.229) = \{x_2\}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom *lag_1* sebagai berikut:

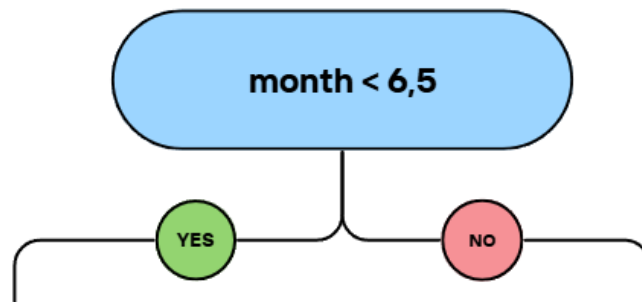
Pertama, hitung kedua term yang dipakai pada RSS:

$$\begin{aligned}\sum_{i:x_i \in R_1(2,817.229)} (y_i - \hat{y}_{R_1})^2 &= (833.627,46 - 941.093,53)^2 + (1.048.559,6 - 941.093,53)^2 \\ &= 23.097.912.402,4898071289 \\ \sum_{i:x_i \in R_2(2,817.229)} (y_i - \hat{y}_{R_2})^2 &= (800.829,74 - 800.829,7397)^2 \\ &= 0,00000009\end{aligned}$$

Selanjutnya, hitung RSS berdasarkan kedua term yang sudah dihitung:

$$\begin{aligned}
 RSS(2; 817.229) &= \sum_{i:x_i \in R_1(1,6.5)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(1,6.5)} (y_i - \hat{y}_{R_2})^2 \\
 &= 23.097.912.402,4898071289
 \end{aligned}$$

Kemudian dipilih jumlah kuadrat selisih (RSS) terkecil yang telah dikalkulasikan untuk *split node* level 1, yaitu sebesar 537.845.218,5992132425 pada kolom bulan dengan *threshold* 6,5.



Gambar 2.2 *Decision Tree Level 0*

2. Node pada level 1:

Selanjutnya, pohon dilanjutkan pada *node* kiri dengan data sebagai berikut:

Tabel 2.6 Dataset *Node Kiri Level 1*

bulan	<i>lag_1</i>	harga
5	406.650	833.627,5
6	833.627,5	800.829,7

Untuk $j=1$:

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 1$ (kolom bulan) yang memuat observasi bernilai 5 dan 6, perhitungan titik tengah antara nilai 5 dan 6 dijabarkan sebagai berikut:

$$s = \frac{5 + 6}{2} = 5,5$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel bulan ($j = 1$) dengan titik potong $s = 5,5$, wilayah R_1 berisi seluruh observasi dengan nilai bulan lebih kecil dari s , sedangkan R_2 berisi observasi

dengan nilai bulan lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

$$R_1(1; 5,5) = \{x_1\}$$

$$R_2(1; 5,5) = \{x_2\}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom bulan sebagai berikut:

$$RSS(1; 5,5) = (833.627,46 - 833.627,4648)^2 + (800.829,74 - 800.829,7397)^2 = 0$$

Untuk $j=2$:

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 2$ (kolom *lag_1*) yang memuat observasi bernilai 406.650 dan 833.627, perhitungan titik tengah antara nilai 406.650 dan 833.627 dijabarkan sebagai berikut:

$$s = \frac{406.650 + 833.627}{2} = 620.139$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel *lag_1* ($j = 2$) dengan titik potong $s = 620.139$, wilayah R_1 berisi seluruh observasi dengan nilai *lag_1* lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai *lag_1* lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

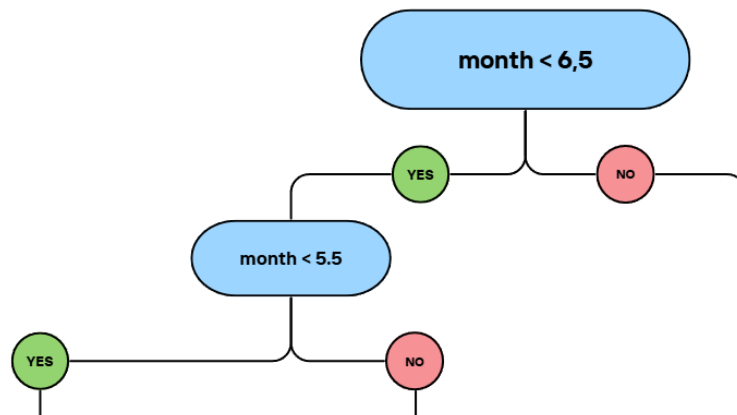
$$R_1(2; 620.139) = \{x_1\}$$

$$R_2(2; 620.139) = \{x_2\}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom *lag_1* sebagai berikut:

$$\begin{aligned} RSS(2; 620.139) &= (833.627,46 - 833.627,46)^2 + (800.829,74 - 800.829,73)^2 \\ &= 0 \end{aligned}$$

Kemudian dipilih jumlah kuadrat selisih (RSS) terkecil yang telah dihitung untuk *split node kiri level 2*, yaitu sebesar 0 pada kolom bulan dengan *threshold* 5,5.

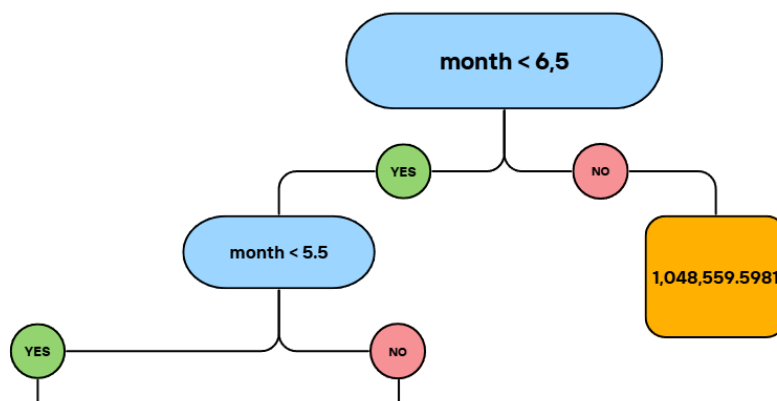
Gambar 2.3 *Decision Tree Node Kiri Level 1*

Selanjutnya, pohon dilanjutkan pada *node* kanan dengan data sebagai berikut:

Tabel 2.7 Dataset *Node Kanan Level 1*

bulan	lag_1	harga
7	800.829,739726	1.048.559,598131

Oleh karena *node* ini hanya memiliki satu sampel, *node* ini dijadikan *leaf* dengan nilai prediksi = 1.048.559,5981.

Gambar 2.4 *Decision Tree Node Kanan Level 1*

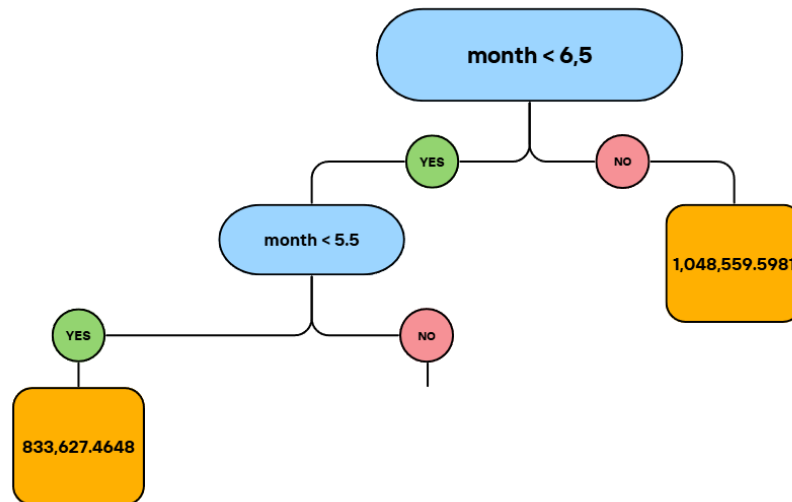
3. *Node* pada level 2:

Selanjutnya, pohon dilanjutkan pada *node* kiri dengan data sebagai berikut:

Tabel 2.8 Dataset *Node Kiri Level 3*

bulan	lag_1	harga
5	406.650	833.627,464789

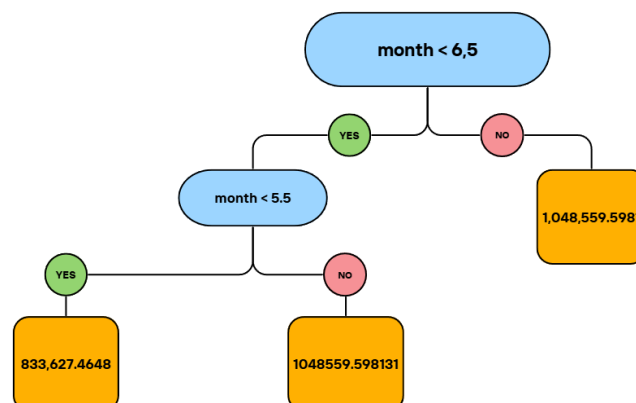
Karena *node* ini hanya memiliki satu sampel, maka *node* ini akan dijadikan *leaf* dengan nilai prediksi 833.627,4648.

Gambar 2.5 *Decision Tree Node Kiri Level 2*

Selanjutnya, pohon dilanjutkan pada *node* kanan dengan data sebagai berikut:

Tabel 2.9 Dataset *Node Kanan Level 2*

bulan	lag_1	harga
7	800.829,739726	1.048.559,598131

Gambar 2.6 *Decision Tree Node Kanan Level 2*

Pruning:

Lalu setelah pohon dibuat, *pruning* akan dilakukan sebagai regulasi agar model tidak mengalami *overfitting*. Berikut nilai *leaf* kiri dan kanan sebelum *pruning* pada *level* terakhir, yaitu *level* 3:

Tabel 2.10 Data *Node* Kiri dan Kanan Sebelum *Pruning*

Leaf	Nilai Aktual y_i
Leaf kiri	833.627,4648
Leaf kanan	800.829,7397

Selanjutnya, menggunakan persamaan 2.7 akan ditentukan nilai CCP sebagai berikut.

Pertama, rata-rata setiap *leaf* dihitung sebagai berikut,

Leaf kiri:

$$\hat{y}_{\text{kiri}} = 833.627,4648$$

Leaf kanan:

$$\hat{y}_{\text{kanan}} = 800.829,7397$$

Selanjutnya, hitung kuadrat selisih tiap observasi,

Leaf kiri:

$$\sum_{\text{Leaf kiri}} (y_i - \hat{y}_{R_m})^2 = (833.627,46 - 833.627,4648)^2 = 0$$

Leaf kanan:

$$\sum_{\text{Leaf kanan}} (y_i - \hat{y}_{R_m})^2 = (800.829,74 - 800.829,7397)^2 = 0$$

Selanjutnya, hitung total *loss* sebelum *pruning*,

$$\begin{aligned} \text{Total Loss sebelum prune} &= \sum_{\text{Leaf kiri}} (y_i - \hat{y}_{R_m})^2 + \sum_{\text{Leaf kanan}} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \\ &= 0 + 0 + 0,30 = 0,30 \end{aligned}$$

Keterangan: $\alpha \cdot |T| = 0,30$, yang nilainya sejalan dengan parameter penalti pada model.

Tahap selanjutnya adalah menyimulasikan proses pemangkasan (*pruning*)

menjadi simpul daun tunggal. Apabila kedua daun tersebut digabungkan menjadi satu daun baru, maka nilai rata-rata yang baru dikalkulasikan menggunakan persamaan berikut:

$$\hat{y}_{\text{baru}} = \frac{833.627,4648 + 800.829,7397}{2} = 817.228,6023$$

Kuadrat selisih masing-masing nilai aktual terhadap rata-rata baru:

$$(833.627,46 - 817.228,6023)^2 = 268.336.059,92$$

$$(800.829,74 - 817.228,6023)^2 = 268.336.059,92$$

Sehingga total RSS untuk *leaf* tunggal:

$$\text{RSS total} = 268.336.059,92 + 268.336.059,92 = 536.672.119,84$$

Dengan penambahan penalti $\alpha \cdot 1 \approx 1 \cdot 0,1 = 0,1$, diperoleh Loss setelah *prune* $\approx 537.845.384,84$, sesuai *output* model.

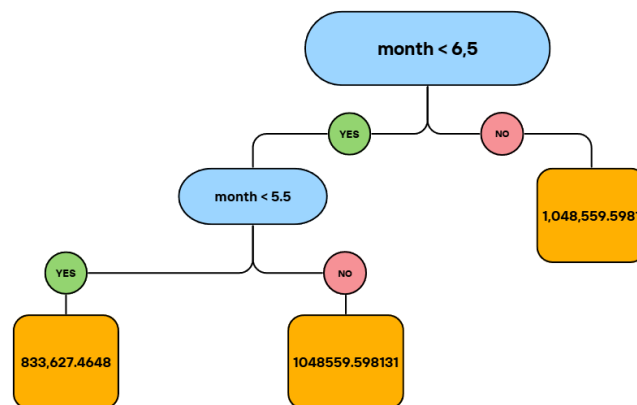
Terakhir, dihitung evaluasi keputusan *pruning*. *Loss* setelah *pruning* adalah 537.845.384,84 yang lebih besar dibandingkan *loss* sebelum *pruning* 0,30. Hal ini berarti pelaksanaan *pruning* tidak menguntungkan.

Contoh Inferensi

Misalkan terdapat data baru dengan atribut:

Tabel 2.11 Dataset Untuk Inferensi

bulan	<i>lag_1</i>	harga
8	1.048.560	—



Gambar 2.7 *Decision Tree* Akhir

Berdasarkan pohon keputusan yang telah dibangun:

1. Evaluasi *Root Node*:

Node root melakukan pemisahan berdasarkan kriteria bulan $\leq 6,5$. Karena nilai bulan pada data baru adalah 8, sampel ini masuk ke cabang kanan.

2. Evaluasi *Node Kanan*:

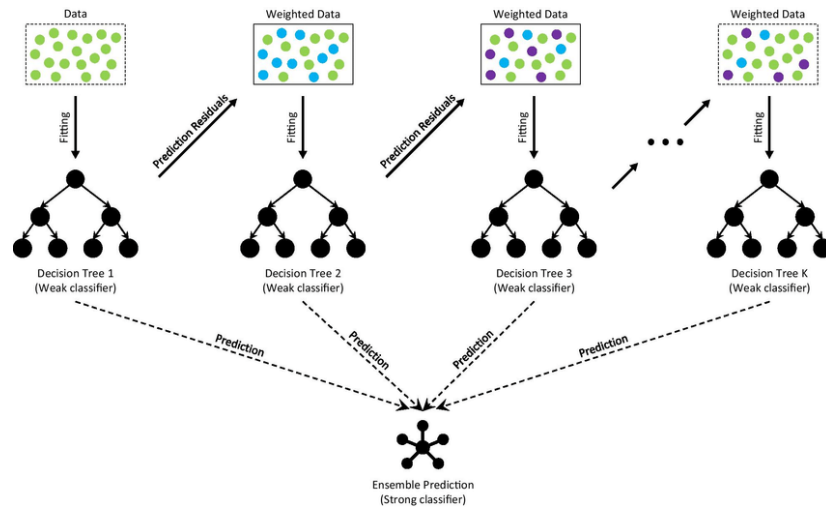
Data diteruskan ke *node* kanan, yang merupakan *leaf* dengan nilai prediksi 1.048.559,5981

3. Prediksi Akhir:

Dengan demikian, nilai prediksi untuk data baru adalah 1.048.559,5981, sesuai dengan *leaf* tempat data tersebut berada.

2.2.4 Gradient Boosting

Teknik pemodelan *gradient boosting* tampaknya beroperasi berdasarkan prinsip dasar *ensemble learning*, di mana sejumlah model prediksi berkapasitas rendah (*weak learners*) diintegrasikan secara sekuensial untuk membentuk sebuah fungsi prediktor yang solid. Dalam mayoritas kerangka kerja komputasi ini, struktur pohon keputusan (*decision trees*) kemungkinan besar dipilih sebagai pembelajar dasar. Berbeda dengan metode *bagging* yang menumbuhkan setiap pohon secara independen, pendekatan *boosting* membangun model secara bertahap. Setiap pohon baru yang ditambahkan diduga kuat bertugas untuk mengoreksi galat sisa dari kombinasi fungsi pohon-pohon yang telah dibangun pada iterasi sebelumnya (Gonçalves dan Carvalho, 2025).



Gambar 2.8 Arsitektur *Gradient Boosting* (Deng dkk., 2021)

Secara konseptual, pengembangan model ini mendekati masalah estimasi fungsi dari sudut pandang optimasi numerik di dalam ruang fungsi (*function space*). Pendekatan tersebut didasarkan pada asumsi bahwa kombinasi aditif dari sekumpulan model berkinerja rendah dapat meminimalkan nilai harapan dari fungsi kerugian (*loss function*), yang dinotasikan sebagai $L(y, F(x))$, dengan y sebagai nilai target aktual (seperti harga kamar sebenarnya) dan $F(x)$ sebagai nilai prediksi yang dihasilkan oleh model.

. Pada kasus regresi, seperti peramalan rata-rata harga kamar, fungsi kerugian kuadrat terkecil (*Mean Squared Error*) umumnya lazim digunakan. Apabila solusi minimisasi untuk fungsi kerugian spesifik dirasa sulit diselesaikan secara langsung, algoritma ini memanfaatkan strategi penurunan gradien (*steepest descent*) untuk memperbarui model (Gonçalves dan Carvalho, 2025).

Diketahui terdapat sebuah fungsi aproksimasi pada iterasi sebelumnya, $F_{m-1}(x)$. Fungsi prediktor yang baru, disimbolkan dengan $h(x; a_m)$, dapat diposisikan sebagai langkah *greedy* terbaik menuju estimasi fungsi sasaran $F^*(x)$, dengan batasan bahwa arah langkah tersebut berada dalam ruang parameter $h(x; a)$. Arah langkah ini dihitung menggunakan gradien negatif dari fungsi kerugian terhadap prediksi saat ini.

Vektor gradien ini hanya terdefinisi secara khusus pada titik-titik data observasi x_i , algoritma memerlukan proses generalisasi agar dapat diterapkan pada

nilai x lainnya. Salah satu kemungkinan penyelesaiannya adalah dengan memilih anggota dari kelas parameter $h(x; a)$ yang memiliki korelasi tertinggi atau paling sejajar dengan vektor gradien $-g_{im}$ di seluruh sebaran data. Optimasi ini dapat dieksekusi melalui minimisasi fungsi kuadrat terkecil terhadap "*pseudo-residual*" (nilai respons semu).

Setelah parameter optimal a_m didapatkan, nilai fungsi turunan ini digunakan untuk menggantikan gradien tak terbatas dalam strategi *steepest descent*. Tahapan berikutnya memerlukan pencarian garis proyektif (*line search*) untuk menemukan panjang langkah atau pembobot mutlak ρ_m yang paling meminimalkan fungsi kerugian awal.

Pada akhirnya, model hampiran akan diperbarui secara aditif dengan menambahkan fungsi pembelajar baru yang telah dikalikan dengan bobot langkahnya. Keseluruhan mekanisme matematis di atas dapat dirangkum ke dalam sebuah struktur algoritma komputasi yang berulang. Susunan operasional untuk prosedur *Gradient Boost* secara umum diuraikan melalui tahapan iteratif berikut (Gonçalves dan Carvalho, 2025):

2.2.4.1 Algoritma 1 (*Gradient Boost*)

1. Lakukan inisialisasi model awal menggunakan estimasi nilai konstan yang meminimalkan fungsi kerugian:

$$F_0(x) = \arg \min_{\rho} \sum_{i=1}^N L(y_i, \rho) \quad (2.8)$$

2. Hitung nilai *pseudo-residual* untuk setiap baris observasi

$$\tilde{y}_i = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}, \quad i = 1, \dots, N \quad (2.9)$$

3. Latih model pembelajar dasar (contohnya pohon regresi) untuk memprediksi *residual* tersebut:

$$a_m = \arg \min_a \sum_{i=1}^N [\tilde{y}_i - h(x_i; a)]^2 \quad (2.10)$$

4. Lakukan pencarian nilai skalar optimal yang meminimalkan total kerugian pada prediksi yang diperbarui:

$$\rho_m = \arg \min_{\rho} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \rho h(x_i; a_m)) \quad (2.11)$$

5. Perbarui model prediksi utama:

$$F_m(x) = F_{m-1}(x) + \rho_m h(x; a_m) \quad (2.12)$$

6. Lakukan hal yang sama untuk $m = 2$, dan seterusnya sampai M iterasi.

Berdasarkan penjabaran konsep dan turunan algoritma tersebut, terlihat bahwa *gradient boosting* memungkinkan pendekatan optimasi fungsi yang adaptif. Daripada merumuskan optimasi yang rumit dalam satu waktu, teknik *gradient boosting* menerjemahkannya ke dalam persoalan minimisasi kesalahan kuadrat secara bertahap, menjadikannya sangat relevan untuk menangkap pola deret waktu non-linear yang kompleks seperti dinamika rata-rata harga kamar hotel.

Dalam *Gradient Boosting*, *decision tree* biasanya digunakan sebagai *weak learner*. Jika kita menggunakan *decision tree* sebagai *weak learner* dan menggunakan *squared error loss*, maka persamaan *update* model dapat dituliskan sebagai berikut:

2.2.4.2 Algoritma 2 (Gradient Boost dengan Decision Tree)

1. Lakukan inisialisasi model awal menggunakan estimasi nilai konstan yang meminimalkan fungsi kerugian:

$$F_0(x) = \arg \min_{\rho} \sum_{i=1}^N L(y_i, \rho) \quad (2.13)$$

$$= \arg \min_{\rho} \sum_{i=1}^N \frac{1}{2} (y_i - \rho)^2 \quad (2.14)$$

$$= \arg \min_{\rho} \frac{1}{2} \sum_{i=1}^N (\rho - y_i)^2 \quad (2.15)$$

Selanjutnya, guna memperoleh nilai minimum, turunan pertama dari fungsi di dalam *argmin* dikalkulasikan dan disamakan dengan nol:

$$\frac{d}{d\rho} \left[\frac{1}{2} \sum_{i=1}^N (\rho - y_i)^2 \right] = 0 \quad (2.16)$$

$$\Leftrightarrow \sum_{i=1}^N \rho - y_i = 0 \quad (2.17)$$

$$\Leftrightarrow \sum_{i=1}^N \rho - \sum_{i=1}^N y_i = 0 \quad (2.18)$$

$$\Leftrightarrow - \sum_{i=1}^N y_i = 0 \quad (2.19)$$

$$\Leftrightarrow \sum_{i=1}^N \rho = \sum_{i=1}^N y_i \quad (2.20)$$

$$\Leftrightarrow N\rho = \sum_{i=1}^N y_i \quad (2.21)$$

$$\Leftrightarrow \rho = \frac{1}{N} \sum_{i=1}^N y_i \quad (2.22)$$

$$\Leftrightarrow F_0(x) = \frac{1}{N} \sum_{i=1}^N y_i \quad (2.23)$$

2. Hitung nilai *pseudo-residual* untuk setiap baris observasi

Diketahui rumus *pseudo-residual*:

$$\tilde{y}_i = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}, \quad i = 1, \dots, N \quad (2.24)$$

Dengan fungsi *loss* sebagai berikut:

$$L(y_i, F(x_i)) = \frac{1}{2} (y_i - F(x_i))^2 \quad (2.25)$$

Maka diperoleh gradien, yaitu bagian di dalam *pseudo-residual* sebagai berikut:

$$\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = -(y_i - F(x_i)) \quad (2.26)$$

Sehingga *pseudo-residual* menjadi sebagai berikut:

$$\tilde{y}_i = (y_i - F(x_i)) \quad (2.27)$$

3. Latih model pembelajar dasar (contohnya pohon regresi) untuk memprediksi *residual* tersebut:

$$a_m = \arg \min_a \sum_{i=1}^N [\tilde{y}_i - h(x_i; a)]^2 \quad (2.28)$$

4. Lakukan pencarian nilai skalar optimal yang meminimalkan total kerugian pada prediksi yang diperbarui:

$$\rho_m = \arg \min_{\rho} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \rho h(x_i; a_m)) \quad (2.29)$$

$$\rho_m = \frac{\sum_{i=1}^N h(x_i; a_m)(y_i - F_{m-1}(x_i))}{\sum_{i=1}^N h(x_i; a_m)^2}, \quad \text{dengan } L(y_i, F) \quad (2.30)$$

$$= \frac{1}{2} (y_i - F)^2$$

5. Perbarui model prediksi utama dengan v sebagai *learning rate* agar tidak *overfitting*:

$$F_m(x) = F_{m-1}(x) + v \rho_m h(x; a_m) \quad (2.31)$$

7. Lakukan hal yang sama untuk $m = 2$, dan seterusnya sampai M iterasi.

Contoh 2.4

Diberikan data seperti pada Tabel 2.12 seperti berikut:

Tabel 2.12 Dataset Untuk *Gradient Boosting*

bulan	lag_1	harga
5	406.650	833.627,464789
6	833.627,464789	800.829,739726

Proses diawali dengan melakukan inisialisasi sebagai berikut:

$$\begin{aligned} F_0(x) &= \frac{833.627,464789 + 800.829,739726}{2} \\ &= \frac{1.634.457,2045147596}{2} \\ &= 817.228,6023 \end{aligned}$$

Kemudian hitung *pseudo-residual*:

$$\begin{aligned}\tilde{y} &= \begin{bmatrix} 833.627,46 \\ 800.829,74 \end{bmatrix} - \begin{bmatrix} 817.228,60 \\ 817.228,60 \end{bmatrix} \\ &= \begin{bmatrix} 16.398,86 \\ -16.398,86 \end{bmatrix}\end{aligned}$$

Selanjutnya, latih *weak learner* pada *pseudo-residual*; *weak learner* menggunakan *decision tree* yang dilatih untuk memprediksi *pseudo-residual*.

1. *Node* pada level 0:

Dataset awal untuk melatih *weak learner* adalah sebagai berikut:

Tabel 2.13 Dataset Latih *Weak Learner*

bulan	<i>lag_1</i>	<i>psuedo-residual</i>
5	406.650	16.398,862531
6	833.627,464789	-16.398,862531

Untuk $j=1$:

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 1$ (kolom bulan) yang memuat observasi bernilai 5 dan 6, perhitungan titik tengah antara nilai 5 dan 6 dijabarkan sebagai berikut:

$$s = \frac{5 + 6}{2} = 5,5$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel bulan ($j = 1$) dengan titik potong $s = 5,5$, wilayah R_1 berisi seluruh observasi dengan nilai bulan lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai bulan lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

$$R_1(1; 5,5) = \{x_1\}$$

$$R_2(1; 5,5) = \{x_2\}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom bulan sebagai berikut:

$$\begin{aligned}RSS(1, 5.5) &= (16.398,86 - 16.398,8625)^2 + (-16.398,86 - -16.398,8625)^2 \\ &= 0,00\end{aligned}$$

Untuk $j=2$:

Pencarian kandidat titik potong (s) dilakukan dengan mengevaluasi nilai tengah (*midpoint*) pada setiap dua data yang berdekatan dalam kolom yang telah diurutkan. Mengingat variabel yang ditinjau adalah $j = 2$ (kolom *lag_1*) yang memuat observasi bernilai 406.650 dan 833.627,464789, perhitungan titik tengah antara nilai 406.650 dan 833.627,464789 dijabarkan sebagai berikut:

$$s = \frac{406.650 + 833.627}{2} = 620.139$$

Selanjutnya dibentuk wilayah R_1 dan R_2 berdasarkan nilai variabel X_j . Untuk variabel *lag_1* ($j = 2$) dengan titik potong $s = 620.139$, wilayah R_1 berisi seluruh observasi dengan nilai *lag_1* lebih kecil dari s , sedangkan R_2 berisi observasi dengan nilai *lag_1* lebih besar atau sama dengan s . Untuk R_1 dan R_2 perhitungannya sebagai berikut:

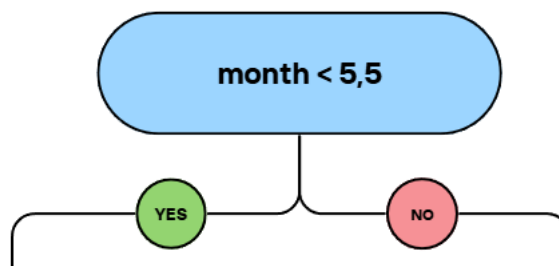
$$R_1(2; 620.139) = \{x_1\}$$

$$R_2(2; 620.139) = \{x_2\}$$

Selanjutnya, hitung RSS untuk mendapatkan nilai *split* pada kolom *lag_1* sebagai berikut:

$$\begin{aligned} RSS(2, 620.138.7) &= (16.398,8 - 16.398,8)^2 + (-16.398,8 - -16.398,8)^2 \\ &= 0 \end{aligned}$$

Kemudian dipilih jumlah kuadrat selisih (RSS) terkecil yang telah dihitung untuk *split node* level 0, yaitu sebesar 0 pada kolom bulan dengan *threshold* 5,5.



Gambar 2.9 *Weak Learner Root*

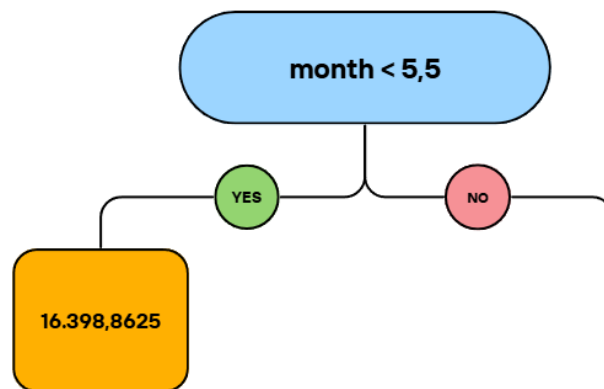
2. Node pada level 1:

Selanjutnya, pohon dilanjutkan pada *node* kiri dengan data sebagai berikut:

Tabel 2.14 Dataset *Weak Learner Level 1 Node Kiri*

bulan	lag_1	psuedo-residual
5	406.650	16.398,862531

Karena *node* ini hanya memiliki satu sampel, *node* ini dijadikan *leaf* dengan nilai prediksi = 16.398,8625.

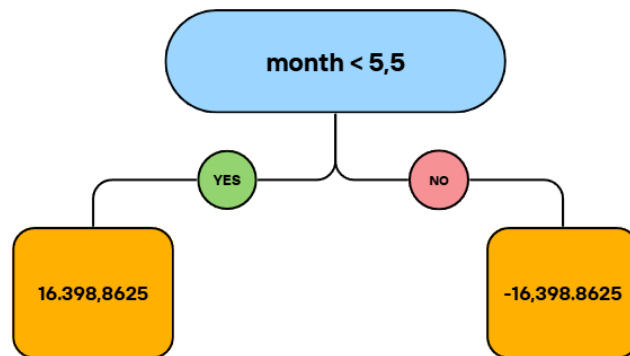
Gambar 2.10 *Weak Learner Level 1 Node Kiri*

Selanjutnya, pohon dilanjutkan pada *node* kanan dengan data sebagai berikut:

Tabel 2.15 Dataset *Weak Learner Level 1 Node Kanan*

bulan	lag_1	psuedo-residual
6	833.627,464789	-16.398,862531

Karena *node* ini hanya memiliki satu sampel, *node* ini dijadikan *leaf* dengan nilai prediksi = -16.398,8625.

Gambar 2.11 *Weak Learner Level 1 Node Kanan*

Pruning tidak diperlukan karena kedalaman pohon terbatas.

Langkah berikutnya adalah menentukan nilai ρ_1 , yang dapat dihitung melalui formulasi berikut:

$$\rho_1 = \frac{\sum_{i=1}^N h(x_i; a_1)(y_i - F_0(x_i))}{\sum_{i=1}^N h(x_i; a_1)^2},$$

Berdasarkan p_1 kita perlu $\sum_{i=1}^N h(x_i; a_1)$ yaitu *output* setiap dataset menggunakan *weak learner* yang baru dibuat sebagai berikut:

Tabel 2.16 Dataset Untuk Prediksi *Weak Learner*

bulan	<i>lag_1</i>
5	406.650
6	833.627,464789

1. Data 1: bulan = 5, *lag_1* = 406.650

Karena nilai bulan kurang dari 5,5, keluaran dari basis fungsi pohon untuk data pertama adalah:

$$h(x_1; a_1) = +16.398,862531$$

2. Data 2: bulan = 6, *lag_1* = 833.627,464789

Karena nilai bulan lebih dari 5,5, keluaran dari basis fungsi pohon untuk data kedua adalah:

$$h(x_2; a_1) = -16.398,862531$$

Maka dengan $(y_i - F_0(x_i))$ sebagai *psuedo-residual*,

$$\begin{aligned}
\rho_m &= \frac{\sum_{i=1}^2 h(x_i; a_1)(y_i - F_0(x_i))}{\sum_{i=1}^2 h(x_i; a_1)^2} \\
&= \frac{h(x_1; a_1)(y_1 - F_0(x_1)) + h(x_2; a_1)(y_2 - F_0(x_2))}{h(x_1; a_1)^2 + h(x_2; a_1)^2} \\
&= \frac{16.398,86 \times 16.398,86 + (-16.398,86) \times (-16.398,86)}{16.398,86^2 + (-16.398,86)^2} \\
&= \frac{268.752.905,45 + 268.752.905,45}{268.752.905,45 + 268.752.905,45} \\
&= \frac{537.505.810,9}{537.505.810,9} \\
&= 1
\end{aligned}$$

Lalu perbarui model prediksi utama:

Diambil asumsi $v = 0,1$, sehingga diperoleh,

$$F_1(x) = F_0(x) + v\rho_1 h(x; a_1)$$

$$F_1(x) = 817.228,6023 + 0,1 \times 1 \times h(x; a_1)$$

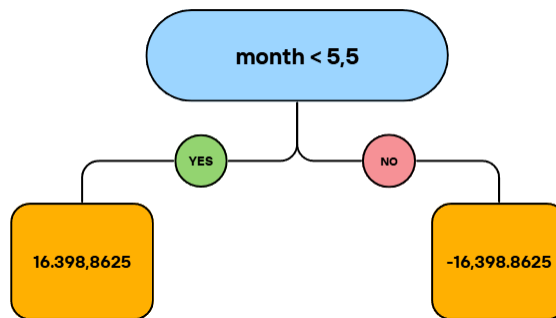
Di sini, $h(x; a_1)$ merepresentasikan *weak learner* berupa *decision tree* yang baru saja dibuat. Dalam contoh ini, hanya dibuat satu pohon, atau dengan kata lain $M = 1$. Namun, pada praktiknya, biasanya $M > 1$, sehingga langkah 3–6 pada algoritma *gradient boosting* dijalankan berulang kali. Pada langkah ke-6 setiap iterasi, model prediksi $F_m(x)$ diperbarui dengan menambahkan $v\rho_m h(x; a_m)$ ke model prediksi sebelumnya $F_{m-1}(x)$, dan proses ini terus diulang hingga seluruh M *weak learner* selesai ditambahkan.

Setelah seluruh iterasi sebanyak M dijalankan, model telah terbentuk dan siap digunakan untuk proses inferensi. Sebagai contoh, pertimbangkan data pengujian yang disajikan pada Tabel 2.17:

Tabel 2.17 Dataset Untuk Testing

bulan	lag ₋₁
7	800.829,739726

Dengan data seperti di atas, proses prediksi dapat dilakukan sebagai berikut,

Gambar 2.12 *Weak Learner* Akhir

Berdasarkan *weak learner* terakhir untuk $m = 1$, karena nilai bulan = 7 lebih besar dari 5,5, diperoleh:

$$h(x; a_1) = -16.398,862531$$

Dengan demikian, persamaan model prediksi setelah iterasi pertama menjadi:

$$\begin{aligned}
 F_1(x) &= F_0(x) + \nu \rho_1 h(x; a_1) \\
 &= 817.228,6023 + 0,1 \times 1 \times (-16.398,862531) \\
 &= 817.228,6023 - 1.639,8862531 \\
 &= 815.588,7167
 \end{aligned}$$

Sehingga hasil prediksi pertama $F_1(x)$ adalah 815.588,7167.

2.2.5 *Light Gradient Boosting Machine (LightGBM)*

Light Gradient Boosting Machine (LightGBM) pada dasarnya merupakan kerangka kerja pembelajaran mesin yang mengadopsi prinsip *gradient boosting* dengan memanfaatkan pohon keputusan sebagai pembelajar dasar (Gonçalves dan Carvalho, 2025). Kehadiran algoritma LightGBM diduga kuat berawal dari kebutuhan praktis untuk mengatasi kendala komputasi pada metode *boosting* konvensional, terutama ketika dihadapkan pada dimensi data yang masif. Berbeda dengan pendekatan tradisional, algoritma LightGBM menawarkan efisiensi memori dan waktu pelatihan yang jauh lebih ringkas tanpa harus mengorbankan akurasi secara drastis (Indah dkk., 2025).

Salah satu inovasi utama yang mendasari efisiensi operasional tersebut tampaknya terletak pada pemanfaatan algoritma berbasis histogram. Pada model

konvensional, penentuan titik belah (*split point*) mengharuskan sistem untuk memindai seluruh nilai fitur kontinu secara berurutan. Sebaliknya, pendekatan histogram dalam algoritma LightGBM mengelompokkan nilai fitur tersebut ke dalam sejumlah *bin* diskret (Indah dkk., 2025). Proses pembentukan histogram diawali dengan menginisialisasi himpunan simpul pohon dan indeks data pelatihan. Untuk setiap kedalaman pohon, algoritma memindai himpunan data yang aktif dan membangun histogram dengan mengakumulasi nilai gradien target serta jumlah kemunculan observasi pada setiap *bin*. Langkah pembentukan histogram pada praktiknya mereduksi kompleksitas komputasi secara signifikan, mengingat evaluasi titik belah hanya perlu dilakukan pada sekumpulan *bin* tersebut, bukan pada keseluruhan observasi secara individual (Gonçalves dan Carvalho, 2025).

Secara matematis, evaluasi kualitas suatu titik belah pada algoritma LightGBM didasarkan pada besarnya penurunan fungsi kerugian (*loss reduction*) yang dihasilkan oleh pemisahan tersebut. Nilai penurunan kerugian umumnya dinyatakan dalam bentuk gain, yang dihitung berdasarkan akumulasi gradien orde pertama dan kedua dari fungsi kerugian pada setiap simpul pohon. Secara umum, nilai gain dapat dirumuskan sebagai berikut:

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \quad (2.32)$$

Keterangan:

g_i : gradien orde pertama, yaitu turunan pertama dari fungsi kerugian (*loss function*) terhadap nilai prediksi pada observasi ke- i .

h_i : gradien orde kedua (*Hessian*), yaitu turunan kedua dari fungsi kerugian yang merepresentasikan kelengkungan fungsi pada observasi ke- i .

I : himpunan seluruh observasi yang berada pada simpul (leaf) sebelum proses pemisahan dilakukan.

I_L : himpunan observasi yang akan dialokasikan ke simpul kiri setelah proses pemisahan (*split*).

I_R : himpunan observasi yang akan dialokasikan ke simpul kanan setelah proses

pemisahan.

λ : parameter regularisasi L2 yang digunakan untuk mengendalikan kompleksitas model dengan memberikan penalti terhadap bobot daun yang terlalu besar sehingga dapat mengurangi risiko *overfitting*.

γ : ambang batas minimum nilai gain yang diperlukan agar suatu pemisahan simpul dapat dilakukan. Apabila nilai gain yang diperoleh lebih kecil dari γ , maka proses pemisahan tidak akan dilakukan.

Selain optimasi pada proses pencarian titik belah, arsitektur model LightGBM juga dikenal dengan strategi pertumbuhan pohonnya yang berorientasi pada daun (*leaf-wise*). Mayoritas algoritma pohon keputusan tradisional cenderung menumbuhkan cabangnya secara simetris atau melebar (*level-wise*). Akan tetapi, strategi *leaf-wise* justru menyeleksi daun yang menjanjikan penurunan kerugian (*max delta loss*) paling besar untuk dimekarkan lebih lanjut. Mekanisme pemekaran tak simetris berpotensi menekan tingkat kesalahan prediktif dengan lebih tajam, meskipun pada ukuran himpunan data yang terlampau kecil, metode *leaf-wise* cukup rentan memicu *overfitting* apabila parameter kedalaman pohon tidak dikendalikan dengan ketat (Indah dkk., 2025).

Kemampuan arsitektur LightGBM dalam memproses data berdimensi tinggi semakin diperkuat oleh kehadiran dua teknik optimasi bawaan, yakni *Gradient-based One-Side Sampling* (GOSS) dan *Exclusive Feature Bundling* (EFB). Konsep GOSS bekerja dengan menyeleksi sampel data pelatihannya berdasarkan nilai residual. Asumsi dasarnya adalah bahwa himpunan data dengan nilai gradien kecil kemungkinan besar telah direpresentasikan dengan baik oleh model. Oleh karena itu, mekanisme GOSS mempertahankan seluruh instansi data yang belum dipelajari dengan baik (bergradien tinggi), sembari hanya mengambil sampel acak pada kelompok data bergradien rendah (Gonçalves dan Carvalho, 2025). Penyesuaian bobot konstan kemudian diterapkan pada sampel acak tersebut agar distribusi data secara keseluruhan tidak menyimpang dari populasi aslinya.

Di sisi lain, teknik EFB berupaya merampingkan dimensi ruang fitur. Berbagai kumpulan data riil di industri, seperti hasil penyandian identitas properti

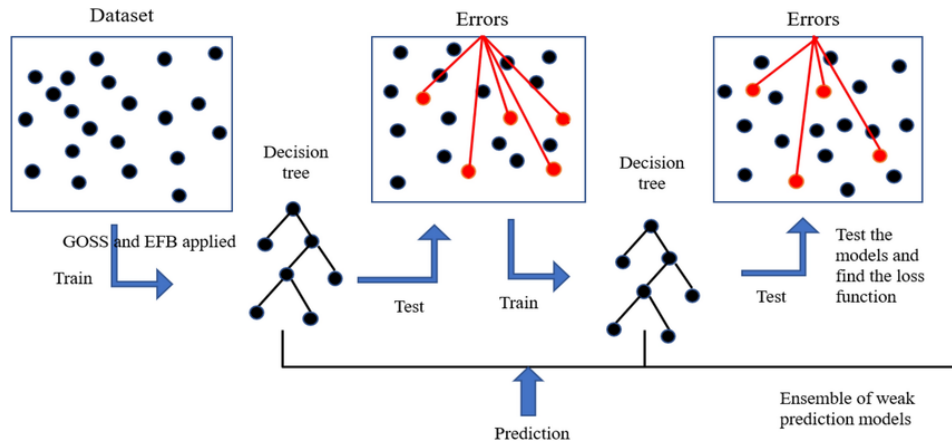
hotel ataupun tipe kamar, sering kali membentuk matriks yang sangat jarang (*sparse*). Fitur-fitur eksklusif yang jarang bernilai bukan nol secara bersamaan akhirnya digabungkan oleh algoritma EFB menjadi sebuah fitur tunggal yang padat. Penggabungan oleh algoritma EFB terbukti mempercepat laju komputasi pada ruang dimensi data yang luas (Gonçalves dan Carvalho, 2025).

Pada tahap akhir pelatihannya, model dibangun secara bertahap melalui proses iteratif dengan menambahkan kontribusi dari setiap pembelajar dasar yang dihasilkan. Pada setiap iterasi ke- m , model sebelumnya diperbarui dengan menambahkan fungsi prediksi dari pohon keputusan baru yang telah dikalibrasi melalui parameter pembobot. Kontribusi pembaruan tersebut juga dikendalikan oleh parameter *learning rate* untuk mengatur besarnya langkah pembelajaran sehingga proses pembelajaran menjadi lebih stabil dan mengurangi risiko *overfitting*. Dengan demikian, pembaruan model pada iterasi ke- m dapat dinyatakan dalam bentuk persamaan rekursif sebagai berikut.

$$F_m(x) = F_{m-1}(x) + v h(x; a_m) \quad (2.33)$$

Dalam rumusan persamaan tersebut, $F_m(x)$ merepresentasikan nilai prediksi model pada iterasi ke- m setelah dilakukan proses pembaruan. Sementara itu, $F_{m-1}(x)$ menunjukkan nilai prediksi model pada iterasi sebelumnya. Fungsi $h(x; a_m)$ menyatakan fungsi prediksi dari pohon keputusan sebagai pembelajar dasar pada iterasi ke- m dengan parameter a_m . Parameter v merupakan *learning rate* yang mengatur besarnya langkah pembaruan model pada setiap iterasi. Dengan mekanisme pembaruan model secara iteratif, setiap pembelajar dasar memberikan kontribusi bertahap dalam meningkatkan akurasi prediksi model secara keseluruhan (Indah dkk., 2025).

Melalui kombinasi strategi penyederhanaan titik belah berbasis histogram, pertumbuhan pohon yang berfokus pada kerugian daun, serta reduksi dimensi fitur, algoritma LightGBM mampu menghadirkan stabilitas komputasi yang tinggi. Karakteristik teknis tersebut menjadikannya sangat relevan dan rasional untuk diimplementasikan dalam menangani kerumitan pemodelan deret waktu yang melibatkan banyak variabel kategorikal prediktif.



Gambar 2.13 Arsitektur LightGBM (Bhupatiraju, 2025)

Berikut disajikan ringkasan algoritma model *Light Gradient Boosting Machine* (LightGBM) yang menggunakan fungsi kerugian *squared error* untuk mengukur selisih antara nilai aktual dan nilai prediksi model. Fungsi kerugian tersebut dinyatakan sebagai berikut.

$$L(y_i, F(x_i)) = \frac{1}{2} (y_i - F(x_i))^2 \quad (2.34)$$

Selanjutnya, tahapan algoritma LightGBM dapat diringkas sebagai berikut:

2.2.5.1 Algoritma 3 (LightGBM dengan *Squared Error Loss*)

1. Lakukan inisialisasi model awal menggunakan estimasi nilai konstan yang meminimalkan fungsi kerugian:

$$F_0(x) = \arg \min_{\rho} \sum_{i=1}^N L(y_i, \rho) \quad (2.35)$$

Mengingat fungsi kerugian (*squared error*) yang digunakan identik dengan pendekatan *gradient boosting* konvensional, proses minimisasi fungsi tersebut merujuk pada penjabaran Persamaan (2.23). Dengan demikian, secara langsung diperoleh nilai inisialisasi sebagai berikut:

$$F_0(x) = \frac{1}{N} \sum_{i=1}^N y_i \quad (2.36)$$

2. Lakukan transformasi fitur menggunakan *histogram binning* dan *Exclusive Feature Bundling* (EFB). Proses EFB diimplementasikan melalui

mekanisme pengelompokan fitur menggunakan metode *Greedy Bundling* dan dilanjutkan dengan prosedur *Merge Exclusive Features* untuk menghasilkan representasi fitur yang lebih padat.

3. Melakukan Transformasi Fitur Menggunakan Histogram Binning:

Histogram binning merupakan teknik diskretisasi fitur kontinu dengan cara membagi rentang nilai setiap fitur ke dalam sejumlah interval (bin) diskret.

Secara matematis, misalkan dataset fitur dinyatakan sebagai matriks:

$$X \in \mathbb{R}^{N \times m} \quad (2.37)$$

Dengan N menyatakan jumlah observasi dan m menyatakan jumlah fitur.

Untuk setiap fitur ke- j , ditentukan nilai minimum dan maksimum sebagai:

$$x_{min}^{(j)} = \min_{i=1, \dots, N} x_{ij} \quad (2.38)$$

$$x_{max}^{(j)} = \max_{i=1, \dots, N} x_{ij} \quad (2.39)$$

Jika jumlah bin yang digunakan adalah B , maka lebar setiap bin dihitung sebagai:

$$w_j = \frac{x_{max}^{(j)} - x_{min}^{(j)}}{B} \quad (2.40)$$

Batas interval setiap bin (bin *edges*) kemudian diperoleh dengan:

$$e_k^{(j)} = x_{min}^{(j)} + k \cdot w_j, \text{ dengan } k = 0, 1, \dots, B \quad (2.41)$$

sehingga terbentuk himpunan batas bin,

$$E^{(j)} = \{e_0^{(j)}, e_1^{(j)}, \dots, e_B^{(j)}\} \quad (2.42)$$

Setiap nilai fitur kemudian dipetakan ke indeks bin menggunakan fungsi pemetaan:

$$b_{ij} = \text{digitize}(x_{ij}, e_1^{(j)}, e_2^{(j)}, \dots, e_{B-1}^{(j)}) \quad (2.43)$$

Hasil transformasi menghasilkan matriks fitur baru:

$$X_{bin} \in \mathbb{Z}^{N \times m} \quad (2.44)$$

Matriks hasil transformasi memiliki dimensi yang sama dengan dataset fitur awal, namun setiap elemennya merupakan indeks bin dari nilai fitur tersebut. Representasi diskrit kemudian digunakan dalam proses

komputasi akumulasi *gradient* dan *hessian* pada algoritma pembelajaran berbasis histogram.

4. Hitung nilai turunan pertama (*gradient*) dan turunan kedua (*hessian*) dari fungsi kerugian terhadap prediksi model:

$$g_i = \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}, \quad i = 1, \dots, N \quad (2.45)$$

$$h_i = \frac{\partial^2 L(y_i, F(x_i))}{\partial F(x_i)^2}, \quad i = 1, \dots, N \quad (2.46)$$

Di mana $F(x_i)$ merupakan nilai prediksi pada iterasi sebelumnya, yaitu $F_{m-1}(x)$.

Dengan *loss* $L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2$ maka,

$$g_i = \frac{\partial}{\partial F(x_i)} \left[\frac{1}{2}(y_i - F(x_i))^2 \right] \quad (2.47)$$

$$= -(y_i - F(x_i)) \quad (2.48)$$

$$= F(x_i) - y_i \quad (2.49)$$

$$h_i = \frac{\partial^2}{\partial F(x_i)^2} \left[\frac{1}{2}(y_i - F(x_i))^2 \right] = 1 \quad (2.50)$$

6. Bentuk pohon keputusan dengan meminimalkan fungsi objektif yang telah diaproksimasi menggunakan deret Taylor orde dua:

$$L^{(t)} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 + \alpha |w_j| \right] + \gamma T \quad (2.51)$$

di mana:

T : jumlah daun pada pohon

I_j : himpunan observasi pada daun ke- j

w_j : bobot prediksi pada daun ke- j

λ : parameter regularisasi L_2

α : parameter regularisasi L_1

γ : penalti kompleksitas pohon

Berdasarkan fungsi objektif tersebut, bobot optimal pada setiap daun dapat ditentukan secara langsung melalui persamaan berikut:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (2.52)$$

Persamaan tersebut berlaku ketika *node* berperan sebagai daun. Adapun untuk menentukan titik pemisahan yang optimal pada setiap *node*, digunakan perhitungan nilai *gain*:

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \quad (2.53)$$

di mana:

I_L : himpunan observasi pada cabang kiri

I_R : himpunan observasi pada cabang kanan

I : himpunan observasi sebelum pemisahan

Nilai *gain* terbesar menunjukkan titik pemisahan yang paling optimal dalam meningkatkan kualitas model.

Data yang digunakan pada tahap ke-6 terdiri atas observasi dengan rasio a dari gradien terbesar serta rasio b dari gradien lainnya yang dipilih sebagai sampel. Untuk menjaga agar distribusi data tetap tidak bias, sampel dengan rasio b tersebut diberikan faktor penyesuaian sebesar $\frac{1-a}{b}$.

7. Perbarui model prediksi utama dengan *learning rate* v :

$$F_m(x) = F_{m-1}(x) + v h(x; a_m) \quad (2.54)$$

Komponen $h(x; a_m)$ merupakan pohon keputusan yang dihasilkan pada iterasi ke- m .

8. Lakukan hal yang sama untuk $m = 2$, dan seterusnya sampai M iterasi.

2.2.5.2 Algoritma *Greedy Bundling*

Berikut algoritma *greedy bundling* yang digunakan pada tahap 2, di mana *greedy bundling* merupakan salah satu bagian dari EFB:

```

Input  : F = fitur, K = jumlah konflik maksimum
Output : bundles

Bangun graf G
searchOrder ← G.sortByDegree()

bundles ← {}
bundlesConflict ← {}

for i dalam searchOrder do
    needNew ← True
    for j = 1 sampai len(bundles) do
        cnt ← ConflictCnt(bundles[j], F[i])
        if cnt + bundlesConflict[i] ≤ K then
            bundles[j].add(F[i])
            needNew ← False
            break
    if needNew then
        Tambahkan F[i] sebagai bundle baru

```

Gambar 2.14 Algoritma *Greedy Bundling*

Berdasarkan pseudokode pada Gambar 2.14, proses pengelompokan fitur dilakukan dengan cara menggabungkan beberapa kolom ke dalam suatu himpunan, selama total konflik antar kolom pada himpunan tersebut tidak melebihi batas maksimal yang telah ditetapkan (K). Konflik pada konteks ini diartikan sebagai banyaknya jumlah observasi saat dua fitur menunjukkan nilai selain nol pada saat yang bersamaan. Dengan diterapkannya metode tersebut, dimensi data kemungkinan besar dapat direduksi secara substansial menjadi sejumlah kecil himpunan fitur yang memiliki tingkat konflik minimal.

2.2.5.3 Algoritma *Merge Exclusive Features*

Berikut algoritma *merge exclusive features* yang digunakan pada tahap 2, di mana *merge exclusive features* juga merupakan salah satu bagian dari EFB:

```

Input  : numData = jumlah data
Input  : F = satu bundle fitur yang saling eksklusif
Output : newBin, binRanges

binRanges ← {0}
totalBin ← 0

for f dalam F do
    totalBin ← totalBin + f.numBin
    binRanges.append(totalBin)

newBin ← Bin baru dengan ukuran numData

for i = 1 sampai numData do
    newBin[i] ← 0
    for j = 1 sampai len(F) do
        if F[j].bin[i] ≠ 0 then
            newBin[i] ← F[j].bin[i] + binRanges[j]

```

Gambar 2.15 Algoritma *Merge Exclusive Features*

Algoritma penggabungan fitur eksklusif (*merge exclusive features*) diaplikasikan untuk memproses himpunan fitur bentukan tahap *greedy bundling* sehingga representasi nilai tunggal dapat dihasilkan pada masing-masing data. Dalam pelaksanaannya, serangkaian batas keranjang kumulatif (*bin ranges*) dibentuk dan diinisialisasi dari angka nol terlebih dahulu. Kemudian, evaluasi dilakukan terhadap setiap observasi asli di dalam himpunan tersebut; apabila suatu nilai non-nol ditemukan, angka tersebut akan ditambahkan dengan nilai batas kumulatif yang bersesuaian pada indeks kolom penemuan. Hasil penjumlahan ini selanjutnya ditetapkan sebagai nilai baru untuk mewakili himpunan fitur pada observasi terkait, dan tahapan pemeriksaan ini diulang secara sistematis ke seluruh baris data dalam himpunan sebelum proses dilanjutkan pada himpunan fitur lainnya.

Contoh 2.5

Sebagai ilustrasi, misalkan data diberikan sebagai berikut:

Tabel 2.18 Dataset Untuk LightGBM

bulan	<i>lag₁</i>	<i>season_{Low}</i>	<i>season_{Mid}</i>	<i>season_{High}</i>	<i>season_{Peak}</i>	harga
5	406.650	0	1	0	0	833.627,464789

Tabel 2.18 Dataset Untuk LightGBM (lanjutan)

6	833.627,464789	0	1	0	0	800.829,739726
---	----------------	---	---	---	---	----------------

Proses diawali dengan melakukan inisialisasi sebagai berikut:

$$\begin{aligned}
 F_0(x) &= \frac{1}{N} \sum_{i=1}^N y_i \\
 &= \frac{833.627,464789 + 800.829,739726}{2} \\
 &= \frac{1.634.457,2045147596}{2} \\
 &= 817.228,6022573798
 \end{aligned}$$

Selanjutnya, lakukan transformasi fitur dengan EFB, yang terdiri dari dua bagian: *greedy bundling* dan *merge exclusive feature*. Untuk *greedy bundling*, langkah-langkahnya adalah sebagai berikut:

Greedy Bundling dengan K=1:

Oleh karena EFB bekerja paling efektif pada fitur yang bersifat *sparse*, hanya kolom kategorikal yang telah diubah melalui *one-hot encoding* dan memiliki sifat *sparse* yang akan digunakan, seperti berikut:

Tabel 2.19 Dataset Sparse

<i>season_Low</i>	<i>season_Mid</i>	<i>season_High</i>	<i>season_Peak</i>
0	1	0	0
0	1	0	0

Langkah pertama adalah menghitung konflik, yaitu banyaknya observasi di mana dua fitur memiliki nilai non-nol secara bersamaan.

$$C = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Matriks konflik C menunjukkan jumlah observasi di mana dua fitur aktif bersamaan, misalnya jika $C_{12} = 2$ berarti ada 2 observasi di mana *season_Low* dan *season_Mid* aktif bersamaan, sedangkan elemen lain bernilai 0 menunjukkan tidak ada konflik antara pasangan fitur tersebut, sehingga fitur-fitur lain dapat dibundle tanpa masalah dalam proses EFB.

Selanjutnya, dihitung *degree* yaitu jumlah konflik per fitur:

$$degree = [0,0,0,0]$$

Semua derajat sama, sehingga *search order* fitur tidak memengaruhi urutan pencarian:

$$searchOrder = [0,0,0,0]$$

Kemudian dibuat *bundle* dan *bundleConflict* awal:

$$bundle = []$$

$$bundleConflict = []$$

Selanjutnya, setiap *searchOrder* akan diproses:

1. Memproses fitur 1

Oleh karena belum ada *bundle*, fitur 0 ditambahkan ke bundle pertama:

$$bundle = [[1]]$$

$$bundleConflict = [0]$$

2. Memproses fitur 2

Untuk *bundle* 1:

Oleh karena $C_{2,1} = 0$ maka,

$$cnt = 0$$

Selanjutnya, karena $cnt + bundleConflict[1] = 0 + 0 = 0 \leq K = 1$, fitur 2 ditambahkan ke bundle pertama:

$$bundle = [[1,2]]$$

Selanjutnya, nilai *cnt* dijumlahkan ke *bundleConflict*[1]

$$bundleConflict = [0 + 0] = [0]$$

3. Memproses fitur 3

Untuk *bundle* 1:

Oleh karena $C_{3,1} = 0$ dan $C_{3,2} = 0$ maka,

$$cnt = 0 + 0 = 0$$

Selanjutnya, karena $cnt + bundleConflict[1] = 0 + 0 = 0 \leq K = 1$, fitur 3 ditambahkan ke *bundle* pertama:

$$bundle = [[1,2,3]]$$

Selanjutnya, nilai *cnt* dijumlahkan ke *bundleConflict*[1]

$$bundleConflict = [0 + 0] = [0]$$

4. Memproses fitur 4

Untuk *bundle* 1:

Oleh karena $C_{3,1} = 0$, $C_{3,2} = 0$, dan $C_{3,3} = 0$ maka,

$$cnt = 0 + 0 + 0 = 0$$

Selanjutnya, karena $cnt + bundleConflict[1] = 0 + 0 = 0 \leq K = 1$, fitur 4 ditambahkan ke *bundle* pertama:

$$bundle = [[1,2,3,4]]$$

Selanjutnya, nilai *cnt* dijumlahkan ke $bundleConflict[1]$

$$bundleConflict = [0 + 0] = [0]$$

Oleh karena fitur-fitur bersifat sangat *sparse*, seluruh fitur dapat digabung dalam satu *bundle*. Pada kondisi fitur yang kurang *sparse*, algoritma akan mengecek setiap *bundle* yang ada untuk fitur tertentu dan hanya menambahkannya jika $cnt + bundleConflict[bundle] \leq K$, jika tidak fitur tersebut akan ditempatkan dalam *bundle* baru.

Selanjutnya, setelah *greedy bundling*, akan dilakukan bagian kedua EFB, yaitu *merge exclusive feature*, sebagai berikut::

Untuk bundle 1:

1. Mencari *binRange*

Untuk *Bundle* 1, pertama-tama dilakukan inisialisasi *binRange* dengan nilai awal [0]. Selanjutnya, dilakukan penambahan total bin per fitur dalam *bundle* secara kumulatif. Total bin setiap fitur dihitung berdasarkan jumlah nilai unik dalam kolom, di mana fitur biner dengan nilai 0 dan 1 hanya dihitung sebagai satu bin.

Proses kumulatif ini dilakukan sebagai berikut:

binRange pertama [0] kemudian ditambahkan dengan total bin fitur pertama ($0 + 1 = 1$), menghasilkan *binRange* [0, 1]. Penambahan berikutnya mempertimbangkan total bin fitur kedua ditambah nilai terakhir *binRange* ($1 + 1 = 2$), sehingga *binRange* menjadi [0, 1, 2]. Proses yang sama diterapkan untuk fitur ketiga dan keempat, menghasilkan *binRange* akhir [0, 1, 2, 3, 4].

Dengan cara ini, *binRange* menunjukkan *offset* kumulatif untuk setiap fitur

dalam *bundle* sehingga setiap nilai fitur dapat dipetakan ke posisi yang unik pada representasi fitur gabungan.

2. Iterasi tiap data

Setiap baris data dalam dataset diproses dengan melakukan iterasi terhadap fitur-fitur yang terdapat dalam sebuah *bundle*. Untuk setiap baris, dicari fitur pertama yang memiliki nilai bukan nol. Nilai fitur ini kemudian digunakan untuk menentukan nilai akhir pada dataset baru, di mana nilai akhir dihitung berdasarkan *binRange* yang sesuai dengan posisi fitur dalam *bundle*. Apabila semua fitur dalam *bundle* memiliki nilai nol, maka nilai akhir untuk baris tersebut tetap diatur menjadi nol.

Untuk mempermudah pemahaman, dataset *Bundle 1* disajikan secara visual sebagai ilustrasi proses transformasi ini:

Tabel 2.20 Dataset Bundle 1

Fitur 1	Fitur 2	Fitur 3	Fitur 4
0	1	0	0
0	1	0	0

a. Untuk data 1:

Oleh karena fitur kedua memiliki nilai bukan nol dan $binRange[2] = 1$, maka nilai akhir dihitung sebagai $val = \text{fitur} + binRange[2] = 1 + 1 = 2$, sehingga menghasilkan dataset baru sebagai berikut:

Tabel 2.21 Dataset Baru Untuk Data 1

<i>bundle_1</i>
2
0

b. Untuk data 2:

Oleh karena fitur kedua memiliki nilai bukan nol dan $binRange[2] = 1$, maka nilai akhir dihitung sebagai $val = \text{fitur} + binRange[2] = 1 + 1 = 2$, sehingga menghasilkan dataset baru sebagai berikut:

Tabel 2.22 Dataset Baru Untuk Data 2

<i>bundle_1</i>
2
2

Dengan demikian, dataset yang menggabungkan fitur hasil *bundle* dengan fitur numerik awal dapat disajikan sebagai berikut:

Tabel 2.23 Dataset Gabungan

bulan	<i>lag_1</i>	<i>bundle_1</i>	harga
5	406.650	2	833.627,464789
6	833.627	2	800.829,739726

Selanjutnya lakukan transformasi fitur *histogram binning* sebagai berikut:

Dataset memiliki 2 observasi dan 3 fitur.

$$X = \begin{bmatrix} 5 & 406.650 & 2 \\ 6 & 833.627.46478873 & 2 \end{bmatrix}$$

Misal jumlah bin yang digunakan adalah $B = 2$, maka setiap fitur akan memiliki 3 bin *edges*.

1. Fitur 1:

Nilai minimum, maksimum, dan lebar bin pada fitur pertama dihitung sebagai berikut:

$$x^{(1)} = [5, 6]$$

$$x_{min}^{(1)} = 5$$

$$x_{max}^{(1)} = 6$$

Lebar bin diperoleh dengan:

$$w_1 = \frac{6 - 5}{2} = 0,5$$

Selanjutnya ditentukan batas setiap bin sebagai berikut:

$$e_0^{(1)} = 5$$

$$e_1^{(1)} = 5 + 0,5 = 5,5$$

$$e_2^{(1)} = 6$$

Dengan demikian diperoleh batas bin (*edge*) dan interval sebagai berikut:

$$edge = \{5; 5,5; 6\}$$

$$Bin\ 0: [5; 5,5)$$

$$Bin\ 1 : [5,5; 6]$$

Tahap berikutnya adalah memetakan setiap nilai fitur ke dalam indeks bin yang sesuai:

$$5 \rightarrow bin\ 0$$

$$6 \rightarrow bin\ 1$$

Sehingga hasil transformasi histogram *binning* untuk fitur pertama adalah:

$$x_{bin}^{(1)} = [0,1]$$

2. Fitur 2:

Nilai minimum, maksimum, dan lebar bin pada fitur kedua dihitung sebagai berikut:

$$x^{(2)} = [406.650; 833.627,46478873]$$

$$x_{min}^{(2)} = 406.650$$

$$x_{max}^{(2)} = 833.627,46478873$$

Lebar bin diperoleh dengan:

$$w_2 = \frac{833.627,46478873 - 406.650}{2}$$

$$w_2 = 213.488.73239437$$

Selanjutnya ditentukan batas setiap bin sebagai berikut:

$$e_0^{(2)} = 406.650$$

$$e_1^{(2)} = 406.650 + 213.488,73239437$$

$$e_1^{(2)} = 620.138,73239437$$

$$e_2^{(2)} = 833.627,46478873$$

Dengan demikian diperoleh batas bin (*edge*) dan interval sebagai berikut:

$$edge = \{406.650; 620.138,73239437; 833.627,46478873\}$$

$$Bin\ 0: [406.650; 620.138,73239437)$$

$$Bin\ 1: [620.138,73239437; 833.627,46478873]$$

Tahap berikutnya adalah memetakan setiap nilai fitur ke dalam indeks bin yang sesuai:

$$406.650 \rightarrow \text{bin } 0$$

$$833.627.46478873 \rightarrow \text{bin } 1$$

Sehingga hasil transformasi histogram *binning* untuk fitur kedua adalah:

$$x_{bin}^{(2)} = [0,1]$$

3. Fitur 3:

Nilai minimum, maksimum, dan lebar bin pada fitur ketiga dihitung sebagai berikut:

$$x^{(3)} = [2,2]$$

$$x_{\min}^{(3)} = 2$$

$$x_{\max}^{(3)} = 2$$

Karena seluruh nilai pada fitur ini sama, proses histogram dengan pembagian interval tetap menghasilkan batas bin sebagai berikut:

$$\text{edge} = [1,5; 2; 2,5]$$

Sehingga interval bin yang terbentuk adalah:

$$\text{Bin } 0: [1,5; 2)$$

$$\text{Bin } 1: [2; 2,5]$$

Tahap berikutnya adalah memetakan setiap nilai fitur ke dalam indeks bin yang sesuai:

$$2 \rightarrow \text{bin } 1$$

$$2 \rightarrow \text{bin } 1$$

Sehingga hasil transformasi histogram *binning* untuk fitur ketiga adalah:

$$x_{bin}^{(3)} = [1,1]$$

Matriks Hasil Histogram *Binning*

Setelah seluruh fitur dipetakan ke bin masing-masing, diperoleh matriks fitur baru

$$X_{bin} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Matriks hasil transformasi ini mempertahankan dimensi yang sama dengan dataset awal, namun dengan setiap elemen yang telah merepresentasikan indeks bin

dari nilai fitur terkait. Pada tahapan berikutnya, kalkulasi nilai gradien dan *hessian* dilakukan secara iteratif dengan merujuk pada tingkat kerugian (*loss*) dari iterasi sebelumnya. Struktur pohon keputusan dibentuk melalui minimisasi fungsi objektif yang diaproksimasi menggunakan deret Taylor orde kedua. Setelah model prediksi utama diperbarui, prosedur ini diulangi sebanyak M kali. Mengingat parameter $\$M\$$ dalam studi ini ditetapkan bernilai 1, maka siklus iterasi hanya dilakukan satu kali. Pelaksanaan tahapan tunggal tersebut dimulai dengan komputasi gradien dan *hessian* yang dijabarkan sebagai berikut:

$$g = F(x) - y$$

$$g = \begin{bmatrix} 817.228,60225738 \\ 817.228,60225738 \end{bmatrix} - \begin{bmatrix} 833.627,46478873 \\ 800.829,73972603 \end{bmatrix}$$

$$g = \begin{bmatrix} -16.398.86253135 \\ 16.398.86253135 \end{bmatrix}$$

Selanjutnya, oleh karena diketahui pada Persamaan (2.50) nilai $h_i = 1$, maka:

$$h = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Nilai g_i dan h_i untuk setiap data direpresentasikan dalam bentuk matriks agar penulisannya lebih ringkas. Setiap elemen pada matriks tersebut tetap merepresentasikan nilai turunan pada data ke- i , sehingga elemen pertama pada matriks g menyatakan g_1 dan elemen pertama pada matriks h menyatakan h_1 .

Tahap berikutnya melibatkan penyusunan *weak learner* yang diawali dengan pembentukan histogram. Mengingat volume data dalam studi ini cukup terbatas, seluruh bin pada histogram untuk setiap fitur diikutsertakan. Representasi bin dari dataset tersebut kemudian dinyatakan sebagai X_{bin} .

$$X_{bin} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Berarti data yang dipakai:

$$X = \begin{bmatrix} 5 & 406.650 & 2 \\ 6 & 833.627,46478873 & 2 \end{bmatrix}$$

Berdasarkan representasi tersebut, data asli yang digunakan dalam perhitungan adalah:

$$g = \begin{bmatrix} -16.398,6253135 \\ 16.398,86253135 \end{bmatrix}$$

$$h = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Selanjutnya dipilih sampel dengan nilai gradien tertinggi dan terendah. Pada contoh ini jumlah data hanya dua sehingga parameter sampling digunakan sebesar $a = 0,5$ dan $b = 0,5$. Dalam praktiknya, nilai tersebut umumnya ditetapkan lebih kecil karena jumlah data jauh lebih banyak.

Gradien dengan nilai terbesar berasal dari bin 1 yaitu,

$$g_1 = 16.398,86253135$$

Sementara itu, gradien dengan nilai terendah berasal dari bin 0, yang kemudian dikalikan dengan faktor penyesuaian $\frac{1-a}{b}$, sehingga diperoleh:

$$g_0 = -16.398,86253135 \times \frac{1 - 0,5}{0,5}$$

$$= -16.398,86253135$$

Dilakukan iterasi pada setiap bin di tiap fitur untuk menentukan bin terbaik, di mana bin memiliki fungsi yang serupa dengan titik pemisah s pada *decision tree*.

Misalnya dengan $\lambda=1$ dan $\gamma=0$:

1. *Node split* pada Fitur 1, Bin 1:

Left child (bin < 1) yang terdiri dari bin 0 (dengan nilai fitur bulan sebesar 5):

$$\sum_{i \in I_L} g_i = -16.398,86253135$$

$$\sum_{i \in I_L} h_i + \lambda = 1 + 1 = 2$$

Right child (bin ≥ 1) terdiri dari bin 1 (dengan nilai fitur bulan sebesar 6):

$$\sum_{i \in I_R} g_i = 16.398,86253135$$

$$\sum_{i \in I_R} h_i + \lambda = 1 + 1$$

Original node:

$$\sum_{i \in I} g_i = -16.398,86253135 + 16.398,86253135 = 0$$

$$\sum_{i \in I} h_i + \lambda = 1 + 1$$

Gain dihitung menggunakan rumus gain:

$$\begin{aligned} \text{Gain} &= \frac{1}{2} \left(\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right) - \gamma \\ &= \frac{1}{2} \left(\frac{(-16.398,86253135)^2}{2} + \frac{(16.398,86253135)^2}{2} - \frac{(-0)^2}{3} \right) - 0 \\ &= \frac{134.461.346,161099 + 134.461.346,161097 - 0}{2} - 0 \\ &= 134.461.346,161098 \end{aligned}$$

2. Node split pada Fitur 2, Bin 1:

Left child (bin < 1) terdiri dari bin 0 (dengan nilai fitur lag_1 sebesar 406.650):

$$\sum_{i \in I_L} g_i = -16.398,86253135$$

$$\sum_{i \in I_L} h_i + \lambda = 1 + 1 = 2$$

Right child (bin ≥ 1) terdiri dari bin 1 (dengan nilai fitur lag_1 sebesar 833627,4647887324):

$$\sum_{i \in I_R} g_i = 16.398,86253135$$

$$\sum_{i \in I_R} h_i + \lambda = 1 + 1 = 2$$

Original node:

$$\sum_{i \in I} g_i = -0$$

$$\sum_{i \in I} h_i + \lambda = 2 + 1 = 3$$

Gain dihitung menggunakan rumus gain:

$$\begin{aligned} \text{Gain} &= \frac{1}{2} \left(\frac{(-16.398,86253135)^2}{2} + \frac{(16.398,86253135)^2}{2} - \frac{(-0)^2}{3} \right) - \gamma \\ &= \frac{134.461.346,161099 + 134.461.346,161097 - 0}{2} \\ &= 134.461.346,161098 \end{aligned}$$

3. *Node split* pada Fitur 3, Bin 1:

Left child kosong.

$$\sum_{i \in I_L} g_i = 0$$

$$\sum_{i \in I_L} h_i + \lambda = 0 + 1 = 1$$

Right child (bin ≥ 1) terdiri dari bin 1 (dengan nilai fitur *bundle_1* sebesar 2):

$$\sum_{i \in I_R} g_i = -0$$

$$\sum_{i \in I_R} h_i + \lambda = 2 + 1 = 3$$

Original node:

$$\sum_{i \in I} g_i = -0$$

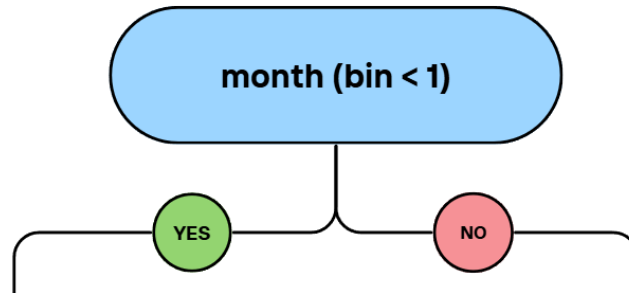
$$\sum_{i \in I} h_i + \lambda = 2 + 1 = 3$$

Gain dihitung menggunakan rumus gain:

$$\begin{aligned} \text{Gain} &= \frac{1}{2} \left(\frac{(0)^2}{1} + \frac{(-0)^2}{3} - \frac{(-0)^2}{3} \right) - 1 \\ &= \frac{0 + 0 - 0}{2} - 1 = 0 \end{aligned}$$

Berdasarkan perhitungan *gain* yang diperoleh, pemisahan terbaik terjadi pada fitur ke-1 dengan kondisi *split* pada bin < 1 . Pemisahan ini kemudian digunakan untuk menentukan pembagian indeks data ke *node* kiri dan kanan pada

tahap selanjutnya. *Split* tersebut menghasilkan nilai *gain* sebesar 134.461.346,161098 yang menunjukkan pengurangan *loss* model sebesar nilai tersebut.



Gambar 2.16 *Weak Learner Root*

Berdasarkan *split* pada fitur bulan dengan kondisi $\text{bin} < 1$, data terbagi menjadi dua *node*. *Left child* berisi data dengan indeks 0, sedangkan *right child* berisi data dengan indeks 1. Karena masing-masing *node* hanya memiliki satu data, maka tidak dilakukan proses *split* lebih lanjut dan kedua *node* tersebut menjadi *leaf node*.

Pada kondisi umum dengan jumlah data yang lebih besar, proses pembentukan pohon keputusan tidak berhenti pada satu *split* saja. Algoritma LightGBM akan menghitung nilai *gain* dari setiap kemungkinan *split* pada seluruh cabang (*branch*) yang tersedia. Cabang dengan nilai *gain* terbesar akan dipilih untuk dilakukan *split*. Setelah *split* dilakukan, cabang tersebut akan menghasilkan dua cabang baru yang kembali dievaluasi nilai *gain*-nya bersama dengan cabang-cabang lainnya. Proses ini dilakukan secara berulang dengan selalu memilih *split* yang menghasilkan *gain* terbesar hingga mencapai kondisi penghentian, sehingga seluruh *node* yang tersisa menjadi *leaf node*.

Nilai *output* untuk *leaf node* kiri dan kanan ditetapkan sebagai berikut (dengan *leaf* kiri merupakan *leaf* ke-1 dan *leaf* kanan *leaf* ke-2):

Leaf kiri (*leaf* ke-1):

Karena *node* kiri terbentuk dari kondisi pemisahan $\text{bin} < 1$, maka nilai gradien dan hessian yang digunakan dalam perhitungan adalah nilai yang berasal

dari data pada bin tersebut, yaitu g_0 dan h_0 . Nilai output pada leaf node kemudian dihitung menggunakan rumus nilai leaf pada LightGBM sebagai berikut:

$$w_1^* = -\frac{\sum_{i \in I_1} g_i}{\sum_{i \in I_1} h_i + \lambda}$$

Berdasarkan data yang terdapat pada *node* kiri, diperoleh:

$$\sum_{i \in I_1} g_i = -16.398,86253135$$

$$\sum_{i \in I_1} h_i + \lambda = 1 + 1 = 2$$

Sehingga nilai *output* pada *leaf node* kiri dapat dihitung sebagai berikut:

$$w_1^* = -\frac{-16.398,86253135}{2}$$

$$w_1^* = 8.199,431265675$$

Dengan demikian, nilai prediksi yang dihasilkan oleh *leaf node* kiri adalah sebesar 8.199,431265675.

Leaf Kanan (leaf ke-2):

Karena node kanan terbentuk dari kondisi pemisahan $\text{bin} \geq 1$, maka nilai gradien dan hessian yang digunakan dalam perhitungan berasal dari data pada bin tersebut, yaitu g_1 dan h_1 . Nilai output pada leaf node dihitung menggunakan rumus nilai leaf pada LightGBM sebagai berikut:

$$w_2^* = -\frac{\sum_{i \in I_2} g_i}{\sum_{i \in I_2} h_i + \lambda}$$

Berdasarkan data yang terdapat pada *node* kanan, diperoleh:

$$\sum_{i \in I_2} g_i = 16.398,86253135$$

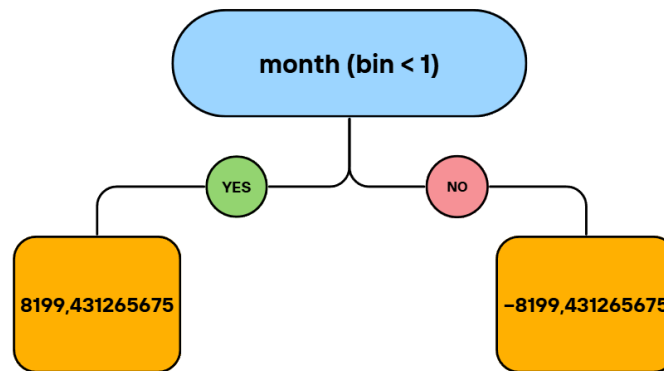
$$\sum_{i \in I_2} h_i + \lambda = 1 + 1 = 2$$

Sehingga nilai *output* pada *leaf node* kanan dapat dihitung sebagai berikut:

$$w_2^* = -\frac{16.398,86253135}{2}$$

$$w_2^* = -8.199,431265675$$

Dengan demikian, nilai prediksi yang dihasilkan oleh *leaf node* kanan adalah sebesar $-8.199,431265675$.



Gambar 2.17 Hasil Akhir *Weak Learner*

Selanjutnya dilakukan pembaruan model, di mana persamaan model pada iterasi ke- m dinyatakan sebagai:

$$F_m(x) = F_{m-1}(x) + v h(x; a_m)$$

Sebagai contoh untuk iterasi pertama:

$$F_1 = F_0 + 0,1 h(x; a_1)$$

$$F_1 = 817.228,6022573798 + 0,1 h(x; a_1)$$

Pada contoh ini, hanya dilakukan pembaruan pada satu iterasi karena $M=1$, namun dalam praktiknya nilai M biasanya lebih besar dari satu. Model dilatih secara bertahap dengan meminimalkan *loss* yang dihasilkan pada $F_m(x)$ untuk data baru, sehingga setiap langkah pelatihan bertujuan memperbarui *split* dan *leaf* pada pohon agar pengurangan *loss* menjadi maksimal, sesuai prosedur yang dijelaskan pada langkah 6.

Langkah terakhir adalah mengakhiri perulangan. Perulangan pada proses pelatihan dihentikan karena dalam contoh ini $M = 1$, sebagaimana telah dijelaskan sebelumnya. Untuk proses inferensi, misalkan terdapat data numerik sebagai berikut:

Tabel 2.24 Data *Testing* Numerik

bulan	<i>lag_1</i>
7	800.829,739726

Serta data *sparse* untuk fitur tentang *season*:

Tabel 2.25 Data *Testing* Sparse

<i>season_Low</i>	<i>season_Mid</i>	<i>season_High</i>	<i>season_Peak</i>
0	0	1	0

Data tersebut kemudian dimasukkan ke dalam prosedur *Merge Exclusive Feature*, karena *bundling* dan rentang bin (*bin range*) sudah diketahui, yaitu:

$$bundle = [1, 2, 3, 4]$$

$$binRange = [0, 1, 2, 3, 4]$$

Nilai pertama yang tidak nol terdapat pada fitur ketiga, sehingga diperoleh:

$$val = fitur + binRange[3] = 1 + 3 = 4$$

Sehingga data baru yang digunakan untuk prediksi adalah:

Tabel 2.26 Data *Testing* LightGBM

bulan	<i>lag_1</i>	<i>bundle_1</i>
7	800.829,739726	4

Selanjutnya, *edge* yang telah dihitung per kolom digunakan untuk menentukan bin masing-masing fitur:

1. Fitur 1 (bulan): [5; 5,5; 6]
2. Fitur 2 (*lag_1*): [406.650; 620.138,73239437; 833.627,46478873]
3. Fitur 3 (*bundle_1*): [1,5; 2; 2,5]

Data yang telah dipetakan ke dalam bin kemudian dimasukkan ke setiap model *weak learner*. Karena hanya terdapat satu observasi, pemetaan bin adalah sebagai berikut:

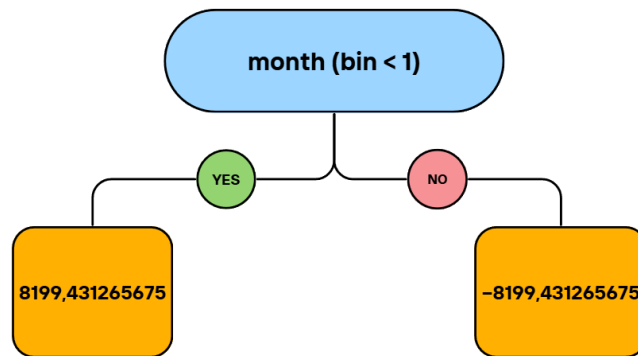
1. Fitur 1: 7 $\rightarrow$ bin 1

2. Fitur 2: 800.829,739726 $\rightarrow$ bin 1

3. Fitur 3: 800.829,739726 $\rightarrow$ bin 2

Setelah seluruh fitur dipetakan ke bin masing-masing, diperoleh matriks fitur baru X_{bin} , sebagai berikut:

$$X_{bin} = [1 \quad 1 \quad 2]$$



Gambar 2.18 *Weak Learner Decision Tree*

Selanjutnya, prediksi menggunakan *weak learner* berupa *decision tree* dilakukan. Karena fitur utama yang dipilih adalah bulan (fitur 1) dan *bin*-nya adalah 1, prediksi *leaf node* tersebut adalah $-8.199,431265675$. Nilai ini kemudian dimasukkan ke dalam formula LightGBM untuk menghitung output model:

$$\begin{aligned}
 F_1(x) &= F_0 + v \cdot h(x; a_1) \\
 &= 817.228,6022573798 + 0,1 \times (-8.199,431265675) \\
 &= 816.408,6591308123
 \end{aligned}$$

Dengan demikian, hasil prediksi untuk harga (*price*) pada bulan ke-7 adalah sebesar 816.408,6591308123. Hasil ini mencerminkan penerapan *LightGBM*, yang merupakan model *gradient boosting* yang bertujuan mengoptimalkan proses pelatihan sambil mempertahankan performa prediksi yang baik. Teknik ini menggunakan *histogram binning*, *exclusive feature bundling (EFB)*, serta *weak learner* berupa *decision tree* dengan strategi pembaruan *leaf-wise* untuk memaksimalkan pengurangan *loss* secara efisien.

2.2.6 Evaluasi Kinerja Model

Evaluasi kinerja model merupakan tahapan krusial dalam pengembangan sistem peramalan harga kamar hotel. Pengujian ini diperlukan guna menakar

seberapa dekat tebakan algoritma terhadap tarif aktual yang terekam secara historis pada saluran Booking.com. Dalam upaya mengukur tingkat akurasi model LightGBM yang akan memproses fitur-fitur berdimensi tinggi seperti identitas properti dan kamar, penelitian ini memanfaatkan beberapa tolok ukur statistik. Penggunaan metrik-metrik tersebut bukan sekadar pemenuhan syarat matematis, melainkan ditujukan untuk menggali interpretasi yang lebih mendalam mengenai letak dan besaran simpangan prediksi yang dihasilkan.

1. *Root Mean Squared Error (RMSE)*

MSE efektif untuk meminimalkan kesalahan besar, namun format kuadratnya membuat hasil pengukuran sulit diinterpretasikan dibandingkan dengan nilai aslinya. Untuk menyiasati hal tersebut, RMSE mengambil nilai akar kuadrat dari perhitungan MSE untuk mengembalikan satuan hasil evaluasi ke bentuk awalnya (Sanjaya, 2024). Sebagai contoh, jika perhitungan data target berbasis nominal rupiah, hasil luaran RMSE juga akan berada dalam besaran rupiah, sehingga memudahkan pihak manajemen operasional untuk menarik masuk akal sebuah peramalan.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.55)$$

Keterangan:

n : Jumlah data pengamatan

y_i : Nilai aktual harga

$\hat{y}_i$: Nilai hasil prediksi harga

Contoh 2.6

Misalkan data hasil prediksi harga kamar yang diperoleh dari model LightGBM (y_{pred}), dan nilai aktualnya (y_{test}) pada beberapa periode tertentu sebagai berikut:

Tabel 2.27 Dataset RMSE

y_{test}	y_{pred}
763.482,873735	803.740,181420
910.959,112854	795.846,059310
1.147.920,130773	896.153,442572
1.152.168,977191	896.153,442572
1.087.690,743703	896.153,442572

RMSE dihitung sebagai akar kuadrat dari nilai MSE, sehingga diperoleh nilai kesalahan yang kembali berada pada satuan yang sama dengan data asli:

$$\begin{aligned}
 \frac{1}{5} \sum_{i=1}^5 (y_i - \hat{y}_i)^2 &= \frac{1}{5} ((763.482,873735 - 803.740,181420)^2 \\
 &\quad + (910.959,112854 - 795.846,059310)^2 \\
 &\quad + (1.147.920,130773 - 896.153,442572)^2 \\
 &\quad + (1.152.168,977191 - 896.153,442572)^2 \\
 &\quad + (1.087.690,743703 - 896.153,442572)^2) \\
 &= \frac{1}{5} \times 180.488.622.896,38647 \\
 &= 36.097.724.579,2773
 \end{aligned}$$

Dengan demikian didapat nilai RMSE sebagai berikut:

$$\begin{aligned}
 \text{RMSE} &= \sqrt{36.097.724.579,2773} \\
 &= 189.994,0119563701
 \end{aligned}$$

Berdasarkan hasil perhitungan tersebut, nilai RMSE sebesar 189.994,0119563701 menunjukkan bahwa rata-rata kesalahan prediksi terhadap nilai aktual berada pada kisaran sekitar 189.994 satuan harga. RMSE memberikan ukuran kesalahan yang mempertimbangkan kuadrat selisih antara nilai aktual dan prediksi, sehingga kesalahan dengan nilai yang lebih besar akan memiliki pengaruh yang lebih signifikan terhadap hasil akhir perhitungan. Oleh karena itu, metrik ini sering digunakan untuk menilai tingkat akurasi model karena mampu menyoroti deviasi prediksi yang relatif besar dari nilai sebenarnya.

2. Mean Absolute Percentage Error (MAPE)

Selain pengukuran absolut, terdapat pula perhitungan MAPE yang

menawarkan sudut pandang evaluasi berbasis persentase. Dalam konteks operasional industri perhotelan, merangkum rasio kesalahan ke dalam bentuk persen umumnya terasa lebih intuitif bagi pengambil keputusan. Mengingat harga dasar tiap tipe kamar sangat bervariasi, selisih nominal beberapa ratus ribu rupiah mungkin berdampak signifikan pada akurasi kamar tipe standar, tetapi terasa wajar untuk tipe vila mewah. MAPE berupaya menetralkan bias bawaan ini dengan memaparkan besaran simpangan secara proporsional terhadap harga aktualnya (Herlando dkk., 2025). Nilai persentase yang kian mengecil menandakan bahwa tingkat kepercayaan peramalan model semakin tinggi.

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100 \quad (2.56)$$

Keterangan:

y_i : Nilai aktual data ke-i

$\hat{y}_i$: Nilai prediksi data ke-i

n : Jumlah data

Kombinasi dari kelima metrik pengujian tersebut tampaknya sanggup menyajikan evaluasi yang cukup menyeluruh terhadap performa prediksi. Hasil pengukuran ini nantinya akan memandu proses kalibrasi algoritma LightGBM, sebelum sistem peramalan ini diserahkan dan difungsikan sebagai landasan penentuan harga dinamis bagi perusahaan.

Contoh 2.7

Dengan menggunakan data pengujian yang sama:

Tabel 2.28 Dataset MAPE

y_{test}	y_{pred}
763.482,873735	803.740,181420
910.959,112854	795.846,059310
1.147.920,130773	896.153,442572
1.152.168,977191	896.153,442572

Tabel 2.29 Dataset MAPE (lanjutan)

y_{test}	y_{pred}
1.087.690,743703	896.153,442572

MAPE dihitung sebagai rata-rata persentase kesalahan antara nilai aktual dan nilai prediksi, sehingga diperoleh:

$$\begin{aligned}
 \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| &= \left(\left| \frac{763.482,873735 - 803.740,181420}{763.482,873735} \right| \right. \\
 &\quad + \left| \frac{910.959,112854 - 795.846,059310}{910.959,112854} \right| \\
 &\quad + \left| \frac{114.7920,130773 - 896.153,442572}{1147920,130773} \right| \\
 &\quad + \left| \frac{1.152.168,977191 - 896.153,442572}{1152168,977191} \right| \\
 &\quad \left. + \left| \frac{1.087.690,743703 - 896.153,442572}{1087690,743703} \right| \right) \\
 &= 0,7967158799288666
 \end{aligned}$$

Dengan demikian diperoleh nilai RMSE sebagai berikut:

$$\begin{aligned}
 MAPE &= \frac{1}{5} (0,7967158799288666) \times 100\% \\
 &= 15,9343\%
 \end{aligned}$$

Berdasarkan hasil perhitungan tersebut, dapat disimpulkan bahwa rata-rata persentase kesalahan antara nilai prediksi dan nilai aktual adalah sekitar 15,93%. MAPE memberikan gambaran mengenai tingkat kesalahan prediksi dalam bentuk persentase terhadap nilai aktual, sehingga memudahkan interpretasi terhadap akurasi model. Semakin kecil nilai MAPE yang diperoleh, maka semakin baik kemampuan model dalam menghasilkan prediksi yang mendekati nilai sebenarnya.

2.2.7 Flowchart

Diagram alir atau *flowchart* merupakan representasi grafis yang memetakan tahapan dalam suatu proses maupun prosedur. Pemodelan ini mengandalkan

sekumpulan simbol standar guna mempermudah pemahaman alur sistem, baik pada konteks analisis bisnis maupun pengembangan aplikasi tingkat lanjut. Representasi visual tersebut membantu analis dalam membedah aliran kegiatan serta dokumen dari sebuah proses bisnis agar rancangan sistem yang dihasilkan lebih terukur dan terdokumentasi dengan baik (Rahmani dan Anandya, 2024). Penggambaran diagram ini umumnya ditarik dari atas ke bawah atau dari kiri ke kanan dengan batasan awal dan akhir proses yang jelas. Setiap tahapan di dalamnya didefinisikan menggunakan kata kerja operasional yang spesifik.

Terdapat beberapa variasi diagram alir yang lazim digunakan untuk mendeskripsikan logika dan aliran kerja, yang secara umum diklasifikasikan sebagai berikut:

1. *Flowchart Sistem (System Flowchart)*

Pemodelan ini memvisualisasikan alur kerja atau aktivitas dari suatu sistem secara menyeluruh. Diagram sistem menampilkan rute aliran data beserta titik pemrosesannya, yang mencakup operasi *online* maupun *offline*. Pendekatan ini berguna untuk memberikan gambaran kerangka besar terkait relasi antarproses dan entitas data dalam sebuah arsitektur sistem (Rahmani dan Anandya, 2024).

2. *Flowchart Program (Program Flowchart)*

Diagram ini merupakan penjabaran tingkat lanjut dari flowchart sistem. Instruksi atau langkah logis dalam program komputer dipetakan secara terperinci. *Programmer* cenderung menggunakan diagram ini untuk merancang logika dasar algoritma, sementara analis sistem memanfaatkannya guna mengurutkan tugas operasional di dalam suatu prosedur komputasi.

3. *Flowchart Skematik (Schematic Flowchart)*

Meskipun memiliki kemiripan fungsional dengan flowchart sistem, diagram skematik mengintegrasikan visualisasi perangkat keras seperti komputer, mesin pencetak, dan periferal pendukung lainnya. Karakteristiknya yang lebih berorientasi pada ilustrasi fisik bertujuan menjembatani komunikasi

teknis dengan pihak manajerial atau pemangku kepentingan yang mungkin kurang familier dengan standarisasi simbol diagram alir.

4. *Flowchart Dokumen (Document Flowchart)*

Fokus utama dari pemodelan ini berada pada aliran data atau dokumen tertulis yang berpindah antar departemen atau *subsistem*. *Flowchart* dokumen menelusuri siklus hidup data mulai dari pemrosesan, pencatatan, hingga penyimpanan akhir. Diagram ini memegang peran krusial dalam pemetaan perpindahan aset informasi fisik pada sistem administrasi.

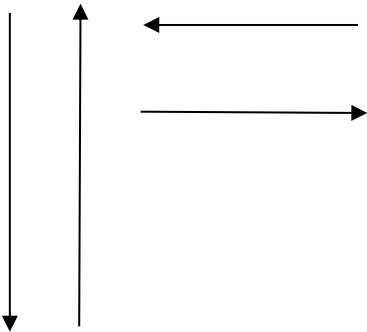
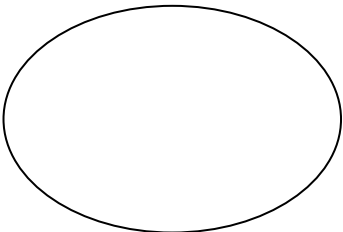
5. *Flowchart Proses (Process Flowchart)*

Diagram proses mendeskripsikan tahapan teknis dari sebuah prosedur secara spesifik. Analisis sistem sering menerapkan model ini untuk memecah alur pelaporan atau pengisian formulir. Visualisasi pada diagram proses dirancang agar rangkaian prosedur operasional yang kompleks menjadi lebih terstruktur dan mudah dievaluasi ulang (Rahmani dan Anandya, 2024).

Seluruh variasi diagram alir tersebut bertumpu pada penggunaan simbol-simbol standar untuk mewakili jenis kegiatan komputasi atau keputusan logis. Simbol-simbol baku tersebut mendefinisikan batas sistem, *input*, proses, hingga *output* yang diharapkan. Bentuk dan penjelasan masing-masing simbol operasional

ini dapat dilihat pada gambar di bawah ini.

Tabel 2.30 Simbol *Flowchart* dan Fungsinya

Simbol	Keterangan
	<i>Flow Direction Symbol</i> , Representasi grafis berupa garis dengan penunjuk arah (tanda panah) yang berfungsi untuk mengindikasikan urutan logika (sekuensi) atau transisi aliran proses dari satu tahapan operasional ke tahapan berikutnya. Simbol ini mendefinisikan navigasi alur kerja sistem secara terstruktur.
	<i>Processing Symbol</i> , Direpresentasikan melalui bangun geometri persegi panjang, simbol ini mendeskripsikan eksekusi sebuah prosedur instruksi, transformasi manipulasi data, atau serangkaian operasi komputasi spesifik yang dilakukan oleh algoritma sistem terhadap sekumpulan data masukan.
	<i>Terminator Symbol</i> , Disimbolkan dengan bangun oval, indikator visual ini bertindak sebagai penanda batas (<i>boundary</i>) absolut yang mendefinisikan titik inisialisasi (awal mula pelaksanaan sistem) serta titik terminasi (penghentian atau hasil akhir) dari keseluruhan siklus alur kerja program.

2.2.8 Optuna

Optuna adalah kerangka kerja optimasi *hyperparameter* sumber terbuka yang dirancang secara khusus untuk alur kerja pembelajaran mesin. Kerangka kerja ini mengadopsi prinsip desain *define-by-run*, yang memungkinkan ruang pencarian *hyperparameter* dikonstruksi secara dinamis selama model dieksekusi (Fadzryan dan Laksana, 2025). Pendekatan dinamis ini diekspektasikan memberikan fleksibilitas struktural yang tinggi, terutama saat mengoptimalkan arsitektur seperti LightGBM dalam memprediksi rata-rata harga kamar hotel di berbagai properti (*property_id*) dan jenis kamar (*room_id*) yang dikelola oleh Genius Hospitality

Management.

Algoritma utama yang digunakan oleh Optuna untuk melakukan eksplorasi ruang parameter secara cerdas adalah *Tree-structured Parzen Estimator* (TPE). TPE merupakan pendekatan optimasi Bayesian terarah yang memodelkan hubungan kepadatan probabilitas dari variabel *hyperparameter* berdasarkan riwayat hasil evaluasi pada iterasi-iterasi sebelumnya (Munshi dkk., 2025).

Secara konseptual matematis, algoritma TPE membagi hasil evaluasi observasi (misalnya metrik tingkat kesalahan seperti *Root Mean Square Error* atau RMSE) menjadi dua kelompok kepadatan probabilitas bersyarat. Pemisahan tersebut didasarkan pada proporsi ambang batas observasi (y^*) tertentu, yang diformulasikan sebagai berikut:

$$p(x | y) = \begin{cases} l(x), & \text{jika } y < y^* \\ g(x), & \text{jika } y \geq y^* \end{cases} \quad (2.57)$$

Keterangan:

x : Nilai atau rentang konfigurasi parameter (*hyperparameter*)

y : Nilai evaluasi target atau metrik kesalahan fungsi objektif

y^* : Nilai ambang batas pembagi yang ditentukan berdasarkan persentil atau kuantil tertentu dari riwayat observasi

$l(x)$: Estimasi fungsi kepadatan probabilitas dari kumpulan observasi yang menghasilkan performa target yang baik

$g(x)$: Estimasi fungsi kepadatan probabilitas dari kumpulan observasi yang menghasilkan performa target yang kurang baik

Untuk menentukan probabilitas pengujian konfigurasi kombinasi parameter pada iterasi selanjutnya, algoritma TPE diarahkan untuk memilih nilai x yang berpotensi memaksimalkan rasio *Expected Improvement* (EI). Secara matematis proporsional, rumusan objektif seleksi tersebut diformulasikan sebagai:

$$EI(x) \propto \frac{l(x)}{g(x)} \quad (2.58)$$

Dengan memprioritaskan maksimisasi pada rasio pemisahan tersebut, algoritma diperkirakan selalu melakukan eksplorasi di area nilai parameter yang memiliki

probabilitas kepadatan tinggi pada fungsi $l(x)$ serta kepadatan rendah pada fungsi $g(x)$. Sebagai tambahan, Optuna mengimplementasikan fitur *early pruning* (pemangkasan awal) secara efisien untuk secara otomatis memberhentikan siklus uji coba (iterasi) yang dinilai tidak akan menghasilkan metrik evaluasi yang lebih baik dari pengujian terbaik sebelumnya, sehingga berpotensi menghemat waktu dan beban komputasi.

Contoh 2.8

Sebagai ilustrasi praktis untuk mengaplikasikan rumusan algoritma TPE pada Optuna, diasumsikan terdapat sebuah proses pencarian nilai parameter *learning_rate* (x) yang ditujukan untuk meminimalkan nilai metrik tingkat kesalahan RMSE (y) dari suatu model. Berdasarkan proses *training* awal, diasumsikan telah selesai dilakukan empat kali uji coba observasi (*trial*) dengan hasil sebagai berikut:

Tabel 2. 31 Riwayat Pendistribusian Uji Coba Pencarian *Hyperparameter*

<i>Trial</i>	<i>learning_rate</i> (x)	RMSE (y)
1	0,10	15,2
2	0,05	14,8
3	0,20	18,5
4	0,01	20,1

Apabila sistem algoritma komputasi diatur untuk memisahkan performa observasi di persentil 50%, maka titik ambang batas kriteria pemisahan performa (y^*) diperkirakan terletak di kurva nilai 15,2. Sesuai dengan pembagian formula sebelumnya, hasil observasi ini dipetakan pada dua fungsi kepadatan probabilitas berbeda:

1. Fungsi Kelompok Baik ($l(x)$) dengan syarat $y \leq 15,2$, yang mencakup titik rentang observasi $x = \{0,10; 0,05\}$
2. Fungsi Kelompok Buruk ($g(x)$) dengan syarat $y > 15,2$, yang mencakup titik rentang observasi $x = \{0,20; 0,01\}$

Algoritma diprogram untuk mengestimasi distribusi probabilitas lanjutan. Untuk menentukan penetapan nilai parameter *learning_rate* pada *trial* pengujian sistem ke-5, model mengevaluasi beberapa simulasi nilai kandidat baru secara

teoretis matematis. Diasumsikan terdapat dua nilai kandidat yang akan diuji evaluasinya pada iterasi TPE, yakni $x_A = 0,08$ dan $x_B = 0,15$.

Berdasarkan proporsi rasio distribusi hasil observasi dari nilai-nilai iterasi sebelumnya, diperoleh angka probabilitas teoretis (disederhanakan) untuk masing-masing nilai kandidat:

Untuk kandidat kombinasi nilai parameter $x_A = 0,08$ (di mana titik nilainya berdekatan dengan sebaran interval kelompok baik):

1. Nilai kepadatan probabilitas pada proyeksi distribusi baik menghasilkan rata-rata nilai $l(x_A) \approx 2,4$
2. Nilai kepadatan probabilitas pada proyeksi distribusi buruk menghasilkan rata-rata nilai $g(x_A) \approx 0,4$

Rasio pemisahan maksimal *Expected Improvement* (EI) dihitung menggunakan penetapan rumusan teoritis dasar:

$$EI(x_A) \propto \frac{l(x_A)}{g(x_A)} = \frac{2,4}{0,4} = 6,0$$

Selanjutnya untuk simulasi parameter kandidat $x_B = 0,15$ (di mana titik nilainya tercatat lebih condong kepada rentang arah kelompok buruk):

1. Nilai kepadatan probabilitas untuk nilai fungsi kelompok distribusi baik menghasilkan capaian $l(x_B) \approx 0,5$
2. Nilai kepadatan probabilitas untuk nilai fungsi kelompok distribusi buruk menghasilkan capaian $g(x_B) \approx 1,8$

Rasio maksimal proporsional *Expected Improvement* dihitung kembali dan diperoleh perbandingan sebagai berikut:

$$EI(x_B) \propto \frac{l(x_B)}{g(x_B)} = \frac{0,5}{1,8} \approx 0,277$$

Berdasarkan rasio komparatif matematis dari perhitungan kedua kombinasi kandidat tersebut, nilai rasio probabilitas $EI(x_A)$ menempati angka yang secara signifikan lebih superior dibandingkan rasio $EI(x_B)$ ($6,0 > 0,277$). Dengan mempertimbangkan indikator maksimisasi probabilitas EI tersebut, diekspektasikan bahwa instrumen pencarian heuristik Optuna akan mengidentifikasi sekaligus menetapkan nilai $x_A = 0,08$ sebagai nilai dari instrumen

evaluasi *hyperparameter learning_rate* pada iterasi komputasional di tahapan uji coba selanjutnya. Mekanisme seleksi probabilistik ini diekspektasikan selalu menyeleksi kandidat paling menguntungkan di setiap putaran *trial*, sehingga proses evaluasi menemukan parameter yang optimal berpeluang jauh lebih efisien dibandingkan pencarian *grid* maupun acak.

BAB III

TINJAUAN UMUM INSTANSI

3.1 Gambaran Umum

PT Genius Utama Indonesia, yang beroperasi menggunakan nama dagang Genius Hospitality Management & Development (GHM), adalah sebuah badan usaha swasta yang berfokus pada sektor manajemen perhotelan. Bidang usaha perusahaan ini mencakup pembangunan fasilitas, pengelolaan operasional akomodasi, serta pengembangan infrastruktur pariwisata yang berkelanjutan. Wilayah operasi utama perusahaan berada di area kepulauan Provinsi Nusa Tenggara Barat.

Secara legalitas, perusahaan didirikan sebagai Perseroan Terbatas (PT) dan terdaftar secara resmi dengan nomor AHU-02000241.AH.01.11 TAHUN 2023, dengan tanggal pendirian 2 Oktober 2023. Kantor operasional PT Genius Utama Indonesia berlokasi di Jalan Kartini, Dusun Gili Air, Kecamatan Pemenang, Kabupaten Lombok Utara, Provinsi Nusa Tenggara Barat. Kegiatan operasional perusahaan meliputi siklus penuh dalam pengembangan properti wisata. Proses tersebut dimulai dari tahapan identifikasi potensi pariwisata di lokasi strategis, pelaksanaan konstruksi bangunan, hingga manajemen operasional harian guna menjaga standar kualitas layanan penginapan. Di samping itu, perusahaan juga mengelola pemasaran dan strategi pendapatan (*revenue management*) untuk mengoptimalkan tingkat hunian kamar. Sejauh ini, perusahaan telah mengelola dan menjalin kemitraan dengan berbagai properti, di antaranya Kaluku Gili Resort, Villa Phyphy, Pandan Bungalow, dan beberapa fasilitas akomodasi lainnya di kawasan Lombok dan sekitarnya.

Gambar 3.1 berikut menampilkan Kaluku Gili Resort, salah satu properti penginapan utama yang dikelola secara langsung oleh PT Genius Utama Indonesia. Keberadaan resor ini mencerminkan fokus operasional perusahaan dalam mengoptimalkan sektor pariwisata kepulauan sekaligus memberikan pengalaman menginap yang berkualitas. Seluruh rekam jejak transaksi pemesanan dari fasilitas

semacam inilah yang pada akhirnya dikumpulkan menjadi sumber himpunan data krusial dalam perancangan model prediksi harga dinamis.



Gambar 3.1 Kaluku Gili Resort – PT Genius Utama Indonesia

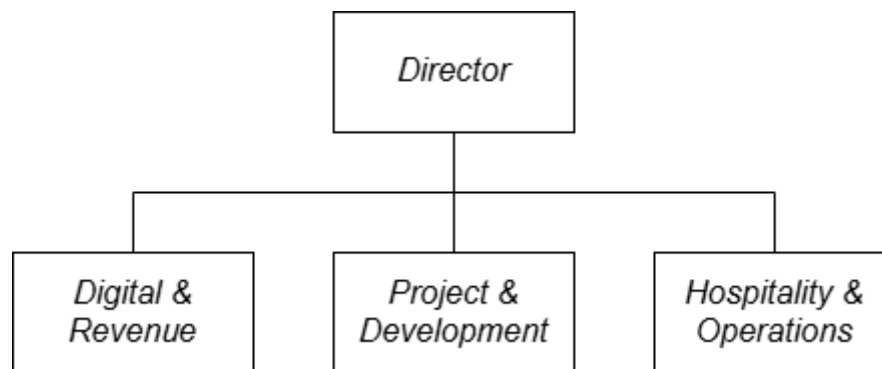
Sebagai pedoman dalam menjalankan kegiatan usahanya, PT Genius Utama Indonesia memiliki visi dan misi perusahaan sebagai berikut.

Visi: Menjadi pilihan utama bagi wisatawan kelas atas yang mencari pengalaman kemewahan dan relaksasi di kawasan kepulauan, serta menjadi merek hospitalitas terkemuka yang mampu menciptakan pengalaman menginap yang berkesan bagi para tamu.

Misi: Mendominasi pasar perhotelan di kawasan Gili Islands dengan menyediakan akomodasi berkualitas, layanan yang bersifat personal, serta pengalaman tamu yang optimal melalui inovasi dan pengembangan kemitraan strategis.

3.2 Struktur Organisasi

Struktur organisasi di PT Genius Utama Indonesia disusun secara ringkas untuk memastikan setiap tahapan bisnis, mulai dari pengembangan properti wisata hingga optimalisasi pendapatan, dapat berjalan selaras. Pembagian wewenang dalam perusahaan ini difokuskan pada tiga divisi utama yang dipimpin oleh tenaga ahli dengan spesialisasi masing-masing. Susunan hierarki dan pembagian tugas tersebut dapat dilihat pada Gambar 3.2.



Gambar 3.2 Struktur Organisasi PT Genius Utama Indonesia

Setiap divisi memiliki peran dan tanggung jawab operasional yang spesifik, sehingga tercipta koordinasi yang efektif dalam pengelolaan berbagai properti seperti hotel dan vila. Berikut adalah penjelasan mengenai susunan struktur organisasi yang diterapkan pada PT Genius Utama Indonesia:

1. *Digital & Revenue*

Divisi *Digital & Revenue* dikelola oleh Bapak Edi Rahmadi, yang bertugas merancang strategi pemasaran digital, manajemen pendapatan (*revenue management*), *e-commerce*, hingga analisis data. Bagian ini memegang peranan krusial dalam menentukan harga sewa yang kompetitif di berbagai saluran pemesanan, khususnya Booking.com. Oleh karena itu, penelitian kerja praktik yang berfokus pada peramalan harga kamar menggunakan algoritma LightGBM dirancang secara spesifik untuk mendukung tugas divisi ini.

2. *Project & Development*

Bagian *Project & Development* dipimpin oleh Bapak M. Kadri, S.T., divisi ini bertanggung jawab atas seluruh proses fisik dan infrastruktur properti. Tugas utamanya mencakup perancangan desain bangunan hotel atau vila, manajemen proyek konstruksi, pemeliharaan fasilitas secara berkala, hingga pengelolaan efisiensi anggaran pembangunan.

3. *Hospitality & Operations*

Bagian *Hospitality & Operations* ini diarahkan oleh Bapak Andrian Yosaredo, dengan fokus pada standar layanan tamu dan operasional harian. Tanggung jawabnya meliputi manajemen keramahtamahan, kepemimpinan

operasional staf di lapangan, pelayanan pelanggan, tata kelola keuangan dasar, serta eksekusi rencana strategis penjualan.

Dengan pembagian tugas yang terarah, proses pengambilan keputusan strategis perusahaan menjadi lebih terukur. Khususnya bagi divisi *Digital & Revenue*, hasil dari pemodelan prediksi harga kamar berbasis deret waktu ini diharapkan dapat memberikan acuan data yang konkret. Dengan demikian, instansi memperoleh kemudahan dalam menetapkan rata-rata harga kamar per tipe di masa mendatang tanpa harus bergantung sepenuhnya pada analisis manual, yang pada akhirnya dapat mengoptimalkan pemasukan (*occupancy* dan ADR) bagi PT Genius Utama Indonesia.

3.3 Jasa

PT Genius Utama Indonesia bergerak di bidang manajemen operasional dan pemasaran properti pariwisata, dengan menitikberatkan pada penerapan strategi manajemen pendapatan (*revenue management*). Secara spesifik, layanan utama yang ditawarkan oleh perusahaan mencakup manajemen perhotelan secara menyeluruh (*full hotel management*) dan manajemen pemasaran terpadu (*marketing management*). Layanan operasional ini mencakup siklus pengelolaan tarif secara terintegrasi, yang dimulai dari tahap kalkulasi biaya dasar kamar, analisis kompetisi pasar, hingga penyesuaian harga secara dinamis untuk merespons fluktuasi permintaan tamu dan pergantian musim. Sebagai bentuk implementasi pemasaran, perusahaan mendistribusikan sekaligus memantau pergerakan harga akomodasi tersebut melalui berbagai agen perjalanan daring, dengan platform Booking.com sebagai salah satu saluran distribusi utama.

Hingga saat ini, portofolio akomodasi yang berada di bawah tata kelola perusahaan mencakup berbagai ragam properti, antara lain Abams Gili Air, Pandan Bungalow, Mantra Gili, Kaluku Gili Resort, Bougenville Homestay, Shefa Private Villa, Salili Bungalow, Bintang Darmawan Villa (BDV), Villa PhyPhy, Villa PhyPhy 3, Bungalow No 7, Sari Gili Bungalow, dan Salili Resort. Mengingat keberagaman fasilitas dari berbagai mitra akomodasi tersebut, rentang harga sewa kamar yang ditawarkan kepada publik sangat bervariasi dan tidak memiliki batasan

angka yang statis. Besaran tarif ini bersifat fluktuatif karena dipengaruhi secara langsung oleh beberapa faktor penentu, seperti dinamika musim pariwisata (*seasonality*), pergerakan harga dari pesaing di pasar, serta spesifikasi dari tipe akomodasi maupun tipe properti itu sendiri.

Keseluruhan aktivitas pemeliharaan tarif digital tersebut pada akhirnya merekam data historis secara terperinci mengenai kinerja pemesanan dari setiap tipe kamar di berbagai properti yang dikelola. Basis data fungsional yang terbentuk inilah yang kemudian menjadi instrumen krusial dalam memfasilitasi analisis tren, sekaligus memberikan landasan empiris untuk memecahkan permasalahan kerja praktik dalam memproyeksikan pergerakan rata-rata harga kamar di masa depan.

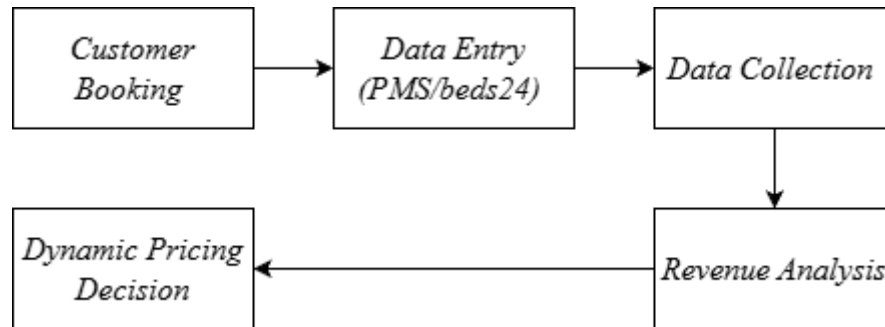
3.4 Proses Bisnis

Genius Hospitality Management menjalankan siklus operasional properti wisata yang berawal dari identifikasi potensi lokasi hingga tahap manajemen harian. Kegiatan utama instansi ini meliputi pembangunan infrastruktur fisik serta pengelolaan manajerial hotel maupun vila. Langkah tersebut diambil guna menjaga standar layanan akomodasi, di samping mengupayakan prinsip keberlanjutan yang turut melibatkan masyarakat sekitar. Seluruh tahapan ini dikelola secara terintegrasi agar produk pariwisata yang ditawarkan tetap kompetitif di pasaran.

Terkait aspek pemasaran dan promosi, perusahaan berfokus pada strategi manajemen pendapatan melalui pendekatan harga dinamis untuk segmen akomodasi kelas atas. Pelaksanaan strategi ini sangat bergantung pada pemrosesan data transaksi masa lalu. Saluran pemesanan Booking.com dipilih sebagai rujukan data utama mengingat pencatatan transaksinya cenderung lebih konsisten dan minim data kosong jika dibandingkan dengan platform pihak ketiga lainnya.

Proses analisis dilakukan dengan memantau rekam jejak pemesanan pada setiap entitas, yang direpresentasikan melalui identitas penginapan (*property_id*) dan tipe kamar (*room_id*). Data historis tersebut kemudian diekstraksi untuk memodelkan peramalan nilai rata-rata harga kamar per bulan. Melalui hasil proyeksi tren permintaan ini, pihak manajemen diharapkan mampu merumuskan kebijakan penetapan tarif yang paling optimal sesuai dengan dinamika pasar.

Rangkaian alur kerja dari pengumpulan data historis hingga penetapan harga dinamis kamar tersebut dapat dilihat lebih jelas pada diagram berikut.



Gambar 3.3 Proses Bisnis Penetapan Harga Kamar

3.5 Deskripsi Data

Dalam mendukung penetapan harga dinamis dan manajemen pendapatan, Genius Hospitality Management menggunakan data transaksi historis dari Booking.com yang telah diagregasi bulanan sebagai dasar peramalan (*forecasting*) rata-rata harga kamar berdasarkan tiap tipe kamar dan properti.

Secara struktur, dataset operasional hasil seleksi ini dikembangkan menjadi sejumlah atribut esensial yang merepresentasikan ringkasan aktivitas pemesanan selama satu bulan penuh. Mengingat harga kamar cenderung tidak hanya dipengaruhi oleh faktor internal hotel, kerangka data ini turut mengintegrasikan variabel eksternal. Atribut-atribut yang digunakan kemungkinan besar dapat diklasifikasikan ke dalam beberapa kelompok utama sebagai berikut:

1. Atribut Identitas dan Waktu Atribut ini berfungsi sebagai pengenalan unik untuk setiap baris data dan periode waktu terjadinya transaksi. Beberapa fitur utama meliputi *month_period* (format tahun dan bulan), *propertyId* (identitas unik properti/hotel), *roomId* (identitas unik tipe kamar), serta fitur turunan waktu kalender seperti *year*, *month*, dan *year_month*.
2. Atribut Finansial dan Transaksi Dasar Atribut ini memuat nilai rata-rata atau total yang berkaitan dengan nominal moneter dan volume pemesanan dalam satu bulan. Fitur yang termasuk di dalamnya adalah *price* (harga total

transaksi), *price_per_day* (harga rata-rata per malam), *commission* (komisi saluran pemesanan), *num_of_nights* (jumlah malam menginap), dan *range_booking_arrival* (jarak hari antara pemesanan dan kedatangan).

3. Atribut Keterisian dan Okupansi Kamar Atribut ini mengukur tingkat sirkulasi pemesanan dan keterisian kamar yang sangat krusial dalam pemodelan algoritma harga dinamis. Fitur tersebut secara spesifik merepresentasikan dinamika ketersediaan yang meliputi variabel *used_room_nights* (jumlah malam yang telah terpesan oleh tamu), serta *leftover_rooms* (sisa ketersediaan kamar akomodasi yang belum terjual).
4. Atribut Metadata Relasional dan Profil Demografi (Fitur Administratif/Volume) Kelompok ini merupakan hasil ekstraksi parameter relasional data induk dan indikator dasar demografi yang melengkapi konteks pemesanan. Fitur di dalamnya mencakup atribut relasional sistem seperti *id* dan *masterId* sebagai penghubung integrasi data akomodasi terpusat, serta atribut *numChild* yang memetakan proporsi rata-rata kehadiran anak-anak di dalam riwayat reservasi.
5. Atribut Makroekonomi dan Kapasitas Fisik Kelompok ini ditambahkan untuk memberikan konteks fundamental di luar rekaman transaksional murni. Fitur di dalamnya mencakup tingkat inflasi bulanan yang diduga kuat memengaruhi pergeseran daya beli konsumen, serta kuantitas kamar (*roomqty*) sebagai indikator batas ketersediaan fisik untuk setiap tipe kamar di suatu properti.

Sample dataset transaksi adalah sebagai berikut:

	A	B	C	D	E	F	G	H	I	J	K	
1	month_period	propertyId	roomId	numChild	price	commission	range_booking_arrival	num_of_nights	price_per_day	used_room_nights	leftover_rooms	
2	2022-06	158986	350455	0	525000	117075		1	262500	4	236	
3	2022-07	158986	350455	0	6.155.815.454.545.450	10.356.963.636.363.600	0.545454545	2.272.727.272.727.270	25.448.418.181.818.100	57	191	
4	2022-08	158986	350455	0.054054054	5.649.540.540.540.540	11.468.567.567.567.500	1.108.108.108.108.100	2.027.027.027.027.020	2.795.584.459.459.450	98	150	
5	2022-09	158986	350455	0	4.527.216.923.076.920	9.190.253.846.153.840		1	3	2.005.180.476.190.470	72	168
6	2022-10	158986	350455	0	469.944.512.195.122	10.091.614.634.146.300	12.439.024.390.243.900	22.439.024.390.243.900	22.549.948.170.731.700	102	146	
7	2022-11	158986	350455	0	4.384.261.904.761.900	8.900.052.380.952.380	15.238.095.238.095.200	20.476.190.476.190.400	22.321.666.666.666.600	65	175	
8	2022-12	158986	350455	0	8.816.208.947.368.420	17.896.905.263.157.800	9.736.842.105.263.150	3.526.315.789.473.680	2.538.327.975.708.500	97	151	
9	2023-01	158986	350455	0	3.328.657.894.736.840	6.757.173.684.210.520	1.068.421.052.631.570	2	2.071.464.912.280.700	44	204	
10	2023-02	158986	350455	0	775218.75	157369.25	25.625	4	21.683.806.818.181.800	40	184	
11	2023-03	158986	350455	0	4.282.413.793.103.440	79233	0.551724138	2.931.034.482.758.620	19.575.208.412.277.300	110	138	
12	2023-04	158986	350455	0	4.078.333.333.333.330	8.279.033.333.333.330	36.666.666.666.666.600	18.333.333.333.333.300	24.116.666.666.666.600	93	147	
13	2023-05	158986	350455	0	8.598.272.222.222.220	17.454.504.444.444.400	32.888.888.888.888.800	3.022.222.222.222.220	29.126.985.185.185.100	179	69	
14	2023-06	158986	350455	0	6.673.752.424.242.420	13.547.713.636.363.600	6.893.939.393.939.390	2.363.636.363.636.360	28.452.681.944.444.400	204	36	
15	2023-07	158986	350455	0.042857143	10.767.982.571.428.500	21.859.008.571.428.500	4.971.428.571.428.570	2.571.428.571.428.570	43.888.863.581.632.600	208	40	
16	2023-08	158986	350455	0.024691358	11.951.094.320.987.600	24.260.722.222.222.200	18.160.493.827.160.400	2.888.888.888.888.880	4.100.666.161.081.710	247	1	
17	2023-09	158986	350455	0	9.815.658.133.333.330	199257.92	5.293.333.333.333.330	2.6	3.912.697.586.666.660	226	14	
18	2023-10	158986	350455	0	8.451.141.285.714.280	1.715.581.857.142.850	20.142.857.142.857.100	24.571.428.571.428.500	3.544.972.116.246.490	214	34	
19	2023-11	158986	350455	0	5.956.813.636.363.630	12.092.336.363.636.300	1.5	2.181.818.181.818.800	2.719.621.022.727.270	89	151	
20	2023-12	158986	350455	0.033333333	712525.5	14.464.266.666.666.600	18.666.666.666.666.600	2.6	268.424.975	140	108	
21	2024-01	158986	350455	0	4.355.981.428.571.420	8.842.657.142.857.140	27.755.102.040.816.300	1.959.183.673.469.380	2.186.840.787.172.010	154	94	
22	2024-02	158986	350455	0	53.531.853.125	10.866.953.125	4.375	240.625	23.021.305.580.357.100	133	99	
23	2024-03	158986	350455	0.029411765	9.297.655.294.117.640	1.887.424.705.882.350	3.411.764.705.882.350	3.852.941.176.470.580	253.769.447.932.114	194	54	
24	2024-04	158986	350455	0	5.677.563.958.333.330	1.152.545.625	31.875	2.125	26.876.494.444.444.400	202	38	
25	2024-05	158986	350455	0.033333333	8.150.393.833.333.330	16.545.296.666.666.600	16.333.333.333.333.300	23.833.333.333.333.300	35.225.638.611.111.100	180	68	
26	2024-06	158986	350455	0	10.369.440.714.285.700	210.499.625	19.107.142.857.142.800	2.892.857.142.857.140	3.500.787.085.104.870	168	72	
27	2024-07	158986	350455	0.107142857	16.236.795.357.142.800	32.960.696.428.571.400	3.928.571.428.571.420	3.107.142.857.142.850	5.438.447.442.472.640	233	15	
28	2024-08	158986	350455	0.038961039	18.542.915.324.675.300	3.827.123.766.233.760	11.948.051.948.051.900	2.831.168.831.168.830	67.718.395.572.047	236	12	
29	2024-09	158986	350455	0	124.929.046.875	2.536.059.375	3.578.125	28.125	44.568.728.504.464.200	220	20	

Gambar 3.4 Sample Dataset

Berbagai variabel internal dan eksternal tersebut kemudian diintegrasikan untuk membentuk konteks temporal serta profil okupansi yang utuh. Langkah penggabungan ini dilakukan dengan harapan bahwa model pembelajaran mesin memiliki pemahaman yang lebih komprehensif terkait fluktuasi harga, tingkat permintaan, hingga kondisi ekonomi makro pada setiap tipe kamar di berbagai rentang waktu.

Selain data transaksi internal, pemodelan ini juga memanfaatkan data makroekonomi yang diperoleh dari Bank Indonesia. Sampel dataset tingkat inflasi bulanan yang digunakan sebagai variabel eksternal dapat dilihat pada tabel berikut:

Tabel 3.1 Sampel Dataset Tingkat Inflasi

No	Periode	Data Inflasi
1	Maret 2026	3.48%
2	Februari 2026	4.76%
3	Januari 2026	3.55%
4	Desember 2025	2.92%
5	November 2025	2.72%
6	Oktober 2025	2.86%
7	September 2025	2.65%
8	Agustus 2025	2.31%
9	Juli 2025	2.37%

Berbagai variabel internal dan eksternal tersebut kemudian diintegrasikan

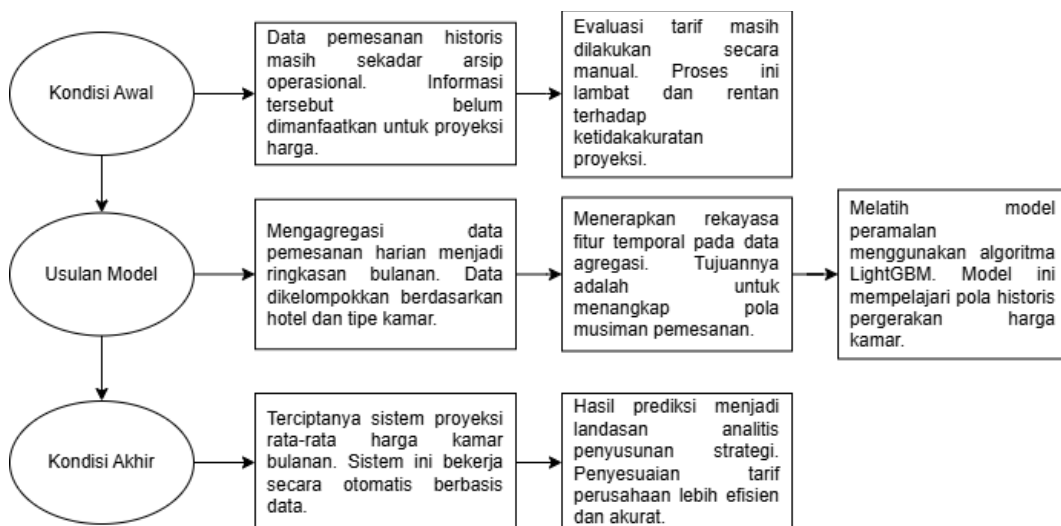
untuk membentuk konteks temporal serta profil okupansi yang utuh. Langkah penggabungan ini dilakukan dengan harapan bahwa model pembelajaran mesin memiliki pemahaman yang lebih komprehensif terkait fluktuasi harga, tingkat permintaan, hingga kondisi ekonomi makro pada setiap tipe kamar di berbagai rentang waktu.

BAB IV

METODE KERJA PRAKTIK

4.1 Analisis Penelitian

Kerangka penelitian ini disusun untuk memberikan gambaran menyeluruh mengenai alur berpikir dalam pelaksanaan studi peramalan rata-rata harga kamar hotel di PT Genius Utama Indonesia. Penelitian ini berangkat dari kondisi operasional penentuan tarif kamar yang dinamis, kemudian dirumuskan sebuah pendekatan berbasis data yang bertujuan mengubah rekaman pemesanan historis menjadi model proyeksi strategis. Alur kerangka ini digambarkan secara visual pada Gambar 4.1.



Gambar 4.1 Alur Kerangka Penelitian

a. Kondisi Awal

Untuk memperjelas alur pelaksanaan penelitian, diagram tersebut memvisualisasikan tahapan dari identifikasi masalah bisnis hingga hasil peramalan yang diharapkan. Penjelasan dari masing-masing tahapan diuraikan sebagai berikut: Kondisi Awal PT Genius Utama Indonesia bergerak di bidang manajemen operasional dan pemasaran properti pariwisata, dengan menitikberatkan pada penerapan strategi manajemen pendapatan. Layanan ini mencakup siklus pengelolaan tarif secara menyeluruh, mulai dari kalkulasi

biaya dasar kamar, analisis kompetisi, hingga penyesuaian harga secara dinamis untuk merespons fluktuasi permintaan. Perusahaan mendistribusikan ketersediaan kamar melalui berbagai agen perjalanan daring, di mana Booking.com menjadi saluran distribusi utama karena konsistensi datanya. Seluruh aktivitas pemesanan ini terekam dalam basis data historis yang memuat variabel seperti identitas properti, identitas kamar, jumlah tamu, harga, dan waktu pemesanan. Masalah utama yang dihadapi adalah belum adanya sistem otomatis yang mampu memproyeksikan pergerakan rata-rata harga kamar di masa depan untuk setiap tipe kamar secara spesifik, yang mana hal ini sangat krusial untuk memaksimalkan potensi pendapatan.

b. Usulan Model

Usulan model untuk menjawab permasalahan tersebut, diusulkan penerapan model peramalan deret waktu menggunakan algoritma *Light Gradient Boosting Machine* (LightGBM). Pendekatan ini dipilih karena kemampuannya dalam menangani data tabular dengan variabel kategorikal yang kompleks, seperti identitas hotel dan tipe kamar, secara efisien.

Proses ini melibatkan beberapa langkah persiapan data dan rekayasa fitur yang spesifik. Tahap pertama difokuskan pada agregasi data pemesanan harian menjadi ringkasan bulanan untuk setiap tipe kamar di masing-masing properti. Variabel yang digunakan mencakup rata-rata harga bulanan (satuan Rupiah/mata uang relevan), total tamu dewasa, total tamu anak-anak, dan jumlah pemesanan. Tahap kedua adalah pembentukan fitur prediktif (*feature engineering*) yang melibatkan variabel waktu (tahun, bulan, kuartal) serta variabel observasi masa lalu. Fitur masa lalu ini mencakup pergeseran harga satu bulan, dua bulan, hingga dua belas bulan sebelumnya, serta rata-rata bergerak (*rolling mean*) untuk periode tiga dan enam bulan. Semua data numerik dan kategorikal ini disusun sebagai struktur observasi analitis yang menjadi masukan bagi model LightGBM.

c. Kondisi Akhir

Melalui implementasi model prediksi ini, hasil akhir yang diharapkan adalah terbentuknya sistem yang mampu memproyeksikan rata-rata harga per

bulan untuk setiap tipe kamar pada berbagai properti yang dikelola. Keluaran ini diproyeksikan dapat memberikan wawasan empiris bagi PT Genius Utama Indonesia dalam menentukan strategi penentuan harga yang lebih akurat, adaptif terhadap tren pasar, serta meminimalkan risiko kehilangan peluang pendapatan di musim ramai.

4.2 Analisis Kebutuhan

Analisis kebutuhan merupakan tahapan awal untuk menentukan spesifikasi teknis serta batasan sistem yang diperlukan dalam pengembangan model peramalan harga kamar. Tahapan ini bertujuan mengidentifikasi seluruh kebutuhan penyelesaian masalah berdasarkan kondisi data historis pemesanan hotel. Analisis kebutuhan sistem pada penelitian ini dibagi menjadi dua bagian utama, yaitu segi fungsional yang mendeskripsikan kemampuan sistem, dan segi nonfungsional yang merangkum infrastruktur pendukung penelitian.

4.2.1 Kebutuhan Fungsional

Sistem peramalan yang dikembangkan harus memiliki kemampuan untuk memprediksi rata-rata harga kamar bulanan pada berbagai tipe kamar di beberapa hotel, seperti jaringan hotel di bawah naungan PT Genius Utama Indonesia. Model peramalan ditugaskan untuk mengenali pola data berdasarkan identitas properti (*property_id*) dan identitas kamar (*room_id*). Guna menjaga konsistensi nilai serta menghindari banyaknya data kosong, sistem membatasi pemrosesan data hanya pada pemesanan yang masuk melalui saluran Booking.com.

Secara umum, kegiatan di dalam sistem mencakup beberapa proses utama. Proses dimulai dengan ekstraksi data mentah dari penyedia layanan, dilanjutkan dengan pemrosesan awal seperti agregasi data harian menjadi rata-rata bulanan dan rekayasa fitur deret waktu (termasuk *lag* dan *rolling mean*). Setelah data siap, sistem melakukan pelatihan menggunakan algoritma *Light Gradient Boosting Machine* (LightGBM) untuk menghasilkan keluaran berupa estimasi rata-rata harga kamar di masa depan.

4.2.2 Kebutuhan Nonfungsional

Kebutuhan nonfungsional berkaitan dengan infrastruktur fisik dan logis yang dimanfaatkan untuk mengeksekusi tahapan penelitian, mulai dari pengambilan data hingga evaluasi model. Kebutuhan ini mencakup perangkat keras (*hardware*) dan perangkat lunak (*software*) utama. Rincian spesifikasi perangkat keras yang digunakan disajikan pada Tabel 4.1.

Tabel 4.1 Perangkat Keras (Hardware) yang Digunakan

Komponen	Spesifikasi
PC Desktop	AMD Ryzen 5 5500
RAM	16 GB DDR4
GPU	NVIDIA RTX 3050 6GB
<i>Storage</i>	512 GB HDD
Sistem Operasi	Windows 11 64-bit

Selain perangkat keras, penelitian ini juga membutuhkan lingkungan pengembangan perangkat lunak yang mumpuni untuk menjalankan algoritma *machine learning*. Rincian perangkat lunak pendukung disajikan pada Tabel 4.2.

Tabel 4.2 Perangkat Lunak yang Digunakan

Kategori	Nama Perangkat	Spesifikasi
Pemrograman	Python	3.12.7
Editor Kode	Visual Studio Code	1.113.0
Pengolah Kata	Microsoft Word	Microsoft 365 (64-bit)
Manajemen Versi	GitHub	-

4.2.3 Bahan Penelitian

Bahan utama yang digunakan dalam penelitian ini adalah dataset historis pemesanan hotel berskala besar. Dataset ini diperoleh dari integrasi Antarmuka Pemrograman Aplikasi (API) layanan manajemen properti Beds24 versi 2 (Beds24, 2024). Pengambilan data dilakukan secara terprogram memanfaatkan metode otentikasi *token* dengan mengakses *endpoint* API terkait pemesanan dan inventaris.

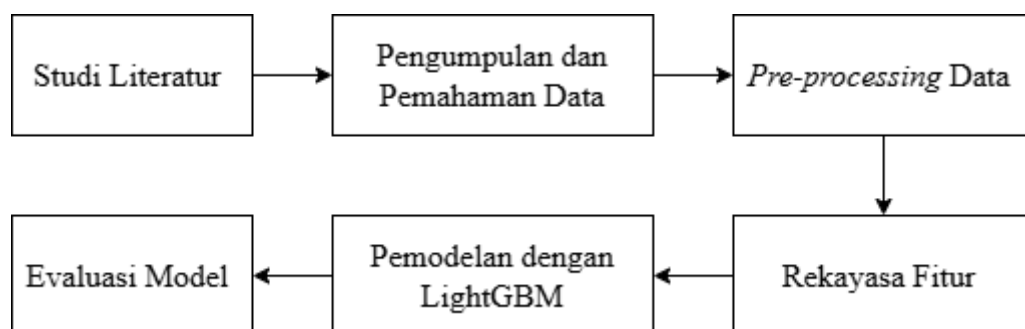
Atribut data mentah yang diekstrak meliputi identitas pemesanan, identitas properti, identitas kamar, status, tanggal kedatangan, tanggal keberangkatan, jumlah tamu dewasa, jumlah anak, harga, saluran pemesanan, serta waktu

pemesanan direkam. Data inilah yang kemudian disaring dan diintegrasikan untuk melatih model LightGBM agar dapat meramalkan fluktuasi harga kamar secara akurat untuk setiap tipe kamar.

Melalui pemenuhan spesifikasi fungsional dan nonfungsional tersebut, sistem diharapkan mampu memproses dataset pemesanan dari API Beds24 secara efisien. Ketersediaan perangkat pemrosesan yang memadai serta fokus pada saluran data yang konsisten menjadi landasan penting agar model LightGBM dapat menghasilkan nilai peramalan harga kamar yang andal.

4.3 Tahapan Penelitian

Proses pengembangan sistem peramalan rata-rata harga kamar hotel menggunakan metode pembelajaran mesin dan algoritma *Light Gradient Boosting Machine* (LightGBM) dilakukan melalui beberapa tahapan terstruktur sebagai berikut:



Gambar 4.1 Alur Tahapan Penelitian

a. Studi Literatur

Tahap ini dilakukan dengan mengkaji berbagai referensi ilmiah, jurnal, dan dokumentasi teknis yang berkaitan dengan peramalan deret waktu (*time series forecasting*) dan algoritma LightGBM. Kajian literatur bertujuan untuk memperkuat dasar teoretis terkait pemodelan prediktif pada industri perhotelan, khususnya dalam memahami bagaimana fluktuasi harga kamar dapat dipetakan berdasarkan data historis pemesanan.

b. Pengumpulan dan Pemahaman Data

Data yang digunakan dalam penelitian ini diperoleh dari basis data PT Genius Utama Indonesia (Genius Hospitality Management). Proses

pengumpulan data difokuskan pada transaksi pemesanan kamar yang berasal secara eksklusif dari saluran Booking.com. Pembatasan saluran ini dilakukan karena data dari sumber lain terindikasi memiliki inkonsistensi serta banyak nilai yang kosong. Kumpulan data awal mencakup sejumlah variabel utama, di antaranya identitas properti (*property_id*), identitas jenis kamar (*room_id*), jumlah orang dewasa (*num_adult*), jumlah anak (*num_child*), harga (*price*), dan waktu pemesanan (*booking_time*). Tahap pemahaman data difokuskan pada pengenalan struktur data historis untuk merancang alur prapemrosesan yang paling sesuai.

c. *Pre-processing Data*

Data mentah yang terkumpul perlu disiapkan agar kompatibel dengan kebutuhan pemodelan deret waktu. Pemrosesan awal mencakup transformasi kolom waktu pemesanan menjadi format penanggalan standar, yang selanjutnya diagregasi ke dalam skala bulanan. Pada tahap agregasi ini, rata-rata harga kamar dihitung untuk setiap bulan, sementara jumlah tamu dan total pemesanan diakumulasikan.

Mengingat sifat data transaksi perhotelan yang sering kali memiliki celah waktu (tidak ada pemesanan pada bulan tertentu), dilakukan pembentukan pola grid sempurna. Pola ini memastikan seluruh kombinasi properti dan jenis kamar memiliki rekaman data yang berkesinambungan setiap bulannya. Nilai harga yang kosong pada bulan tanpa transaksi diatasi menggunakan metode pengisian ke depan (*forward fill*) dengan asumsi harga cenderung stabil, dan dilanjutkan dengan pengisian ke belakang (*backward fill*) bila diperlukan. Variabel yang berkaitan dengan jumlah tamu dan pemesanan diisi dengan nilai nol, mengindikasikan ketiadaan aktivitas pada periode tersebut.

d. *Rekayasa Fitur (Feature Engineering)*

Tahap rekayasa fitur diimplementasikan untuk mengekstrak informasi temporal yang dapat dikenali oleh model. Konsep utama yang diangkat adalah pembentukan variabel jeda (*lag features*) dan rata-rata bergerak (*rolling mean*). Variabel komponen waktu diekstrak menjadi informasi tahun, bulan, dan kuartal untuk menangkap pola musiman.

Secara konseptual, harga kamar pada masa depan sangat dipengaruhi oleh tren harga sebelumnya. Oleh sebab itu, dibentuk variabel jeda harga untuk satu bulan (*lag_1*), dua bulan (*lag_2*), hingga dua belas bulan sebelumnya (*lag_12*). Selain itu, rata-rata pergerakan harga selama 3 bulan dan 6 bulan terakhir dihitung untuk meredam volatilitas data bulanan dan menangkap tren jangka menengah. Variabel tingkat hunian bulan sebelumnya (*occ_lag_1*) juga disertakan karena terdapat kemungkinan bahwa kepadatan pemesanan pada masa lalu memengaruhi penyesuaian harga di bulan berikutnya. Seluruh variabel kategorikal, seperti identitas properti dan kamar, dikonversi menjadi format numerik menggunakan teknik *Label Encoding*.

e. Pemodelan dengan LightGBM

Data yang telah diperkaya dengan fitur-fitur baru selanjutnya dibagi menjadi data latih dan data uji. Validasi model dilakukan dengan menyisihkan data 12 bulan terakhir sebagai data uji. Proses pelatihan model menggunakan algoritma *Light Gradient Boosting Machine* (LightGBM). Pendekatan ini dipilih karena LightGBM memiliki kecepatan komputasi yang tinggi serta kapabilitas bawaan dalam menangani variabel kategorikal yang kompleks pada data tabular. Parameter model seperti kedalaman pohon (*max_depth*) dan laju pembelajaran (*learning_rate*) dikonfigurasi agar algoritma mampu meminimalkan galat tanpa mengalami kondisi terlalu pas (*overfitting*) pada data latih.

f. Evaluasi Model

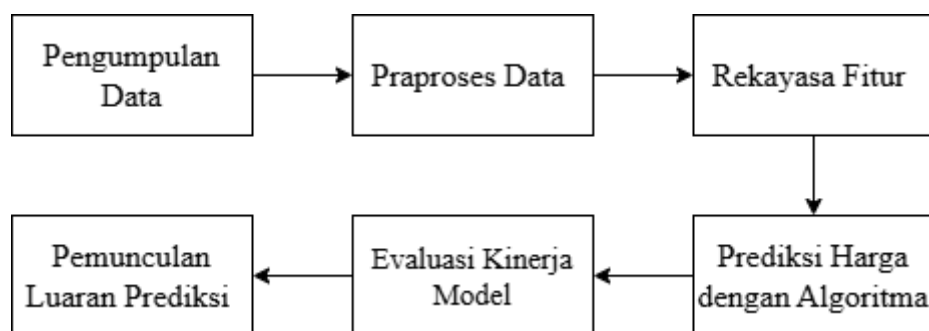
Tahap akhir merupakan proses pengujian kinerja model dalam memprediksi rata-rata harga kamar menggunakan data uji. Evaluasi tersebut diukur melalui metrik statistik standar pada pemodelan regresi, yakni *Mean Absolute Percentage Error* (MAPE) dan *Root Mean Squared Error* (RMSE). Nilai yang diperoleh dari kedua metrik ini berfungsi sebagai tolok ukur untuk mengevaluasi tingkat akurasi hasil prediksi harga dari sistem terhadap rata-rata harga aktual di lapangan.

4.4 Perancangan Sistem

Sistem yang dirancang pada penelitian ini bertujuan untuk memprediksi rata-rata harga kamar per bulan untuk setiap tipe kamar di berbagai hotel di bawah naungan PT Genius Utama Indonesia. Pendekatan ini memanfaatkan data historis reservasi pelanggan yang difilter khusus dari saluran Booking.com guna meminimalisasi inkonsistensi data. Sistem bekerja dengan cara mengumpulkan data pemesanan mentah, melakukan agregasi dan praproses, mengekstrak fitur waktu dan riwayat masa lalu (*lag*), lalu melatih algoritma prediktif untuk menghasilkan nilai estimasi harga di masa depan.

4.4.1 Alur Kerja Sistem

Untuk memberikan gambaran operasional sistem secara visual, tahapan kerja disusun secara berurutan mulai dari penerimaan data mentah hingga pemunculan hasil akhir.



Gambar 4.2 Diagram Alur Kerja Sistem

Penjelasan dari masing-masing tahapan pada diagram alur di atas dijabarkan sebagai berikut:

a. Pengumpulan Data

Masukan Tahap awal melibatkan ekstraksi data pemesanan yang memuat atribut penting seperti identitas properti (*property_id*), identitas kamar (*room_id*), status, waktu kedatangan, jumlah tamu dewasa, jumlah anak, serta harga kamar. Kualitas dan kelengkapan data pada tahap ini sangat menentukan kemampuan model dalam mengenali pola harga.

b. Praproses Data

Data mentah yang masuk selanjutnya diproses untuk mengatasi anomali dan format yang tidak seragam. Langkah ini meliputi agregasi data ke dalam format bulanan untuk menghitung rata-rata harga dan total pemesanan. Kekosongan data harga ditangani menggunakan metode pengisian nilai ke depan (*forward fill*) dan ke belakang (*backward fill*) untuk mempertahankan kontinuitas deret waktu, sedangkan kekosongan pada data jumlah tamu diisi dengan nilai nol.

c. Rekayasa Fitur

Peningkatan kualitas data latih dilakukan melalui pembentukan fitur prediktor baru. Atribut kalender seperti tahun, bulan, dan kuartal diekstraksi dari tanggal pemesanan. Selain itu, fitur tertunda (*lag*) dan rata-rata bergerak (*rolling mean*) diterapkan pada variabel harga dan jumlah pemesanan guna menangkap pola tren serta siklus tingkat hunian masa lalu.

d. Prediksi Harga dengan Algoritma

Data yang telah diproses kemudian dianalisis menggunakan algoritma *machine learning*. Sistem menggunakan parameter yang telah dioptimalkan untuk memproses data tabular, mengenali pola keterkaitan antara riwayat pemesanan masa lalu, dan meramalkan harga kamar untuk periode selanjutnya.

e. Evaluasi Kinerja Model

Sistem mengukur tingkat penyimpangan prediksi dengan membandingkan harga estimasi terhadap harga aktual pada periode data uji. Hasil evaluasi ini menentukan seberapa layak model tersebut diimplementasikan pada lingkungan produksi.

f. Pemunculan Luaran Prediksi

Tahapan akhir sistem adalah menghasilkan proyeksi harga rata-rata kamar. Nilai tersebut dapat divisualisasikan langsung pada antarmuka pengguna atau disimpan sebagai basis data rujukan bagi manajemen hotel dalam menentukan strategi harga.

4.4.2 Perancangan Antarmuka

Perancangan antarmuka pengguna (*user interface*) dilakukan untuk memfasilitasi interaksi antara manajemen PT Genius Utama Indonesia dan sistem prediktif. Desain dibuat lugas dan informatif agar pengguna dapat memantau proyeksi harga tanpa memerlukan pemahaman teknis mendalam mengenai komputasi di balik layar.

1. Tampilan Halaman Utama

Halaman awal berfungsi sebagai pusat kendali navigasi. Pada antarmuka ini, pengguna disajikan formulir masukan yang terdiri atas pilihan nama hotel dan jenis kamar yang tersedia. Pengguna cukup memilih properti spesifik melalui menu tarik-turun (*dropdown*) dan menekan tombol eksekusi untuk memulai proses prakiraan harga.

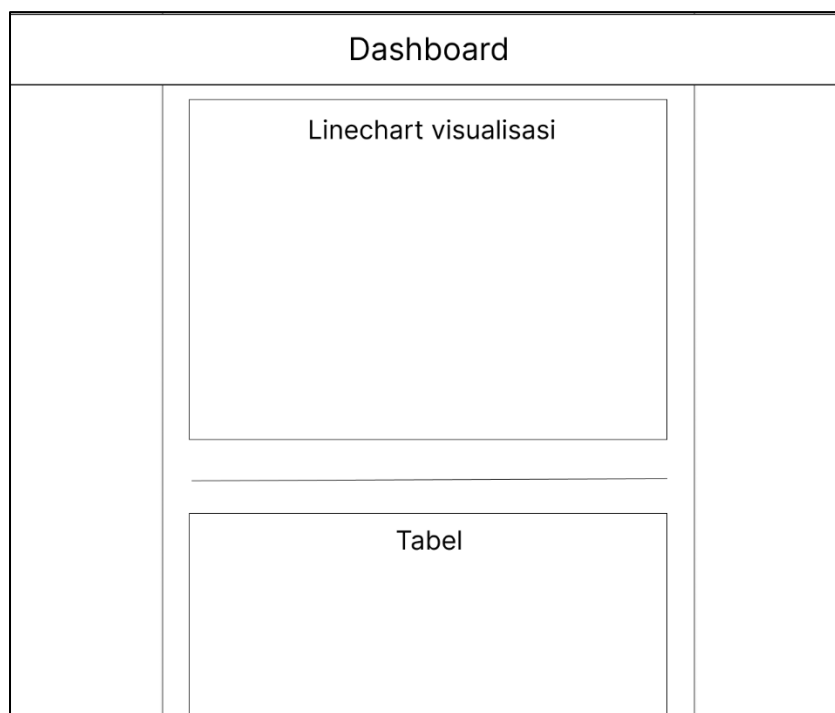
The wireframe illustrates the layout of the 'Halaman Utama' (Main Page). It features a central form titled 'Prediksi Harga Hotel'. This form contains two dropdown menus: 'Nama Hotel' and 'Jenis Kamar', each with a downward-pointing triangle icon. Below these inputs is a button labeled 'Prediksi Harga'. The entire form is centered within a larger rectangular frame representing the page layout.

Gambar 4.3 *Wireframe* Halaman Utama

2. Halaman Dasbor Hasil Analisis

Setelah sistem memproses permintaan, antarmuka akan beralih ke halaman hasil analisis. Bagian atas halaman menampilkan grafik garis (*line chart*) yang menyandingkan fluktuasi harga aktual di masa lalu dengan garis

proyeksi harga di masa depan. Pada bagian bawah grafik, disajikan tabel ringkasan yang merinci estimasi harga kamar per bulan secara numerik. Hal ini memberikan referensi ganda bagi pembuat keputusan operasional.



Gambar 4.4 *Wireframe* Dasbor Hasil Analisis

4.5 Perancangan Model

Fokus utama perancangan model difokuskan pada pemanfaatan algoritma *Light Gradient Boosting Machine* (LightGBM) untuk menangani data deret waktu yang bersifat multivariat. Tahapan spesifik perancangan model meliputi proses seleksi algoritma, skenario pembagian data, serta penentuan indikator keberhasilan.

4.5.1 Pemilihan Model

LightGBM diterapkan sebagai algoritma utama karena efisiensinya dalam memproses data tabular berskala besar, seperti catatan reservasi ribuan tamu. Metode ini mengimplementasikan teknik pertumbuhan pohon berbasis daun (*leaf-wise tree growth*), yang terbukti mempercepat komputasi pelatihan sekaligus menekan angka kesalahan pada data yang memiliki kategori berdimensi tinggi, seperti gabungan `property_id` dan `room_id`.

4.5.2 Skenario Pembagian Data

Data historis dipisahkan menjadi kelompok data latih dan data uji untuk memvalidasi kemampuan generalisasi model. Pada penelitian ini, observasi pada dua belas bulan terakhir disisihkan secara khusus sebagai data uji. Strategi pemotongan berbasis waktu (*time-based split*) ini penting diterapkan pada kasus deret waktu agar model tidak secara tidak sengaja mempelajari informasi masa depan saat fase pelatihan berlangsung.

4.5.3 Konfigurasi Parameter Pelatihan

Sejumlah pengaturan hiperparameter ditetapkan untuk memandu proses belajar algoritma. Konfigurasi mencakup penggunaan fungsi objektif *squared error* untuk kasus regresi, serta penyesuaian parameter seperti laju pembelajaran (*learning rate*), kedalaman maksimum pohon (*max depth*), dan jumlah estimator. Nilai-nilai ini dikalibrasi untuk menjaga keseimbangan antara performa deteksi pola dan pencegahan *overfitting*.

4.5.4 Evaluasi Model

Keandalan model divalidasi menggunakan metrik performa khusus regresi:

- a. *Root Mean Squared Error* (RMSE): Digunakan untuk memberikan penalti yang lebih tegas terhadap prediksi yang terlampaui jauh dari data aktual, memastikan model tetap stabil meskipun terjadi lonjakan harga dadakan akibat musim liburan.
- b. *Mean Absolute Percentage Error* (MAPE): Diterapkan untuk mengukur rata-rata persentase galat absolut antara harga hasil prediksi dan harga aktual. Metrik ini memberikan gambaran tingkat kesalahan prediktif dalam bentuk persentase relatif, sehingga memudahkan interpretasi akurasi model terlepas dari perbedaan skala atau nominal harga kamar yang dievaluasi.

Tahapan perancangan sistem dan model telah dirumuskan secara sistematis untuk mengakomodasi kebutuhan prediksi harga kamar hotel yang kompleks. Dengan mengintegrasikan rekayasa fitur khusus deret waktu dan penerapan

algoritma LightGBM, sistem diharapkan mampu memberikan landasan data yang solid bagi strategi operasional manajemen.

BAB V

IMPLEMENTASI DAN PEMBAHASAN

5.1 Implementasi

Implementasi pada penelitian ini difokuskan pada pembangunan model peramalan rata-rata harga kamar per bulan untuk setiap kombinasi *property_id* dan *room_id* pada lingkungan Genius Hospitality Management. Data yang digunakan hanya berasal dari *channel* Booking.com karena kanal lain memiliki pola yang tidak konsisten dan cenderung menyisakan banyak nilai kosong. Dalam konteks ini, *property_id* digunakan sebagai identitas hotel, sedangkan *room_id* digunakan sebagai identitas tipe kamar di dalam hotel tersebut. Dengan demikian, model tidak hanya mempelajari pergerakan harga pada level hotel, tetapi juga pada level tipe kamar yang lebih spesifik.

Tahap implementasi dilakukan mulai dari penyiapan data mentah, pembentukan data bulanan, penyusunan fitur berbasis waktu dan riwayat harga, hingga pelatihan model LightGBM untuk melakukan *forecasting* secara langsung pada dua belas horizon ke depan. Karena beberapa *room_id* memiliki riwayat data 2026 yang belum lengkap, proses implementasi juga menambahkan mekanisme penanganan data hilang dan penggunaan *anchor* cadangan agar model tetap dapat digunakan pada *room* yang historinya tidak sepenuhnya utuh.

5.1.1 Penyiapan Data dan Agregasi Bulanan

Tahap awal implementasi adalah menyiapkan data transaksi pemesanan yang masih berbentuk data mentah. Kolom *booking_time* diubah ke format tanggal agar dapat digunakan pada proses pengelompokan berdasarkan bulan. Setelah itu, data difilter hanya pada *channel* booking.com. Pemilihan *channel* ini dilakukan karena pola datanya lebih stabil dibandingkan *channel* lain yang banyak memiliki nilai kosong atau tidak konsisten.

Setelah data dipilih, proses berikutnya adalah agregasi ke level bulanan untuk setiap *property_id* dan *room_id*. Pada tahap ini, harga dihitung sebagai rata-rata bulanan, sedangkan jumlah tamu dewasa, anak, dan jumlah *booking*

dijumlahkan. Hasil agregasi ini kemudian digunakan sebagai dasar pembentukan data analitik yang akan dipakai dalam model.

Kode inti yang digunakan pada tahap ini adalah sebagai berikut.

```
import pandas as pd
import numpy as np

df = pd.read_csv("data/raw/beds24_bookings_2023_2028.csv")

df["booking_time"] = pd.to_datetime(df["booking_time"],
errors="coerce")
df["property_id"] = df["property_id"].astype(str)
df["room_id"] = df["room_id"].astype(str)
df["channel"] =
df["channel"].fillna("Unknown").astype(str)

df = df[df["channel"] == "booking.com"].copy()
df["month_date"] =
df["booking_time"].dt.to_period("M").dt.to_timestamp()

agg_funcs = {
    "price": "mean",
    "num_adult": "sum",
    "num_child": "sum",
    "booking_id": "count"
}

monthly_df = df.groupby(["property_id", "room_id",
"month_date"]).agg(agg_funcs).reset_index()
monthly_df = monthly_df.rename(columns={"booking_id":
"booking_count"})
```

5.1.2 Pembentukan *Grid* Bulanan dan Pembersihan Data

Karena beberapa *room_id* tidak memiliki data pada bulan tertentu, dibuat *grid* bulanan penuh agar urutan waktu tetap konsisten. Langkah ini penting untuk

forecasting berbasis riwayat, sebab model membutuhkan urutan waktu yang seragam agar fitur *lag* dapat dibentuk dengan benar. Setelah *grid* dibentuk, nilai harga yang hilang diisi menggunakan *forward fill* dan *backward fill* pada level *property_id* dan *room_id*. Sementara itu, variabel jumlah tamu dan *booking_count* diisi dengan nol karena tidak adanya transaksi pada bulan tertentu memang berarti tidak ada aktivitas pencatatan.

Tahap ini juga membantu menjaga konsistensi data pada kamar yang riwayatnya belum lengkap, terutama untuk 2026 yang masih belum utuh pada beberapa *room_id*. Dengan cara ini, model tetap dapat menerima *input* yang stabil tanpa mengabaikan kamar yang memiliki data parsial.

Kode inti yang digunakan adalah sebagai berikut.

```
all_months = pd.date_range(monthly_df["month_date"].min(),
monthly_df["month_date"].max(), freq="MS")

idx = pd.MultiIndex.from_product(
    [monthly_df["property_id"].unique(),
monthly_df["room_id"].unique(), all_months],
    names=["property_id", "room_id", "month_date"]
)

full_df = monthly_df.set_index(["property_id", "room_id",
"month_date"]).reindex(idx).reset_index()

full_df["price"] = full_df.groupby(["property_id",
"room_id"])["price"].transform(lambda x:
x.ffill().bfill())
full_df[["num_adult", "num_child", "booking_count"]] =
full_df[["num_adult", "num_child",
"booking_count"]].fillna(0)

full_df =
full_df.dropna(subset=["price"]).reset_index(drop=True)
```

```
full_df["time_idx"] = (full_df["month_date"].dt.year * 12
+ full_df["month_date"].dt.month)
full_df["time_idx"] -= full_df["time_idx"].min()
```

5.1.3 Pembentukan Fitur

Setelah data bulanan siap, tahap berikutnya adalah membentuk fitur yang digunakan untuk pelatihan model. Fitur yang dibangun tidak hanya berisi informasi kalender seperti tahun, bulan, dan kuartal, tetapi juga fitur riwayat harga berbasis *lag* serta fitur interaksi yang lebih sesuai untuk peramalan. Pada implementasi akhir, pendekatan yang digunakan adalah *direct forecasting*, yaitu setiap horizon dipelajari oleh model tersendiri.

Untuk menjaga akurasi pada kondisi data yang tidak lengkap, digunakan *anchor* harga 12 bulan sebelumnya sebagai basis utama prediksi. Jika *anchor* 12 bulan tidak tersedia, sistem menggunakan *fallback* ke 24 bulan atau 36 bulan sebelumnya. Mekanisme ini penting karena beberapa *room_id* di 2026 belum memiliki urutan data yang utuh. Dengan demikian, model tetap memiliki referensi harga yang masuk akal saat menghitung prediksi.

Secara umum, target model dibentuk sebagai selisih logaritmik antara harga saat ini dan harga acuan. Dengan formulasi ini, model belajar memprediksi perubahan harga relatif terhadap kondisi historis, bukan harga absolut secara langsung.

Kode inti yang digunakan pada tahap *feature engineering* adalah sebagai berikut.

```
def advanced_hybrid_fe_direct(data, h=1):
    df = data.copy()
    df = df.sort_values(["roomId",
"month_period"]).reset_index(drop=True)

    df["date"] =
pd.to_datetime(df["month_period"].astype(str))
    df["month"] = df["date"].dt.month
    df["quarter"] = df["date"].dt.quarter
```

```

df["year"] = df["date"].dt.year

season_map = {1: 0, 2: 0, 11: 0, 3: 1, 4: 1, 5: 1, 6:
1, 10: 1, 7: 2, 8: 2, 9: 2, 12: 3}
df["season_strength"] = df["month"].map(season_map)
df["is_peak_season"] = df["month"].isin([7, 8,
12]).astype(int)
df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

base_lag = h
lags_to_create = list(set([base_lag, base_lag + 1,
base_lag + 2, base_lag + 5, base_lag + 11, base_lag + 12,
12, 24, 36]))

for lag in lags_to_create:
    df[f"price_lag_{lag}"] =
df.groupby("roomId")["price"].shift(lag)

df["dynamic_anchor_price"] = df["price_lag_12"]
df["anchor_age_months"] = 12

mask_24 = df["dynamic_anchor_price"].isna() &
df["price_lag_24"].notna()
df.loc[mask_24, "dynamic_anchor_price"] =
df.loc[mask_24, "price_lag_24"]
df.loc[mask_24, "anchor_age_months"] = 24

mask_36 = df["dynamic_anchor_price"].isna() &
df["price_lag_36"].notna()
df.loc[mask_36, "dynamic_anchor_price"] =
df.loc[mask_36, "price_lag_36"]
df.loc[mask_36, "anchor_age_months"] = 36

```

```

    mask_base = df["dynamic_anchor_price"].isna() &
df[f"price_lag_{base_lag}"].notna()
    df.loc[mask_base, "dynamic_anchor_price"] =
df.loc[mask_base, f"price_lag_{base_lag}"]
    df.loc[mask_base, "anchor_age_months"] = base_lag

    df["is_fallback_anchor"] = (df["anchor_age_months"] !=
12).astype(int)

    windows = [3, 6, 12]
    for w in windows:
        df[f"price_roll_mean_{w}"] =
df.groupby("roomId")[f"price_lag_{base_lag}"].transform(la
mbda x: x.rolling(w, min_periods=1).mean())
        df[f"price_roll_std_{w}"] =
df.groupby("roomId")[f"price_lag_{base_lag}"].transform(la
mbda x: x.rolling(w, min_periods=2).std())

    eps = 1e-5
    df["price_diff_1"] = df[f"price_lag_{base_lag}"] -
df[f"price_lag_{base_lag + 1}"]
    df["price_growth_1"] = df["price_diff_1"] /
(df[f"price_lag_{base_lag + 1}"] + eps)
    df["price_base_vs_anchor"] =
df[f"price_lag_{base_lag}"] / (df["dynamic_anchor_price"]
+ eps)

    return df

```

5.1.4 Pelatihan Model LightGBM

Pelatihan model dilakukan menggunakan LightGBM karena karakteristiknya cukup sesuai untuk data *tabular* dengan banyak fitur numerik dan *kategorikal*. Pada implementasi ini, model dilatih secara terpisah untuk dua belas horizon, sehingga setiap horizon memiliki parameter terbaiknya sendiri.

Pendekatan ini dipilih agar model tidak bergantung pada prediksi bulan sebelumnya. Dengan begitu, kesalahan pada satu horizon tidak langsung merambat ke horizon berikutnya.

Target yang digunakan adalah log *difference* berbasis harga sekarang dan harga acuan 12 bulan sebelumnya. Selama proses *training*, fitur *kategorikal* seperti *roomId*, *propertyId*, dan informasi konfigurasi kamar diubah ke tipe *category* agar dapat diproses oleh LightGBM. Parameter model dicari menggunakan Optuna, lalu model terbaik disimpan untuk dipakai kembali pada tahap *retraining* produksi.

Kode inti yang digunakan pada pelatihan adalah sebagai berikut.

```
import gc
import lightgbm as lgb
import optuna
from sklearn import metrics
from sklearn.model_selection import TimeSeriesSplit

optuna.logging.set_verbosity(optuna.logging.WARNING)

data = data.sort_values(["roomId",
"month_period"]).reset_index(drop=True)
data["price"] = data["price"].replace(0, np.nan)
data["price"] =
data.groupby("roomId")["price"].transform(lambda x:
x.fffll().bfill())

if pd.api.types.is_period_dtype(data["month_period"]):
    data["date"] = data["month_period"].dt.to_timestamp()
else:
    data["date"] = pd.to_datetime(data["month_period"])

data["year"] = data["date"].dt.year
data["month"] = data["date"].dt.month

overall_holdout_mape = []
```

```

overall_holdout_mae = []
best_params_by_h = {}
trained_models = {}

def safe_mape(y_true, y_pred, eps=1e-6):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    denom = np.where(np.abs(y_true) < eps, eps,
np.abs(y_true))
    return np.mean(np.abs(y_true - y_pred) / denom)

all_holdout_results = []

for h in range(1, 13):
    step_data = advanced_hybrid_fe_direct(data, h=h)
    anchor_col = "dynamic_anchor_price"

    if anchor_col not in step_data.columns:
        step_data[anchor_col] =
step_data.groupby("roomId")["price"].shift(12)

    step_data = step_data.dropna(subset=[anchor_col,
"price"]).reset_index(drop=True)

    train_data = step_data[step_data["date"] < "2025-01-
01"].copy()
    test_data = step_data[step_data["date"] >= "2025-01-
01"].copy()

    if len(train_data) < 60:
        continue

    y_train = np.log1p(train_data["price"]) -
np.log1p(train_data[anchor_col])

```



```

cols_to_drop = [
    "price", "log_price", "target_diff", "target_pct",
    "date", "month_period", f"log_price_lag_{h}",
    "log_price_lag_12", "id"
]

x_train_full = train_data.drop(columns=[c for c in
cols_to_drop if c in train_data.columns], errors="ignore")
x_test_full = test_data.drop(columns=[c for c in
cols_to_drop if c in test_data.columns], errors="ignore")

cat_cols = [
    "roomId", "propertyId", "room_cfg_room_type",
    "room_cfg_restriction_strategy",
    "room_cfg_overbooking_protection",
    "room_cfg_unit_allocation", "room_cfg_tier"
]

for c in cat_cols:
    if c in x_train_full.columns:
        x_train_full[c] =
x_train_full[c].astype("category")
    if c in x_test_full.columns:
        x_test_full[c] =
x_test_full[c].astype("category")

x_train_full =
x_train_full.select_dtypes(include=["number", "category",
"bool"]).copy()
x_test_full =
x_test_full.reindex(columns=x_train_full.columns).copy()

```

```

cat_cols_present = [c for c in cat_cols if c in
x_train_full.columns]
for c in cat_cols_present:
    x_test_full[c] = pd.Categorical(x_test_full[c],
categories=x_train_full[c].cat.categories)

n_splits = 4 if len(train_data) >= 180 else 3 if
len(train_data) >= 100 else 2
tscv = TimeSeriesSplit(n_splits=n_splits)

def objective(trial):
    num_boost_round =
trial.suggest_int("num_boost_round", 100, 800)
    params = {
        "objective": "regression",
        "metric": "rmse",
        "boosting_type": "gbdt",
        "verbosity": -1,
        "random_state": 222,
        "linear_tree": True,
        "num_leaves": trial.suggest_int("num_leaves",
10, 40),
        "max_depth": trial.suggest_int("max_depth", 3,
8),
        "min_data_in_leaf":
trial.suggest_int("min_data_in_leaf", 5, 30),
        "learning_rate":
trial.suggest_float("learning_rate", 0.01, 0.1, log=True),
        "feature_fraction":
trial.suggest_float("feature_fraction", 0.6, 1.0),
        "bagging_fraction":
trial.suggest_float("bagging_fraction", 0.6, 1.0),
        "bagging_freq":
trial.suggest_int("bagging_freq", 1, 10),

```

```

        "min_child_samples":
trial.suggest_int("min_child_samples", 20, 150),
        "lambda_l1": trial.suggest_float("lambda_l1",
1e-6, 20.0, log=True),
        "lambda_l2": trial.suggest_float("lambda_l2",
1e-6, 20.0, log=True),
        "max_bin": trial.suggest_int("max_bin", 63,
255),

        "extra_trees":
trial.suggest_categorical("extra_trees", [False, True]),
    }

    fold_mae = []

    for train_idx, valid_idx in
tscv.split(x_train_full):
        X_trn = x_train_full.iloc[train_idx]
        X_val = x_train_full.iloc[valid_idx]
        y_trn = y_train.iloc[train_idx]
        y_val = y_train.iloc[valid_idx]

        dtrain = lgb.Dataset(X_trn, label=y_trn,
categorical_feature=cat_cols_present, free_raw_data=False)
        dvalid = lgb.Dataset(X_val, label=y_val,
categorical_feature=cat_cols_present, reference=dtrain,
free_raw_data=False)

        model = lgb.train(
            params=params,
            train_set=dtrain,
            num_boost_round=num_boost_round,
            valid_sets=[dvalid],

```

```

callbacks=[lgb.early_stopping(stopping_rounds=80,
verbose=False)]

    )

    pred_val_diff = model.predict(X_val,
num_iteration=model.best_iteration)

    base_val =
train_data[anchor_col].iloc[valid_idx].values
    pred_val_price = np.expml(np.loglp(base_val) +
pred_val_diff)

    true_val_price =
train_data["price"].iloc[valid_idx].values

fold_mae.append(metrics.mean_absolute_error(true_val_price
, pred_val_price))

    return float(np.mean(fold_mae))

study = optuna.create_study(
    direction="minimize",
    sampler=optuna.samplers.TPESampler(seed=222),
pruner=optuna.pruners.MedianPruner(n_warmup_steps=1),
    )
    study.optimize(objective, n_trials=50,
show_progress_bar=False)

best_params = study.best_params.copy()
best_rounds = int(best_params.pop("num_boost_round"))
best_params.update({
    "objective": "regression",
    "metric": "rmse",

```

```

        "boosting_type": "gbdt",
        "verbosity": -1,
        "random_state": 222,
    })

    dtrain_full = lgb.Dataset(
        x_train_full,
        label=y_train,
        categorical_feature=cat_cols_present,
        free_raw_data=False
    )

    final_model = lgb.train(
        params=best_params,
        train_set=dtrain_full,
        num_boost_round=best_rounds
    )

    best_params_by_h[h] = {"num_boost_round": best_rounds,
**best_params}
    trained_models[h] = final_model

    pred_train_diff = final_model.predict(x_train_full)
    final_train_predictions =
np.expm1(np.log1p(train_data[anchor_col].values) +
pred_train_diff)
    train_mape = safe_mape(train_data["price"].values,
final_train_predictions)
    train_mae =
metrics.mean_absolute_error(train_data["price"].values,
final_train_predictions)

    if len(x_test_full) > 0:
        pred_test_diff = final_model.predict(x_test_full)

```

```

        final_test_predictions =
np.expm1(np.log1p(test_data[anchor_col].values) +
pred_test_diff)

        test_mape = safe_mape(test_data["price"].values,
final_test_predictions)

        test_mae =
metrics.mean_absolute_error(test_data["price"].values,
final_test_predictions)

        overall_holdout_mape.append(test_mape)
        overall_holdout_mae.append(test_mae)

        step_results = pd.DataFrame({
            "roomId": test_data["roomId"].values,
            "propertyId": test_data["propertyId"].values
if "propertyId" in test_data.columns else "Unknown",
            "horizon": h,
            "actual_price": test_data["price"].values,
            "predicted_price": final_test_predictions
        })
        all_holdout_results.append(step_results)

```

5.1.5 Implementasi *Retaining* Produksi dan Evaluasi

Pada tahap ini, model LightGBM dilatih ulang menggunakan parameter terbaik untuk masing-masing horizon prediksi 1 sampai 12 bulan. Proses *retraining* dilakukan pada data produksi agar model mengikuti pola terbaru pada data harga kamar. Untuk menjaga kualitas pembelajaran, data pada bagian akhir periode yang belum lengkap tidak dimasukkan ke proses pelatihan. Evaluasi dilakukan pada data *tail* yang dikeluarkan dari pelatihan dengan menggunakan metrik MAPE dan RMSE.

Hasil evaluasi menunjukkan bahwa model memiliki kinerja yang cukup stabil di seluruh horizon prediksi. Rata-rata MAPE sebesar 12,96% dan rata-rata RMSE sebesar Rp 154.188,97. Performa terbaik terdapat pada horizon ke-3 dengan MAPE 9,13% dan RMSE Rp 113.462,79. Sementara itu, performa terendah berdasarkan MAPE terjadi pada horizon ke-11 dengan nilai 16,05%, sedangkan RMSE tertinggi terdapat pada horizon ke-5 sebesar Rp 197.738,81.

Tabel 5.1 Hasil Evaluasi

Horizon	MAPE	RMSE
1	15,08%	Rp 179.248,41
2	12,20%	Rp 149.447,93
3	9,13%	Rp 113.462,79
4	10,42%	Rp 123.047,62
5	14,80%	Rp 197.738,81
6	15,43%	Rp 187.635,26
7	14,18%	Rp 164.325,36
8	12,12%	Rp 142.349,78
9	13,40%	Rp 143.888,62
10	10,77%	Rp 130.287,75
11	16,05%	Rp 186.767,88
12	11,96%	Rp 132.067,42

Secara keseluruhan, hasil tersebut menunjukkan bahwa model mampu memberikan prediksi yang relatif konsisten, meskipun akurasi pada beberapa horizon masih dipengaruhi oleh variasi historis harga dan kelengkapan data pada kamar tertentu.

5.2 Pembahasan

5.2.1 Pengumpulan dan *Pre-processing* Data

Data yang digunakan pada penelitian ini berasal dari transaksi pemesanan kamar pada *channel* Booking.com. Pemilihan *channel* tersebut dilakukan karena data pada *channel* lain tidak konsisten dan masih memiliki banyak kekosongan. Dengan demikian, penggunaan satu *channel* yang lebih stabil dianggap lebih

representatif untuk membangun model *forecasting* yang lebih terukur. Dataset yang digunakan memuat informasi utama berupa *property_id*, *room_id*, *status*, *arrival*, *departure*, *num_adult*, *num_child*, *price*, *channel*, *booking_time*, dan *modified_time*. Dalam konteks penelitian ini, *property_id* merepresentasikan identitas hotel, sedangkan *room_id* merepresentasikan identitas tipe kamar pada hotel tersebut.

Tahap awal *pre-processing* dilakukan dengan mengubah data transaksi menjadi bentuk agregasi bulanan. Setiap kombinasi *property_id* dan *room_id* dihitung nilai rata-rata harga kamarnya per bulan sehingga data yang semula bersifat transaksi harian menjadi data deret waktu bulanan. Bentuk agregasi ini sesuai dengan tujuan penelitian, yaitu memprediksi rata-rata harga kamar per bulan untuk setiap tipe kamar pada masing-masing hotel. Selain itu, proses agregasi ini juga membantu mengurangi *noise* yang muncul dari variasi harga antartransaksi dalam periode yang sama.

Setelah data dibentuk menjadi panel bulanan, dilakukan penyusunan kalender kontinu agar setiap kamar memiliki urutan waktu yang lengkap dari bulan pertama sampai bulan terakhir. Langkah ini penting karena data observasi aktual pada beberapa kamar tidak selalu tersedia secara penuh di setiap bulan. Dengan kalender kontinu, bulan yang kosong tetap tercatat sehingga pola waktu dapat dipelajari secara lebih konsisten oleh model. Pada tahap berikutnya, nilai harga yang kosong diisi dengan pendekatan berurutan ke depan dan ke belakang pada level kamar, sedangkan variabel kategorikal yang bersifat statis, seperti properti dan konfigurasi kamar, disesuaikan agar tetap konsisten di seluruh periode.

Selain harga, beberapa fitur tambahan juga dipertahankan untuk memperkaya informasi prediktif. Fitur waktu seperti tahun, bulan, kuartal, dan komponen musiman dipakai untuk menangkap pola musiman harga kamar. Di samping itu, variabel inflasi turut digabungkan sebagai indikator makroekonomi yang dapat memengaruhi pergerakan harga. Pendekatan ini membuat data yang digunakan tidak hanya berisi histori harga, tetapi juga konteks waktu dan kondisi eksternal yang relevan. Secara keseluruhan, hasil *pre-processing* menghasilkan

data panel bulanan yang lebih rapi, lebih konsisten, dan lebih siap digunakan untuk pelatihan model *forecasting*.

5.2.2 Pembahasan Model

Model utama yang digunakan dalam penelitian ini adalah LightGBM karena mampu menangani data tabular dengan jumlah fitur yang cukup banyak, bekerja efisien pada data berskala menengah hingga besar, serta cocok untuk pola prediksi yang dipengaruhi oleh fitur historis dan musiman. Pemodelan dilakukan secara langsung untuk horizon 1 sampai 12 bulan. Artinya, setiap horizon memiliki model tersendiri sehingga prediksi untuk bulan ke depan tidak bergantung pada prediksi bulan sebelumnya. Skema ini dipilih agar kesalahan prediksi tidak menumpuk seperti pada pendekatan *recursive forecasting*. Untuk targetnya, model mempelajari perubahan harga terhadap *anchor* historis 12 bulan sebelumnya, dengan *fallback* ke riwayat 24 bulan dan 36 bulan apabila data 12 bulan tidak tersedia. Pendekatan ini tampaknya lebih stabil untuk dataset hotel yang memiliki histori tidak seragam antar kamar.

Pada tahap produksi, model dilatih ulang menggunakan parameter terbaik yang telah diperoleh pada proses sebelumnya. *Retraining* dilakukan agar model mengikuti pola data yang lebih baru. Namun, data pada bagian akhir periode 2026 yang belum lengkap tidak dimasukkan ke proses pelatihan. Keputusan ini penting karena data yang belum utuh dapat menimbulkan bias pada model, terutama pada *room_id* yang memiliki histori pendek atau bulan-bulan yang hilang. Karena itu, evaluasi dilakukan pada *tail* data yang dikeluarkan dari pelatihan untuk melihat seberapa baik model tetap bekerja pada data yang tidak ikut dipakai saat fit ulang.

Evaluasi dilakukan menggunakan dua metrik utama, yaitu MAPE dan RMSE. MAPE digunakan untuk melihat rata-rata kesalahan dalam bentuk persentase terhadap nilai aktual, sehingga memudahkan pembacaan akurasi relatif model. Sementara itu, RMSE menunjukkan besar kesalahan absolut dengan penalti yang lebih tinggi terhadap *error* yang besar. Kombinasi dua metrik ini memberikan gambaran yang lebih seimbang, karena MAPE baik untuk membaca proporsi kesalahan, sedangkan RMSE lebih sensitif terhadap deviasi prediksi yang besar.

Hasil evaluasi produksi pada 12 horizon ditunjukkan pada tabel berikut.

Tabel 5.2 Hasil Evaluasi

Horizon	MAPE	RMSE
1	15,08%	Rp 179.248,41
2	12,20%	Rp 149.447,93
3	9,13%	Rp 113.462,79
4	10,42%	Rp 123.047,62
5	14,80%	Rp 197.738,81
6	15,43%	Rp 187.635,26
7	14,18%	Rp 164.325,36
8	12,12%	Rp 142.349,78
9	13,40%	Rp 143.888,62
10	10,77%	Rp 130.287,75
11	16,05%	Rp 186.767,88
12	11,96%	Rp 132.067,42

Berdasarkan hasil tersebut, rata-rata MAPE seluruh horizon sebesar 12,96 persen, sedangkan rata-rata RMSE sebesar Rp 154.188,97. Hasil ini menunjukkan bahwa model memiliki performa yang cukup konsisten di sepanjang horizon prediksi. Performa terbaik terlihat pada horizon ke 3 dengan MAPE 9,13 persen dan RMSE Rp 113.462,79. Sebaliknya, performa terendah berdasarkan MAPE terjadi pada horizon ke 11, sedangkan RMSE tertinggi terdapat pada horizon ke 5. Pola ini menunjukkan bahwa beberapa horizon masih lebih sulit diprediksi, kemungkinan karena variasi harga yang lebih tajam atau kelengkapan histori kamar yang lebih terbatas.

Jika dilihat secara umum, model cenderung lebih stabil pada horizon menengah, yaitu sekitar bulan ke 3 sampai bulan ke 4 dan bulan ke 8 sampai bulan ke 10. Pada beberapa *room_id*, kesalahan yang lebih besar tampaknya berkaitan dengan histori data yang pendek, harga yang sangat fluktuatif, atau bulan yang hilang pada periode terakhir. Dengan demikian, meskipun model sudah mampu menghasilkan prediksi yang relatif baik, hasilnya masih perlu dibaca dengan mempertimbangkan kualitas histori masing-masing kamar dan properti.

Pembahasan pada level kamar dan properti juga penting karena tujuan penelitian ini memang memprediksi harga pada kombinasi *property_id* dan *room_id*, bukan hanya rata-rata global. Hasil evaluasi granular menunjukkan bahwa model dapat membentuk prediksi yang cukup baik pada kamar dengan histori data yang lebih lengkap. Pada kamar yang datanya tidak merata, kesalahan prediksi cenderung meningkat. Temuan ini memperkuat kesimpulan bahwa kualitas histori data sangat memengaruhi hasil *forecasting*, terutama pada deret waktu dengan banyak *missing period* dan variasi musiman yang kuat.

Secara keseluruhan, hasil pembahasan model menunjukkan bahwa LightGBM layak digunakan untuk *forecasting* rata-rata harga kamar per bulan pada masing-masing properti dan tipe kamar. Model sudah mampu menangkap pola historis, pola musiman, dan pengaruh beberapa fitur tambahan yang relevan. Meski demikian, hasil evaluasi juga memperlihatkan bahwa model masih sensitif terhadap kelengkapan data dan kestabilan harga pada kamar tertentu. Oleh karena itu, pada pengembangan selanjutnya, model masih dapat dioptimalkan kembali melalui penyempurnaan fitur, penyesuaian skema validasi, atau penambahan data historis yang lebih lengkap.

DAFTAR PUSTAKA

- Andy Hermawan, Aji Saputra, Nabila Lailinajma, Reska Julianti, Timothy Hartanto, & Troy Kornelius Daniel. (2025). Predicting Hotel Booking Cancellations Using Machine Learning for Revenue Optimization. *Router : Jurnal Teknik Informatika dan Terapan*, 3(1), 37–48.
<https://doi.org/10.62951/router.v3i1.400>
- Aulady, Moch. A., Hudawi AS, A., & Arifin, Z. (2024). Improve Metode Lightgbm untuk Prediksi Harga Mobil Bekas Menggunakan Hyper-Parameter Tuning. *TRILOGI: Jurnal Ilmu Teknologi, Kesehatan, dan Humaniora*, 5(3). <https://doi.org/10.33650/trilogi.v5i3.9000>
- Budi Darma, P. S., & Wirawati, N. G. P. (2019). Pengaruh Pendapatan Kamar dan Biaya Komisi Penjualan Terhadap Average Room Rate Hotel di Daerah Denpasar. *E-Jurnal Akuntansi*, 2129.
<https://doi.org/10.24843/eja.2019.v27.i03.p18>
- Gonçalves, V. de S. F., & Carvalho, V. R. de. (2025). A Review of Interpretability Methods for Gradient Boosting Decision Trees. Dalam *Journal of the Brazilian Computer Society* (Vol. 31, Nomor 1, hlm. 640–654). Brazilian Computing Society. <https://doi.org/10.5753/jbcs.2025.5324>
- Gusti Ayu Riyani Astarani, I., Gede Surya Rahayuda, I., Studi Informatika, P., Matematika dan Ilmu Pengetahuan Alam, F., Raya Kampus Udayana, J., Jimbaran, B., & Selatan, K. (2025). Analisis Perbandingan XGBoost dan LightGBM dalam Prediksi Penjualan Ritel Walmart Store Sales. *JNATIA*, 3(4).
- Indah, Y. M., Aristawidya, R., Fitrianto, A., Erfiani, E., & Jumansyah, L. M. R. D. (2025). Comparison of Random Forest, XGBoost, and LightGBM Methods for the Human Development Index Classification. *Jambura Journal of Mathematics*, 7(1), 14–18. <https://doi.org/10.37905/jjom.v7i1.28290>
- Lima Santos, L., Gomes, C., Malheiros, C., Crespo, C., & Bento, C. (2024). Factors Influencing Hotel Revenue Management in Times of Crisis: Towards Financial Sustainability. *International Journal of Financial Studies*, 12(4).
<https://doi.org/10.3390/ijfs12040112>
- Montgomery, D. C., & Jennings, C. L. (2024). *Introduction to Time Series Analysis and Forecasting* (3rd ed.). John Wiley & Sons.
- Naufal Malik Herlambang, & Harun Al Azies. (2026). TIME SERIES FORECASTING SAHAM PT ASTRA MENGGUNAKAN ALGORITMA AUTOREGRESSIVE INTEGRATED MOVING AVERAGE DAN PROPHET. *Rabit : Jurnal Teknologi dan Sistem Informasi Univrab*, 11(1), 1105–1116. <https://doi.org/10.36341/rabit.v11i1.7231>
- Pramita, D. A. K., Saraswati, N. W. S., Sandana, I. P. D., Dewi, D. A. P. R., & Krismentari, N. K. B. (2026). Forecasting Hotel Demand with Time Series Prediction Model Using Random Forest Regression. *sinkron*, 10(1).
<https://doi.org/10.33395/sinkron.v10i1.15655>
- Raafi Herlando, M., Hadi Wijoyo, S., & Akbar, M. A. (2025). *ANALISIS PERBANDINGAN MACHINE LEARNING UNTUK MEMPREDIKSI*

- HARGA MINYAK DENGAN REGRESI LINEAR DAN SUPPORT VECTOR REGRESSION* (Vol. 9, Nomor 6). <http://j-ptiik.ub.ac.id>
- Rahayu Dwiastuti Pujiningrum, & Sulistijanti, W. (2024). Peramalan Tingkat Penghunian Kamar Hotel Bintang Lima Provinsi Bali Menggunakan Metode ARIMA dan Fuzzy Time Series Lee. *JURNAL SOSIAL EKONOMI DAN HUMANIORA*, 10(2), 158–166. <https://doi.org/10.29303/jseh.v10i2.520>
- Rahman, A., Yusnita, A., & Ekawati, H. (2025). *Bulletin of Information Technology (BIT) Penerapan Algoritma XGBoost Dalam Prediksi Harga Sewa Kos Di Kota Samarinda*. 6(4), 379–390. <https://doi.org/10.47065/bit.v5i2.2304>
- Sanjaya, K. D. (2024). *PREDIKSI HARGA RUMAH DENGAN METODE REGRESI LINEAR DAN SUPPORT VECTOR REGRESSION DI DAERAH TEBET JAKARTA SELATAN* (Vol. 19, Nomor 2). Versi Cetak.
- Truong, T. T. P., & Novák, V. (2023). Forecasting and Detection of Structural Breaks. Dalam O. Valenzuela, F. Rojas, L. J. Herrera, H. Pomares, & I. Rojas (Ed.), *Theory and Applications of Time Series Analysis and Forecasting: Selected Contributions from ITISE 2021*. Springer.

LAMPIRAN